

Introduction to Machine Learning

by Khallil Ebrahim Benyattou
kbenyattou.github.io

ジェネレーション
パソコン・世代

July 7, 2026

This page intentionally left almost blank.

Contents

Guide

Notation

I Foundations

Chapter 1: Introduction

- 1.1 Paradigms 9
- 1.2 Tasks and Models 9

Chapter 2: Supervised Learning

- 2.1 Probability to Model Uncertainty 13
Probabilistic Model.
- 2.2 Loss and Risk 15
Empirical Risk.

Chapter 3: Regression

- 3.1 Setup 20
- 3.2 Estimating the Regression Function 21
- 3.3 Linear Regression 22
Extra Assumptions.
- 3.4 Ordinary Least-Squares (OLS) 26
Normal Equations; Is there always a solution to the normal equations?.
- 3.5 Iterative Methods 31
Gradient Descent; Stochastic Gradient Descent.
- 3.6 Normality Assumptions (Ordinary Linear Regression) 33
Consequences of Joint Conditional Normality of Our Errors; Maximum Likelihood Estimation.

Chapter 4: Classification

- 4.1 Linear Classifiers 39
Aggregation/Scoring T ; Threshold f .
- 4.2 The Deceptron 41
Setup; Surrogate Deceptron Problem; Classical Deceptron.
- 4.3 Logistic Regression 50
The Model.
- 4.4 Newton's Method 53
Newton-Raphson.

Chapter 5: Support Vector Machines

- 5.1 The Plight of Linearly Inseparable Data 55
- 5.2 Separable Case (Optimal Margin Classifier) 58
Margins; Primal Setup; Dual Setup; Duality and KKT Conditions; Solving the Dual Problem.
- 5.3 Linearly Inseparable Case 68

II Deep Learning

Chapter 6: Neural Networks

- 6.1 The Limitations of Linear Classifiers 71
- 6.2 What is a Neural Network, Roughly? 72
The Single-Layer Perceptron; Logistic Regression as a Neural Network; Multi-Class Classification.

6.3 What is a Neural Network, Precisely?	75
A Single Neuron; A Single Layer; The Whole Network; Forward Batch Propagation.	
6.4 Neural Network Loss	80
6.5 Back-propagation Equations	81
Update Derivatives; Input Into Neuron.	

Appendices

Chapter A: Differentiation

A.1 Fréchet Derivative	89
Chain Rule.	
A.2 Matrix Calculus	92

Chapter B: Optimisation

B.1 First-Order Optimality Conditions (Unconstrained)	94
B.2 Second-Order Optimality Conditions	96
B.3 Upgrading Local to Global Solutions	98
B.4 Characterisations of Convex Functions	99
Convexity Along All Lines; First and Second-Order Characterisations.	
B.5 Strictly Convex Functions	100
Characterisation of Strict Convexity.	
B.6 Lagrange Multipliers	101
Equality Constraints.	

Guide

If anybody is trying to sell you these notes, ~~I am flattered~~, these notes are freely available for public access and should not, under any circumstances, be sold or distributed for monetary gain.

These notes are hosted in only one location, my website:

<https://kbenyattou.github.io/notes/>

All errors herein are my own.

References

If I'm referring to a set of notes that I've personally written, I'll put **xyz** next to the reference. The numbering system for my personal notes is:

- **100** Introduction to Python & Algorithms
Topics: Syntax and Semantics, Data Types, Variables, Assignments, Modules, Namespaces and Scope, Searching/Sorting Algorithms, Time Complexity, Classes and Object-Oriented, Iterables, List Comprehensions
- **101** Introduction to Data Science
Topics: Greedy Algorithms, Dynamic Programming, Graphs (DFS, BFS), Random Walks, k -Nearest Neighbours
- **102** Introduction to Machine Learning
Topics: Decision Theory, Supervised Learning Probability Framework, Regression, Least-Squares, Normal Equations, Classification, Linear Classifiers, Classical Perceptron, Logistic Regression, Neural Networks, Forward and Back-Propagation Equations, Support Vector Machines, Differential and Matrix Calculus
- **103** Conditional Expectations
Topics: Probability Spaces, Constructing Measures, Random Variables, Lebesgue Integration, Probability Distributions, Point Processes (Random Scatters), Moment-Generating Functions, Populations and Sampling, Estimators, Statistics, Abstract Conditional Expectation, Regular Conditional Probability, Disintegration, Mathematical Statistics (Approximation and Goodness of Estimators), Data Reduction (Sufficiency)
- **104** Linear Algebra
Topics: Linear Maps, Change of Basis, Diagonalisability, Bilinear Maps, Inner Products, Adjoints, Orthogonal Projection, Spectral Decomposition, Polar and Singular Value Decompositions, Ordinary Least-Squares

Key

- **Keywords** are in this colour.
 - **Words to look up later** are in this colour.
 - **Things** I want to
- This environment denotes something I'm uneasy about or have left as a gap to fill in later.
- **link together** are in this colour. These will (almost always) be on the *same page* and occur in pairs so the link is clear.

Notation

- $\mathbb{R}^{n,1}$ (or $\mathbb{R}^n$ when clear from context) is the set of column vectors of length n whose entries are real numbers e.g.

$$\mathbf{v} = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^{n,1}$$

- $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ is the standard ordered basis of Euclidean space, where

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \text{etc.}$$

- $A \in \mathbb{R}^{m,n}$ is an m by n matrix with real entries.

PART I

Foundations

Introduction

[The] human perception system is rife with all ways of getting it wrong. [...] All it takes is a few sketches that are cleverly done and your brain can't figure it out! So, we are poor data-taking devices. That's why we have such a thing as science. Because we have machines that don't care what side of the bed they woke up (on) in the morning. [...] They'll get the data right.

Neil deGrasse Tyson, 2009
@ Cosmic Quandaries

An **algorithm** is an unambiguous sequence of instructions designed to complete a task. For simple tasks, it's relatively easy to manufacture an appropriate (and perhaps even optimal) algorithm. For more complicated tasks, it's often more practical to help the computer "learn" how to develop its own algorithm. I define **learning** as the process of turning information into knowledge and understanding.

Machine learning (ML) is the name given to the study of computer algorithms that improve *automatically* through experience (data).

An early definition of ML can be attributed to Arthur Samuel (1901–1990) — a pioneer of artificial intelligence research:

[Machine Learning is] a field of study that gives computers the ability to learn without being explicitly programmed.

Arthur Samuel, 1959

One of the first examples of a computer learning a task without being explicitly programmed was Samuel's own work on simulating the game of checkers. The computer even ended up surpassing Samuel himself. Nowadays, it isn't so surprising that computers are able to surpass human-level performance (on narrow tasks) but it was a real doozie back in the day.

A formal definition of the types of algorithms studied in machine learning is attributed to computer scientist Tom Mitchell:

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks T , as measured by P , improves with experience E .

Tom M. Mitchell, 1997

e.g. For the task T of playing checkers, one could define the performance measure P as the percentage of games won against opponents, and the training experience E could be measured by the number of games simulated by the computer against itself. More concretely:

Experience E presents itself in the form of data. Data are observations. Thus, machine learning is a subfield of computer science that focuses on the study of algorithms that can learn from data and make predictions.

So far we've been very general and it helps to establish some form of taxonomy in the field of machine learning.

1.1 Paradigms

At a high level, a **paradigm** in machine learning is the broader philosophy or theoretical framework under which an algorithm is trained to solve a particular task. The paradigm one works in is informed by the type (structure and availability) of (**training**) **data** that has been collected and the nature of the feedback the algorithm receives i.e. how, if, or when the algorithm is informed about its performance. The broadest paradigms¹ are:

- **Supervised learning**
 - Type of data: A static (fixed) collection of labelled² data.
 - Feedback: Explicit and correct labels to compare our algorithm's predictions against.
- **Unsupervised learning**
 - Type of data: Static, unlabelled data.
 - Feedback: No explicit feedback and the algorithm must come to its own conclusions about the pattern or structure of the input data.
- **Semi-supervised learning**
 - Type of data: Static — a combination of labelled and unlabelled data.
 - Feedback: Explicit labels for a portion of the data (as in supervised learning) but the algorithm is designed to (often) infer patterns about the unlabelled data from the labelled data.
- **Reinforcement learning** is based on an agent-environment interaction framework driven by reward maximisation (or penalty minimisation).
 - Type of data: Dynamic — a set of observations (states, actions and transition probabilities) and rewards (and discounts for future reward value) that evolve over time.
 - Feedback: Rewards or penalties over time (immediate or delayed) based on the agent's sequential actions.

1.2 Tasks and Models

Suppose like before, that we are given a set of data and a task to accomplish within a particular paradigm of machine learning. The task is to investigate some real-life/natural phenomenon e.g. a *binary* classification task on e-mail data organises e-mails into *two* distinct groups — spam and not spam.

Oftentimes, such problems are particularly complex. To this end, it's beneficial to make some assumptions (influenced by either knowledge a priori or through exploratory data analysis) about which features/components are relevant, and how they relate to each other:

¹There are more emerging paradigms but these are the broadest ones.

²The data presents itself as a set of (feature vector, label) pairs.

In general parlance, a **model** is a mathematical, logical or physical representation of a process, concept or item. We use models to idealise complex phenomena and extract important information. The value of a model is that it approximates reality in a way that's useful.

All models are wrong, but some are useful.

George Box (probably)

Simpler and less flexible models tend to be easier to interpret. The tradeoff of a simple model is that it's less likely to capture enough of a phenomenon's inherent complexity — the resulting errors in calculation are called **bias**. There's an important balance to strike between flexibility and interpretability.

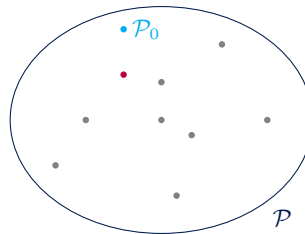
The term **machine learning model** is perhaps the most ambiguous term available to man. I'll try to be specific:

- A **probabilistic model** is the name given to the assumptions made about a process that generates training data.
- A **training/learning algorithm** is the procedure that uses the training data to learn a suitable function.
- I reserve the term **(machine learning) model** to refer to the **learned function** that has been **output** by a training algorithm.

Mathematics and statistics are natural languages to formalise a model.

A *mathematical model* is a description of a system using mathematical concepts and language. A *statistical model* is a type of mathematical model which embodies a set of statistical assumptions³ concerning the generation of sample data. Statistical models differ from other mathematical models by being non-deterministic. Thus, in a statistical model, some of the variables don't have specific values and instead have probability distributions — these variables are *stochastic*.

Formally, a statistical model can be thought of as a pair $(\Omega, \mathcal{P})$ where Ω is a set of possible observations (the outcome/sample space) and $\mathcal{P}$ is a set of probability distributions on Ω . The thinking behind this definition is that there exists a true probability distribution $\mathcal{P}_0$ that dictates the generation of the sample data that's been collected. $\mathcal{P}$ is typically (always?) parameterised in some way so we can write $\mathcal{P} = \{\mathcal{P}_\theta : \theta \in \Theta\}$. This collection contains a **distribution** that adequately approximates $\mathcal{P}_0$.



A statistical model $(\Omega, \mathcal{P} = \{\mathcal{P}_\theta : \theta \in \Theta\})$ is called *parametric* if Θ has finite dimension i.e. $\Theta \subseteq \mathbb{R}^k$ where $k \in \mathbb{Z}^+$. k is called the dimension of the model. A model is called *non-parametric* if Θ is infinite-dimensional.

³Usually specified as a mathematical relationship between random variables and non-random variables.

Most of the recent economic value generated by machine learning is through supervised learning but there are important use-cases for unsupervised learning.

Andrew Ng (2018)

We begin with supervised learning ↓

Supervised Learning

Supervised learning is the paradigm of machine learning under which we use labelled training data to employ an algorithm that outputs a machine learning model which can correctly classify an unseen input, or predict an outcome based on said input.

The labelled training data is a set of $n \in \mathbb{Z}$ (object, label) pairs

$$\mathcal{T} := \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$$

We call this set the **training set**. Each pair $(\mathbf{x}^{(i)}, y^{(i)})$ is called a **training example**:

- Each $\mathbf{x}^{(i)}$ is an **input vector** of dimension p which contains information about each object:

$$\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix}.$$

- In many applications, there is typically some form of feature-extraction that is performed on the original data variables. This results in a **feature vector**:

$$\phi(\mathbf{x}^{(i)}) = \begin{bmatrix} \phi_1(\mathbf{x}^{(i)}) \\ \vdots \\ \phi_M(\mathbf{x}^{(i)}) \end{bmatrix},$$

where each feature $\phi_j(\mathbf{x}^{(i)})$ is the result of evaluating the j^{th} basis function ϕ_j at $\mathbf{x}^{(i)}$. Each ϕ_j can be non-linear so this allows for greater flexibility in modelling.

- $y^{(i)}$ is called the **label** or **response** corresponding to $x^{(i)}$.

Let's say the input vectors $\mathbf{x}^{(i)}$ live in a space $\mathcal{X}$ called the **feature space**, and the labels $y^{(i)}$ live in $\mathcal{Y}$. Then, $\mathcal{T} \subseteq \mathcal{X} \times \mathcal{Y}$.

The goal of a supervised learning algorithm is to use a set of training data $\mathcal{T}$ to learn a function $h: \mathcal{X} \rightarrow \mathcal{Y}$ that predicts the value of $y \in \mathcal{Y}$ corresponding to a given input feature vector $\mathbf{x} \in \mathcal{X}$.

Such a function h has historically been called a **hypothesis**.

A successful (and if we're lucky, optimal) h will exhibit the following two properties:

- For each example $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{T}$, the prediction $h(\mathbf{x}^{(i)})$ is a “good” estimate for the true label $y^{(i)}$.
- The estimate h also generalises well in the sense that its output $h(\mathbf{x})$ for some unseen vector $\mathbf{x} \in \mathcal{X}$ is a “good” predictor for its corresponding expected value of $y \in \mathcal{Y}$ given $\mathbf{x}$.

A supervised learning problem is called

- a **regression problem** if $\mathcal{Y} = \mathbb{R}$, or
- a **classification problem** if the output label y can only take on a discrete number of values i.e. $\mathcal{Y} = \{c_1, \dots, c_K\}$ for $K < \infty$. In this case, h is typically called a **classifier**.

2.1 Probability to Model Uncertainty

Supervised learning is interesting because the response may not be a deterministic function of the inputs. In such a case, one can employ a statistical model that makes assumptions about the nature of the data-generating phenomenon under investigation.

A good first step is always to plot the data $\mathcal{T}$ to see if there's some obvious relationship between the inputs and their respective responses. From this, we can build up some picture of what a good hypothesis h relating them could look like.

Example 2.1.1

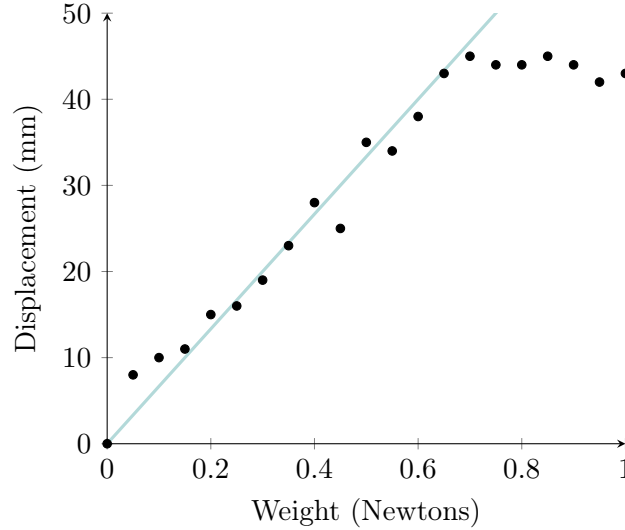


Figure 2.1: Plotting the relationship between the displacement of a spring and the weight placed on it to investigate Hooke's Law. The weight values live in $\mathcal{X}$, and displacement in $\mathcal{Y}$.

From the above data, we could conceivably use some form of positive square root (or even some linear transformation of a logarithmic) function for h . There are many such candidate functions h and we call their collective $\mathcal{H}$ a **hypothesis class**.

Let's say we chose to use the collection of linear transformations of positive square root functions $\mathcal{H} = \{a + b\sqrt{x+c} \text{ where } x \geq c: a, b, c \in \mathbb{R}\}$. We may write each hypothesis as $h_{a,b,c}(x)$ and so the constants a , b , and c parameterise $\mathcal{H}$. Typically one collects these parameters into a column vector.

Definition 2.1.2 In general, we write

$$\mathcal{H} := \{h_{\theta} : \theta \in \Theta\}$$

for the **parametric hypothesis class** depending on a parameter $\theta \in \Theta$.

The parameter space Θ is always some subset of $\mathbb{R}^n$, and so θ is a column vector. ►

The choice of $\mathcal{H}$ limits the scope of functions through which we can search (by varying the parameters) in order to find the “best” hypothesis that approximates the true relationship between inputs and responses. It's clear to see that this choice has far-reaching consequences for all subsequent analysis.

Going back to the plotted data, we also observe that the data-collection method is almost never perfect, so there will be some random error in the observed values. This can be seen in the figure by the **data** being distributed somewhat randomly about a positive linear trend (represented by the **straight line**) which plateaus at around 0.6 Newtons. Incorporating such uncertainty is another motivator for the use of a statistical model, written in the language of probability.

2.1.1 PROBABILISTIC MODEL

We require a few definitions to properly state the assumptions being made.

Definition 2.1.3

- Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. This will model the randomness governing the phenomenon from which we wish to sample our data.
- Let $(E, \mathcal{E})$ be another measurable space. A **random element** is an $(\mathcal{F}, \mathcal{E})$ -measurable map $X: \Omega \rightarrow E$ i.e. $\forall B \in \mathcal{E}, X^{-1}(B) \in \mathcal{F}$. Such maps mathematically formalise the process of drawing a single observable (number, vector etc. e.g. height) corresponding to an outcome (e.g. the drawing of a student from a class).
 - If $(E, \mathcal{E}) = (\mathbb{R}, \mathcal{B}_{\mathbb{R}})$, then X is a real-valued **random variable**.
 - If $(E, \mathcal{E}) = (\mathbb{R}^n, \mathcal{B}_{\mathbb{R}^n})$, then X is a real-valued **random vector**, and we denote this by a boldface $\mathbf{X}$.
- A **random sample** of size n is a collection of random elements $X_i: (\Omega, \mathcal{F}) \rightarrow (E, \mathcal{E})$ defined on the same probability space $(\Omega, \mathcal{F}, \tilde{\mathbb{P}})$ that are mutually $\tilde{\mathbb{P}}$ -independent¹, and identically distributed with distribution $\mathbb{P}$ s.t. $\tilde{\mathbb{P}} = \otimes_n \mathbb{P}$. We denote this by

$$X_i \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}.$$

Let $(\mathbf{X}, Y): \Omega \rightarrow \mathcal{X} \times \mathcal{Y}$ be the random element representing the process of observing one input-output pair (x, y) from an outcome realised by the underlying phenomenon (which is governed by Ω). Henceforth, I am going to let $\mathbb{P}$ denote the distribution governing the underlying phenomenon, and so the push-forward measure $(\mathbf{X}, Y)_\# \mathbb{P}$ (which I will denote by $\mathbb{P}_{(\mathbf{X}, Y)}$) is the true distribution governing the observation of input-output pairs (x, y) .



The assumption we will repeatedly make throughout is that our training set $\mathcal{T}$ is a realisation of a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{(\mathbf{X}, Y)}.$$

This is called the **random design setup**.

In particular, each $(\mathbf{X}^{(i)}, Y^{(i)}): \Omega \rightarrow \mathcal{X} \times \mathcal{Y}$ is a random element and its realisation in our training set is $(\mathbf{x}^{(i)}, y^{(i)})$.

Our goal is to optimally predict the response Y given the predictors $\mathbf{X}$. This means we're interested in $\mathbb{E}[Y | \mathbf{X}]$ — the best $\sigma(\mathbf{X})$ -measurable approximation of Y . In the language of ordinary human beings, the random variable $\mathbb{E}[Y | \mathbf{X}]$ is our best guess for Y given the knowledge/information $\sigma(\mathbf{X})$ that the predictors $\mathbf{X}$ offer.

¹The reason for using $\tilde{\mathbb{P}}$ is a technical one which involves a suppression of notation. We wish for our underlying probability measure $\tilde{\mathbb{P}}$ to be able to support statements about several random variables so it's defined on a σ -algebra on Ω , and Ω can be thought of as the n -fold product of the outcome space Ω of each single run of the experiment (which we represent by X_i). Accordingly, we redefine X_i to be defined on $\tilde{\Omega}$ as the indicator of the i^{th} component. Thus, when we write Ω in our definition, we are writing it as a placeholder for $\tilde{\Omega}$. For more on this, see **Chapter 13** from [1].

2.2 Loss and Risk

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, and $X: \Omega \rightarrow \mathbb{R}$ be a random variable i.e. X is $\mathcal{F}$ -measurable.

- I use ordinary parentheses for the expectation of X

$$\mathbb{E}(X) = \int_{\Omega} X(\omega) d\mathbb{P}(\omega),$$

- and square parentheses for random variables e.g.
 - The conditional expectation $\mathbb{E}[X | \mathcal{G}]$ of X given a sub- σ -algebra $\mathcal{G} \subseteq \mathcal{F}$. Any random variable Z is called a conditional expectation of X given $\mathcal{G}$ if Z is $\mathcal{G}$ -measurable, and satisfies the averaging property i.e. for every $B \in \mathcal{G}$:

$$\mathbb{E}(\mathbb{1}_B Z) = \mathbb{E}(\mathbb{1}_B \mathbb{E}[X | \mathcal{G}]).$$

- The conditional probability of $A \in \mathcal{F}$ given $\mathcal{G}$ is denoted by $\mathbb{P}[A | \mathcal{G}]$
- I write $\text{Var}[X | \mathcal{G}]$ to represent the conditional variance (as seen later).

By virtue of the randomness in our assumptions so far, making (almost) any choice of hypothesis will yield predictions $h(\mathbf{x}^{(i)})$ that fall short from their respective observed labels $y^{(i)}$ for $\mathbf{x}^{(i)}$ i.e. we incur a loss.

Definition 2.2.1 The **loss² of a particular hypothesis** $h \in \mathcal{H}$ associated with a particular training example $(\mathbf{x}^{(i)}, y^{(i)})$ is made precise as a non-negative real-valued function $\mathcal{L}_h: \mathcal{X} \times \mathcal{Y} \rightarrow [0, +\infty)$

$$(\mathbf{x}^{(i)}, y^{(i)}) \mapsto \mathcal{L}_h(\mathbf{x}^{(i)}, y^{(i)}) := \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)})$$

which quantifies how good our hypothesis is at predicting $y^{(i)}$ from $\mathbf{x}^{(i)}$. The function $\mathcal{L}$ that maps from $\mathcal{Y} \times \mathcal{Y} \rightarrow [0, +\infty)$ is called a **loss function**.

In statistics, it's common practice to **average the losses** over an entire population:

Definition 2.2.2 The deterministic number $R(h)$, called the **statistical risk** (or simply **risk**), defined by the following **expected** value

$$\begin{aligned} R(h) &:= \mathbb{E}(\mathcal{L}(h(\mathbf{X}), Y)) := \int_{\Omega} \mathcal{L}(h(\mathbf{X}(\omega)), Y(\omega)) d\mathbb{P}(\omega) \\ &= \int_{\mathcal{X} \times \mathcal{Y}} \mathcal{L}(h(\mathbf{x}), y) d\mathbb{P}_{(\mathbf{X}, Y)}(\mathbf{x}, y) \end{aligned}$$

captures how bad our hypothesis is on average in predicting the label of an unseen input.

Remarks 2.2.3

- After a change of variables, $R(h)$ is an integral with respect to the true distribution of the data. If we know $\mathbb{P}_{(\mathbf{X}, Y)}$ then we may compute this integral.
- We think of the statistical risk as the expectation of a composition $R(h) = \mathbb{E}(\mathcal{L}_h \circ (\mathbf{X}, Y))$. Note that the domains and codomains match up properly.
 - $(\mathbf{X}, Y): \Omega \rightarrow \mathcal{X} \times \mathcal{Y}$
 - $\mathcal{L}_h: \mathcal{X} \times \mathcal{Y} \rightarrow [0, +\infty)$

So $\mathcal{L}_h \circ (\mathbf{X}, Y): \Omega \rightarrow [0, +\infty)$ is a measurable function (assuming that h and L are appropriately measurable) that we can take the expectation of.

²Sometimes a problem requires one to maximise $-L$, in which case $-L$ is called a **reward**, or **profit** function. Each subfield has its own terminology but the same idea persists.

- A natural follow-up question to ask is: In the case that we know the joint distribution of $(\mathbf{X}, Y)$, what is the minimal achievable expected loss, and at which hypothesis is it attained?

A function that minimises the statistical risk would be the best hypothesis h available with respect to the given loss $\mathcal{L}$. We denote this minimiser by h^* and give it a name:

Definition 2.2.4 Let h^* be an optimal solution to the minimisation problem $\min_h R(h)$. We call h^* a **Bayes predictor**.

By the tower law of conditional expectation,

$$R(h) := \mathbb{E}(\mathcal{L}_h(\mathbf{X}, Y)) = \mathbb{E}(\mathbb{E}[\mathcal{L}_h(\mathbf{X}, Y) | \sigma(\mathbf{X})])$$

and the inner expression (the conditional expectation) is a $\sigma(\mathbf{X})$ -measurable random variable. By the Doob-Dynkin³ Representation Theorem, there exists some measurable⁴ $e_h: \mathcal{X} \rightarrow \mathbb{R}$ s.t.

$$\mathbb{E}[\mathcal{L}_h(\mathbf{X}, Y) | \sigma(\mathbf{X})] = e_h(\mathbf{X}).$$

The map e_h is called the pointwise conditional expectation function, and for reasons that will become apparent by reading the following side-barred summary, we denote it by

$$e_h(\mathbf{x}) =: \mathbb{E}[\mathcal{L}_h(\mathbf{X}, Y) | \mathbf{X} = \mathbf{x}].$$

The actual value of $e_h(\mathbf{x})$ is understood as an integral via the disintegration theorem associated with the conditional law of $\mathcal{L}_h \circ (\mathbf{X}, Y)$ given $\mathbf{X}$. For a detailed account of this, **Section 18.3 ‘I’m Disintegrating’** from [1] is the place to go.

WILDLY AMBITIOUS ATTEMPT TO SUMMARISE CONDITIONAL PROBABILITY

- σ -algebras can be thought of as information pertaining to random variables. A random variable X being $\mathcal{F}$ -measurable means that the information in $\mathcal{F}$ is enough to completely characterise X .
- The conditional expectation of X given a sub- σ -algebra $\mathcal{G} \subseteq \mathcal{F}$ is “the” best $\mathcal{G}$ -measurable approximation of X . (We remark that it’s only unique $\mathbb{P}$ -a.s. because an integral-based equation is its defining property.)
- We define the conditional probability of $A \in \mathcal{F}$ given $\mathcal{G}$ by the conditional expectation of $\mathbb{1}_A$ given $\mathcal{G}$. The random variable $\mathbb{1}_A$ encodes whether or not A occurs. This r.v. may or may not be something we can determine from the information $\mathcal{G}$ alone. If we can’t, then we turn to $\mathbb{P}[A | \mathcal{G}]$ as the best $\mathcal{G}$ -measurable approximation to A .
 - $\mathbb{P}[A | \mathcal{G}]$ can be thought of as encoding the updated belief about A given the partial information $\mathcal{G}$, from which we can reproduce (via the defining property of conditional expectation) the probabilities $\mathbb{P}(A \cap B)$ for $B \in \mathcal{G}$ i.e.

$$\begin{aligned} \mathbb{P}(A \cap B) &= \mathbb{E}(\mathbb{1}_B \mathbb{1}_A) \\ &= \mathbb{E}(\mathbb{1}_B \mathbb{E}[\mathbb{1}_A | \mathcal{G}]) \\ &= \mathbb{E}(\mathbb{1}_B \mathbb{P}[A | \mathcal{G}]) \\ &= \int_B \mathbb{P}[A | \mathcal{G}](\omega) d\mathbb{P}(\omega). \end{aligned}$$

- We can extend this concept to every $A \in \mathcal{F}$, and define the conditional probability on $\mathcal{F}$ given $\mathcal{G}$ as the map $\kappa: \mathcal{F} \times \Omega \rightarrow [0, 1]$ defined for every $A \in \mathcal{F}$ by

$$\omega \mapsto \kappa(A, \omega) := \mathbb{P}[A | \mathcal{G}](\omega).$$

³A more detailed statement of this can be found in Theorem A.42 [2, p. 587].

⁴Indeed, suppose that $(\mathcal{X}, \mathcal{E})$ is the measurable space that $\mathbf{X}: \Omega \rightarrow \mathcal{X}$ maps into. Then the function here is more precisely defined as $e_h: \text{Im}(\mathbf{X}) \rightarrow \mathbb{R}$ and is $\mathcal{E}|_{\mathbf{X}(\Omega)}\text{-}\mathcal{B}_{\mathbb{R}}$ -measurable, where $\mathcal{E}|_{\mathbf{X}(\Omega)}$ is the trace σ -algebra of $\mathcal{E}$ on $\mathbf{X}(\Omega)$.

- If we look to the other half of this map, fixing $\omega \in \Omega$, we would like to have a probability measure for each ω . The historical reasoning for this seems to be that we can use κ to model a stochastic process where each ω can be considered a starting “state” and so $\kappa(\cdot, \omega)$ would be a probability measure that we can use to model how we arrive at the next state.
- A technical problem arises. For every $\kappa(\cdot, \omega)$ to be a probability measure, we demand countable additivity of each map. If we demand countable additivity from $\kappa(\cdot, \omega) = \mathbb{P}[\cdot | \mathcal{G}](\omega)$ **for every** ω , then we have potentially uncountably many null sets to exclude for all of these equalities to hold. Such a union may be non-null. To rectify this, we define what is known as a regular conditional probability on $\mathcal{F}$ given $\mathcal{G}$ as a map for which:
 - **For every** $\omega \in \Omega$, the map $\kappa(\cdot, \omega)$ is a probability measure on $(\Omega, \mathcal{F})$.
 - For every $A \in \mathcal{F}$, the map $\kappa(A, \cdot)$ is a version of $\mathbb{P}[A | \mathcal{G}](\omega)$.
- Given that an r.c.p. on $\mathcal{F}$ given $\mathcal{G}$ exists, we may re-write the defining property of $\kappa(A, \cdot)$ in the suggestive form

$$\begin{aligned} \mathbb{E}(\mathbb{1}_B \mathbb{1}_A) &= \mathbb{E}(\mathbb{1}_B \kappa(A, \cdot)) \\ &= \int_B \kappa(A, \omega) d\mathbb{P}(\omega) \\ &= \int_B \int_{\Omega} \mathbb{1}_A(\omega') d\kappa(\cdot, \omega)(\omega') d\mathbb{P}(\omega) \end{aligned}$$

which expresses that the conditional expectation $\kappa(A, \cdot) = \mathbb{P}[A | \mathcal{G}](\cdot) = \mathbb{E}[\mathbb{1}_A | \mathcal{G}](\cdot)$ is equal $\mathbb{P}$ -a.s. to the Lebesgue integral of $\mathbb{1}_A$ with respect to the probability measure $\kappa(\cdot, \omega)$.

- This can be extended from indicators $\mathbb{1}_A$ to any $X \in L^1(\Omega, \mathcal{F}, \mathbb{P})$ and so we have

$$\mathbb{E}[X | \mathcal{G}] = \int_{\Omega} X(\omega') d\kappa(\cdot, \omega)(\omega') \quad \text{for } \mathbb{P}\text{-a.e. } \omega \in \Omega.$$

- Now we focus on the case where $\mathcal{G}$ is generated by a random variable $Y: (\Omega, \mathcal{F}) \rightarrow (E, \mathcal{E})$ i.e. $\mathcal{G} = \sigma(Y)$. By the Doob-Dynkin representation theorem, since $\kappa(A, \cdot) = \mathbb{P}[A | \sigma(Y)](\cdot) = \mathbb{E}[\mathbb{1}_A | \sigma(Y)](\cdot)$ is $\sigma(Y)$ -measurable, it factors through Y to give a re-parameterisation of $\kappa: \mathcal{F} \times \Omega$ to $\kappa_Y: \mathcal{F} \times E \rightarrow [0, 1]$.
- This regular conditional probability on $\mathcal{F}$ generated by Y (i.e. given $\mathcal{G} = \sigma(Y)$) is a map $\kappa_Y: \mathcal{F} \times E \rightarrow [0, 1]$ satisfying the following:
 - For every $y \in E$, $\kappa_Y(\cdot, y)$ is a probability measure on $\mathcal{F}$
 - For each $A \in \mathcal{F}$, the mapping $y \mapsto \kappa_Y(A, y)$ is
 - * an $\mathcal{E}$ -measurable function satisfying for $\mathbb{P}$ -a.e. $\omega \in \Omega$:

$$\kappa_Y(A, Y(\omega)) = \mathbb{P}[A | \sigma(Y)](\omega),$$

- * and is $\mathbb{P}_Y$ -integrable, satisfying the following disintegration formula for all $A \in \mathcal{F}$ and $D \in \mathcal{E}$:

$$\mathbb{P}(A \cap Y^{-1}(D)) = \int_D \kappa_Y(A, y) d\mathbb{P}_Y(y).$$

- This disintegration formula can be re-written in a way that generalises handily for any integrable random variable X and $D \in \mathcal{E}$:

$$\int_{Y^{-1}(D)} X(\omega) d\mathbb{P}(\omega) = \int_D \left(\int_{\Omega} X(\omega') d\mathbb{P}^y(\omega') \right) d\mathbb{P}_Y(y).$$

- From this, there's a bit more re-writing (noticing that the LHS is an expectation of X against the indicator $\mathbb{1}_{Y^{-1}(D)}$, and so we may use the defining property of conditional expectation to write

$$\int_{Y^{-1}(D)} \mathbb{E}[X | \sigma(Y)] d\mathbb{P}(\omega) = \int_D \left(\int_{\Omega} X(\omega') d\mathbb{P}^y(\omega') \right) d\mathbb{P}_Y(y).$$

- By the Doob-Dynkin theorem, the integrand on the LHS can be written as a function h_Y of Y . Indeed, this h_Y is the pointwise conditional expectation function and that reveals itself by a change of variables to get the above equality in terms of integrals with respect to $\mathbb{P}_Y$!

Definition 2.2.5 We call the assignment h_Y the **pointwise conditional expectation function** of X given $Y = y$, and we also denote it by

$$y \mapsto h_Y(y) := \int_{\Omega} X(\omega') d\mathbb{P}^y(\omega') =: \mathbb{E}[X | Y = y].$$

It follows from the above technical considerations that our risk is given by the following expectation

$$\begin{aligned} R(h) &:= \mathbb{E}(e_h \circ \mathbf{X}) = \int_{\Omega} e_h(\mathbf{X}(\omega)) d\mathbb{P}(\omega) = \int_{\mathcal{X}} e_h(\mathbf{x}) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}) \\ &=: \int_{\mathcal{X}} \mathbb{E}[\mathcal{L}_h(\mathbf{X}, Y) | \mathbf{X} = \mathbf{x}] d\mathbb{P}_{\mathbf{X}}(\mathbf{x}) \\ &= \int_{\mathcal{X}} \left(\int_{\Omega} \mathcal{L}_h(\mathbf{x}, Y(\omega')) d\mathbb{P}^{\mathbf{x}}(\omega') \right) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}) \\ &= \int_{\mathcal{X}} \left(\int_{\mathcal{Y}} \mathcal{L}_h(\mathbf{x}, y) d\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})(y) \right) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}) \\ &= \int_{\mathcal{X}} \left(\int_{\mathcal{Y}} \mathcal{L}(h(\mathbf{x}), y) d\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})(y) \right) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}), \end{aligned}$$

where in the last line we have “pushed forward” the conditional probability measure $\mathbb{P}^{\mathbf{x}}(\cdot) := \kappa_{\mathbf{X}}(\cdot, \mathbf{x})$ on $\mathcal{F}$ given $\mathbf{X}$ by Y (in the sense of the brief discussion in **Section 18.3.2** [1]) to obtain the conditional law of Y given $\mathbf{X}$.

The above expression tells us that the risk $R(h)$ can be thought of as a weighted average of pointwise conditional risks $e_h(\mathbf{x})$. We also demonstrated that each $e_h(\mathbf{x})$ depends on h only through the value $h(\mathbf{x})$ because we integrate out the y term (with respect to the conditional distribution of Y given $\mathbf{X}$ at $\mathbf{x}$) in the expression

$$e_h(\mathbf{x}) = \int_{\mathcal{Y}} \mathcal{L}(h(\mathbf{x}), y) d\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})(y).$$

This observation tells us that to minimise $R(h)$, we are free to choose for each $\mathbf{x} \in \mathcal{X}$ the value $z = h(\mathbf{x})$ that minimises

$$e_h(\mathbf{x}) = \int_{\mathcal{Y}} \mathcal{L}(z, y) d\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})(y).$$

Thus, a⁵ Bayes predictor is defined pointwise $\mathbb{P}_{\mathbf{X}}$ -a.e. by

$$h^*(\mathbf{x}) \in \operatorname{argmin}_{z \in \mathcal{Y}} \int_{\mathcal{Y}} \mathcal{L}(z, y) d\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})(y).$$

Definition 2.2.6 The **Bayes risk** R^* is the (statistical) risk of any Bayes predictor h^* i.e.

$$R^* := R(h^*) := \mathbb{E}(\mathcal{L}_{h^*}(\mathbf{X}, Y)).$$

2.2.1 EMPIRICAL RISK

In practice, we don’t know the joint distribution $\mathbb{P}_{(\mathbf{X}, Y)}$ of our population so we can’t compute $R(h)$ directly. However, we do have access to our training data $\mathcal{T}$, and we are working under the assumption that it’s a realisation of a random sample drawn independently and identically from the underlying phenomenon’s true distribution. We use this random sample to define an estimator to estimates the statistical risk:

Definition 2.2.7 The **empirical risk** of our hypothesis is the estimator

$$\widehat{R}(h) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_h(\mathbf{X}^{(i)}, Y^{(i)}).$$

⁵The reason for the membership $\in$ symbol is that in general for an optimisation problem, if a solution exists then it may not be unique. In our case, any Bayes predictor is only unique up to $\mathbb{P}_{\mathbf{X}}$ -null sets because the “outer” expectation is an integral, hence my insistence on ‘a’ Bayes predictor throughout.

A realisation of this estimator is the average loss on the training set

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)}).$$

Questions I had at the time I encountered the concept of empirical risk:

1. What is the probability distribution $\mathbb{P}_{\blacksquare}$ we're using instead of $\mathbb{P}_{(\mathbf{X}, Y)}$? If the risk is (after a change of variables) obtained by integrating our loss $\mathcal{L}_h \circ (\mathbf{X}, Y)$ with respect to the distribution $\mathbb{P}_{(\mathbf{X}, Y)}$, and it turns out to be a sum

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)}),$$

then maybe $\mathbb{P}_{\blacksquare}$ is a discrete-looking thing?

2. When I think of an estimator, I think of the composition $T \circ \mathbf{X}$ of a statistic T and a random sample $\mathbf{X}$. What is T in the case of $\widehat{R}(h)$?
3. In which sense is the empirical risk a good estimator of the statistical risk?

A few months later I've returned to this and now have some answers to the first two questions:

We're working under the random design setup so our random sample can be written as a single random variable for brevity:

$$\mathbf{Z} := ((\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)})) : \Omega \rightarrow (\mathcal{X} \times \mathcal{Y})^n.$$

To align the empirical risk with my personal definition of an estimator as the composition $T \circ \mathbf{X}$, the statistic T is the map $T_h : (\mathcal{X} \times \mathcal{Y})^n \rightarrow \mathbb{R}$ defined by

$$T_h((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)})$$

in order to conclude that $\widehat{R}(h) = T_h \circ \mathbf{Z}$. This means that our loss $\mathcal{L}$ must necessarily be measurable.

Furthermore, for a realised training set $\mathcal{T} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1, \dots, n}$ one can define the associated empirical measure

$$\widehat{\mathbb{P}}_{\mathcal{T}} := \frac{1}{n} \sum_{i=1}^n \delta_{(\mathbf{x}^{(i)}, y^{(i)})}.$$

Then our empirical risk comes from the statistical risk with respect to this measure:

$$\widehat{R}(h) = \int_{\mathcal{X} \times \mathcal{Y}} \mathcal{L}(h(\mathbf{x}), y) d\widehat{\mathbb{P}}_{\mathcal{T}}(\mathbf{x}, y) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)}).$$

The idea is that we approximate the true underlying distribution $\mathbb{P}_{(\mathbf{X}, Y)}$ with the empirical measure, and as a consequence it should be the case that $\widehat{R}(h)$ converges to $R(h)$ as $n \rightarrow +\infty$.

We can view the empirical measure as a realisation of an “empirical” random measure $\widehat{\mathbb{P}}_n$ from Ω into the collection of probability measures on $\mathcal{X} \times \mathcal{Y}$ defined by

$$\omega \mapsto \widehat{\mathbb{P}}_n(\omega) = \frac{1}{n} \sum_{i=1}^n \delta_{(\mathbf{x}^{(i)}, Y^{(i)})}(\omega).$$

Thus, for a fixed ω_0 , $(\mathbf{X}^{(i)}, Y^{(i)})(\omega_0) = (\mathbf{x}^{(i)}, y^{(i)})$ and $\widehat{\mathbb{P}}_n(\omega_0) = \widehat{\mathbb{P}}_{\mathcal{T}}$. I'm not sure if this will be useful at some point but I leave it here as a loose end.

Regression

Let $\mathcal{X} = \mathbb{R}^p$ and $\mathcal{Y} = \mathbb{R} = (-\infty, +\infty)$.

Regression is a term used as a shorthand for regression analysis — a series of statistical techniques developed to investigate the relationship between a collection of (one or more) independent/predictor variables (the regressors) $X_1, \dots, X_p$, and a¹ real-valued dependent/response variable (the regressand) Y .

3.1 Setup

As established in the previous section, we assume that there is some true joint distribution between predictors and response, represented by $(\mathbf{X}, Y) \sim \mathbb{P}_{(\mathbf{X}, Y)}$. Our goal is to optimally predict the response Y given the predictors $\mathbf{X}$ so we're interested in the random variable $\mathbb{E}[Y | \mathbf{X}]$. By the Doob-Dynkin representation theorem, there exists some true (deterministic function) $f: \mathbf{X}(\Omega) \rightarrow \mathcal{Y}$ that is suitably measurable² s.t.

$$f(\mathbf{X}) = \mathbb{E}[Y | \mathbf{X}].$$

Definition 3.1.1 This f is called the **true regression function** of Y on $\mathbf{X}$.

Since $\mathbf{X}$ and $\mathbf{Y}$ are random variables³, there will naturally be some error between the prediction $f(\mathbf{X})$ and Y . We denote this error by the random variable

$$\varepsilon := Y - f(\mathbf{X}) = Y - \mathbb{E}[Y | \mathbf{X}]$$

which captures all the residual error that isn't predictable/explainable by the information that $\mathbf{X}$ provides (in the form of $f(\mathbf{X}) = \mathbb{E}[Y | \sigma(\mathbf{X})] := \mathbb{E}(Y | \mathbf{X})$). We may re-write this equality in the familiar form $Y = f(\mathbf{X}) + \varepsilon$. From this definition of ε , it follows that

$$\begin{aligned} \mathbb{E}[\varepsilon | \mathbf{X}] &= \mathbb{E}[Y - f(\mathbf{X}) | \mathbf{X}] \\ &= \mathbb{E}[Y | \mathbf{X}] - \mathbb{E}[f(\mathbf{X}) | \mathbf{X}] \\ &= \mathbb{E}[Y | \mathbf{X}] - f(\mathbf{X}) \\ &= \mathbb{E}[Y | \mathbf{X}] - \mathbb{E}[Y | \mathbf{X}] \\ &= 0. \end{aligned}$$

This property $\mathbb{E}[\varepsilon | \mathbf{X}] = 0$ is called strict exogeneity. It means that our errors have zero mean conditional on the regressors $\mathbf{X}$ i.e. that all the information relevant for predicting Y is captured by $\mathbf{X}$.

Definition 3.1.2 A **regression model** is a choice of function h to approximate f so that $h \circ \mathbf{X}$ approximates $f \circ \mathbf{X} = \mathbb{E}[Y | \mathbf{X}]$.

This is colloquially referred to as “assuming the conditional mean has the form $\blacksquare$ ”.

¹Strictly speaking, this would be a multivariable univariate regression — variable refers to predictors, and variate refers to responses.

²I've found myself sidelining these technical details quite a bit. If $(\mathcal{X}, \mathcal{E})$ and $(\mathcal{Y}, \mathcal{E}_Y)$ are measurable spaces, then the measurability in this case will be $(\mathcal{E} |_{\mathbf{X}(\Omega)}, \mathcal{E}_Y)$ -measurability.

³Does Y need to be L^1 for the conditional expectation to exist?

3.2 Estimating the Regression Function

Since we don't know $\mathbb{P}_{(\mathbf{X}, Y)}$, we can't calculate $\mathbb{E}[Y | \mathbf{X}]$ directly. Instead, we must estimate f by choosing a hypothesis⁴ h from a (suitable) class of candidate functions $\mathcal{H}$ called the **hypothesis class**. Then we shall use our sampled data $\mathcal{T}$ to optimise h in a way that best approximates $\mathbb{E}[Y | \mathbf{X}]$.

For regression, a common choice⁵ of loss function is:

Definition 3.2.1 The **squared** or **quadratic loss** is defined by $\mathcal{L}(h(\mathbf{X}), Y) = (h(\mathbf{X}) - Y)^2$.

For quadratic loss, the statistical risk of h is $R(h) := \mathbb{E}((Y - h(\mathbf{X}))^2)$ and is called the **mean squared error (MSE)** for short). (I believe we require $Y - h(\mathbf{X})$ to be in L^2 for this minimisation problem to be meaningful.)

A function that minimises the MSE would be the best hypothesis available under quadratic loss. This minimiser is the Bayes predictor discussed at the end of the previous chapter:

$$h^* := \operatorname{argmin}_h \mathbb{E}((Y - h(\mathbf{X}))^2).$$

What form does h^* take in this special case with squared loss? Recall that the Bayes predictor can be found by minimising pointwise i.e.

$$h^*(\mathbf{x}) \in \operatorname{argmin}_z \mathbb{E}[(Y - z)^2 | \mathbf{X} = \mathbf{x}]$$

This argmin is actually equal to the conditional expectation $\mathbb{E}[Y | \mathbf{X} = \mathbf{x}]$ of Y given $\mathbf{X} = \mathbf{x}$.

There are three ways to verify this:

1. Add and subtract $\mathbb{E}(Y | \mathbf{X})$ from the statistical risk expression and expand.
2. Observe that

$$L^2(\Omega, \mathbb{P}) := \{\text{random variables } Y: \Omega \rightarrow \mathbb{R}$$

such that $\mathbb{E}(|Y|^2) < \infty\}$ is a vector space and when equipped with the inner product $\langle X, Y \rangle = \mathbb{E}(XY)$ becomes a Hilbert space. Then we can use the theory of orthogonal projections to prove that $\mathbb{E}(Y | \mathbf{X}) := \mathbb{E}(Y | \sigma(\mathbf{X}))$ (where $\sigma(X)$ is the sigma-algebra generated by $\mathbf{X}$) is equal to $\pi_{\sigma(\mathbf{X})}$ ($\mathbb{P}$ -almost surely) where

$$\pi_{\sigma(\mathbf{X})}: L^2(\Omega, \mathcal{A}, \mathbb{P}) \rightarrow L^2(\Omega, \sigma(\mathbf{X}), \mathbb{P})$$

is the projection map defined by

$$\|Y - \pi_{\sigma(\mathbf{X})}(Y)\| = \inf\{\|Y - Z\|: Z \in L^2(\Omega, \sigma(\mathbf{X}), \mathbb{P})\}$$

where $\sigma(\mathbf{X})$ is a sub-algebra of $\mathcal{A}$. In summary, this is a fancy way to say that the geometry of Hilbert space tells us that minimisation in said space becomes a problem of orthogonal projection.

3. A differentiation argument.

This minimiser $h^*(\mathbf{x}) = \mathbb{E}[Y | \mathbf{X} = \mathbf{x}]$ coincides⁶ with f , and so the terms **regression function** and **pointwise conditional expectation function** are synonymous.

⁴We will use this function to make predictions for unseen examples $\mathbf{x} \in \mathcal{X}$.

⁵Why quadratic loss in the first place? It feels a bit like the choice was justified in hindsight (which I don't take issue with but I can't really figure it out).

- Develop the method
- Realise that convergence issues happen in the algorithms you devise and then
- go back to alter the loss to have properties that make things more convenient e.g. convexity.

⁶As observed earlier because $\mathbb{E}[Y | \mathbf{X}] = f \circ \mathbf{X}$.

As stated at the top of this section, we don't know the (joint distribution of $(\mathbf{X}, Y)$ and hence) conditional distribution of Y given $\mathbf{X}$. We get around this by modelling the regression function by some simple functional form h and then use the training data to minimise the empirical risk instead. In the case that the regression function h is parametric i.e. $h = h_{\boldsymbol{\theta}}$, we can search for

$$\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2.$$

This is an example of **parametric learning**. Below we discuss a particular choice of $\mathcal{H}$ consisting of “linear” hypotheses.

3.3 Linear Regression

So far there's been no restriction on how to represent the hypothesis h i.e. $\mathcal{H}$ is a very broad class of functions. We'd like to, in some meaningful way, limit the family of hypotheses over which we'll learn. Making assumptions about the functional form of h will restrict $\mathcal{H}$. This section will assume that h is a linear hypothesis — the simplest form.

A **linear hypothesis** takes the form of an affine function

$$h_{\boldsymbol{\theta}}(\mathbf{X}) = \theta_0 + \theta_1 X_1 + \dots + \theta_p X_p,$$

where we collect the unknown quantities θ_j , called the **parameters** (or **weights**) of our model, into a vector

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_p \end{bmatrix} \in \mathbb{R}^{p+1}.$$

The linearity of our hypothesis refers to **linearity in the coefficients** θ_j .



Example 3.3.1 The hypothesis $h_{\boldsymbol{\theta}}(X) = \theta_0 + \theta_1 X^3$ is linear in $\boldsymbol{\theta} = [\theta_0 \ \theta_1]^\top \in \mathbb{R}^2$, but the hypothesis $h_{\boldsymbol{\theta}}(X) = \theta_0 + \sin(\theta_1)X$ is not linear in $\boldsymbol{\theta}$. ▶

Such a function $h_{\boldsymbol{\theta}}$ is also called a **linear predictor**, or a **discriminant function**. For notational convenience, one can define an intercept random variable $X_0 = 1$ and prepend it to the random vector $\mathbf{X}$ so that h may be re-written as

$$h_{\boldsymbol{\theta}}(\mathbf{X}) = \langle \boldsymbol{\theta}, \mathbf{X} \rangle := \sum_{i=0}^p \theta_i X_i = \mathbf{X}^\top \boldsymbol{\theta},$$

where $\mathbf{X} = (X_0, X_1, \dots, X_p)$ is defined by $\mathbf{X}(\omega) = \begin{bmatrix} 1 \\ X_1(\omega) \\ \vdots \\ X_p(\omega) \end{bmatrix} \in \mathbb{R}^{p+1}$, and we call θ_0 a **bias** term.

A nice way to interpret a linear hypothesis is by the expression $\partial_j h(\mathbf{X}) = \theta_j$ i.e. holding all other predictors fixed, θ_j represents the effect on h of a one unit increase in X_j . This behaviour characterises a **linear-response model**⁷.

⁷A **linear response model** is suitable for situations where the response variable can vary, to a good approximation, indefinitely in either direction. More generally, a linear-response model works for any quantity that only varies by a relatively⁸ small amount compared to the variation in the predictive values e.g. human height. These assumptions aren't suitable for some types of response variables and this will be discussed later under GLMs as a flexible generalisation of linear regression.

Here I collect the above assumptions to define a linear regression model for the conditional expectation of Y given $\mathbf{X}$.

Let $(\mathbf{X}, Y) \sim \mathbb{P}_{(\mathbf{X}, Y)}$. A (population-level) **linear regression model** is a regression model that assumes linearity of the conditional mean i.e. we approximate the true regression function f with a parametric linear hypothesis $h_{\boldsymbol{\theta}}$ i.e.

$$\mathbb{E}[Y | \mathbf{X}] \approx h_{\boldsymbol{\theta}}(\mathbf{X}) = \mathbf{X}^\top \boldsymbol{\theta}.$$

On the sample-level, we introduce more assumptions that allow us to relate how our data

$$\mathcal{T} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_1^n$$

is related to the population. As usual, we assume that our training set is a realisation of a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{(\mathbf{X}, Y)}.$$

We can think of these as identical copies of $(\mathbf{X}, Y)$ so the population assumptions hold for each $(\mathbf{X}^{(i)}, Y^{(i)})$. The linearity assumption can be written in matrix-form to summarise the n equations into one.

For the sake of simplifying notation, we'll collect the predictor random vectors into a single random vector (of random vectors)

$$\mathbf{X} = (\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) : (\Omega, \mathcal{F}) \rightarrow ((\mathbb{R}^{p+1})^n, \mathcal{B}_{(\mathbb{R}^{p+1})^n}).$$

A realisation of $\mathbf{X}$ is given by

$$\mathbf{X}(\omega) = \begin{bmatrix} \mathbf{X}^{(1)}(\omega) \\ \vdots \\ \mathbf{X}^{(n)}(\omega) \end{bmatrix} = \begin{bmatrix} \mathbf{x}^{(1)} \\ \vdots \\ \mathbf{x}^{(n)} \end{bmatrix}$$

Statistics textbooks represent $\mathbf{X}(\omega)$ more presentably by identifying this vector of column vectors $(\mathbb{R}^{p+1})^n$ with a matrix in $\mathbb{R}^{n, p+1}$. Doing so allows one to succinctly represent linearity over all the training examples (a system of n equations) as a matrix equality. I'll denote the map that makes such an identification⁹ by Φ so a realisation of $\mathbf{X}(\omega)$ is identified with the matrix

$$\mathcal{X} := \Phi(\mathbf{X}(\omega)) = \begin{bmatrix} (\mathbf{x}^{(1)})^\top \\ \vdots \\ (\mathbf{x}^{(n)})^\top \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_p^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_p^{(n)} \end{bmatrix}.$$

This matrix $\mathcal{X}$ whose rows are the input features of each example in $\mathcal{T}$ is called the **design matrix** of our model. When I need to speak about conditioning on a random variable I will refer to $\mathbf{X}$, and when I wish to speak of systems of equations (like I will immediately below) I will refer to $\mathcal{X}$. On the level of random variables, we have that

$$\Phi(\mathbf{X}) = \Phi(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) = \begin{bmatrix} (\mathbf{X}^{(1)})^\top \\ \vdots \\ (\mathbf{X}^{(n)})^\top \end{bmatrix}.$$

The rest of the notation is straight-forward. Simply collect the responses and errors into their own respective random vectors $\mathbf{Y}$ and $\boldsymbol{\varepsilon}$, and denote their realisations by labels $\mathbf{y}$ and residuals $\boldsymbol{\epsilon}$:

$$\mathbf{Y} = \begin{bmatrix} Y^{(1)} \\ \vdots \\ Y^{(n)} \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}, \boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon^{(1)} \\ \vdots \\ \varepsilon^{(n)} \end{bmatrix}, \text{ and } \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon^{(1)} \\ \vdots \\ \epsilon^{(n)} \end{bmatrix}.$$

⁹I believe no information is lost by this identification because the σ -algebras generated by both are the same, just re-labelled, and so the ensuing conditional expectation calculations would be the same given either $\sigma(\mathbf{X})$ or $\sigma(\Phi(\mathbf{X}))$.

It follows that

$$\Phi(\mathbf{X})\boldsymbol{\theta} = \begin{bmatrix} (\mathbf{X}^{(1)})^\top \boldsymbol{\theta} \\ \vdots \\ (\mathbf{X}^{(n)})^\top \boldsymbol{\theta} \end{bmatrix}.$$

Finally, we may write all n equations as¹⁰

$$\mathbf{Y} = \Phi(\mathbf{X})\boldsymbol{\theta} + \boldsymbol{\varepsilon},$$

and so a realisation $\mathcal{T}$ of the experiment that these random variables formalise can be represented by the system:

$$\mathbf{y} = \mathcal{X}\boldsymbol{\theta} + \boldsymbol{\epsilon}.$$

3.3.1 EXTRA ASSUMPTIONS

Since the linear hypothesis $h_{\boldsymbol{\theta}}$ is only an approximation for f (the true regression function) and a priori f need not even be linear, it then follows that

$$\mathbb{E}[\varepsilon | \mathbf{X}] = \mathbb{E}[Y - h_{\boldsymbol{\theta}}(\mathbf{X}) | \mathbf{X}] = \mathbb{E}[Y | \mathbf{X}] - h_{\boldsymbol{\theta}}(\mathbf{X})$$

is not necessarily zero. Thus, strict exogeneity is a genuine modelling assumption:

2. Strict exogeneity: The errors have 0 mean conditional on the features i.e. for $i = 1, \dots, n$:

$$0 = \mathbb{E}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})].$$

3. The **rank** of the n by $(p + 1)$ regression matrix, $\mathcal{X}$, is $p + 1$ with probability 1.
4. Constant conditional second moment of errors given predictors:
An assumption that the variances of all the error terms are equal (or similar):

$$\mathbb{E}[(\varepsilon^{(i)})^2 | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = \sigma^2 > 0.$$

5. No correlation between observations:

$$\mathbb{E}[\varepsilon^{(i)}\varepsilon^{(j)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = 0 \quad \text{for } i, j = 1, \dots, n \text{ where } i \neq j$$

Assumption 3 is left stated in the form Hayashi does (with respect to fixed design i.e. in the situation where one has the design matrix X after observing the random sample). I'm not yet sure what the random regressors version of the statement is.

Definition 3.3.2 Let A be an m by n matrix with real entries.

- The **column space** of A , denoted by $\text{colspace}(A)$, is the subspace of $\mathbb{R}^m$ spanned by the columns of A .
- The **column rank** of A is the dimension of the column space of A . Equivalently, the column rank is equal to the size of the largest linearly independent subset of its columns.

¹⁰Not only is this succinct but by *vectorising* the data in a programming language, we also benefit from linear algebra being well-optimised in built-in language packages/modules like `numpy` in Python. Such packages, in general, yield faster results than basic `for`-loops.

Analogous definitions hold if one replaces ‘column’ with ‘row’. It doesn’t really matter which we use because:

Theorem 3.3.3 The row rank and column rank of a matrix A are equal. Thus, we simply refer to the **rank** of a matrix.

Furthermore, $\text{rank}(A) \leq \min(m, n)$.

Remarks 3.3.4

- Since we are assuming that our sample is i.i.d., the strict exogeneity assumption is sometimes written as conditioning on only one of the $\mathbf{X}^{(i)}$ instead of on the whole of $\sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})$. It turns out that these are equivalent.

Proof. Let $\mathcal{G}_n := \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})$. Then $\sigma(\mathbf{X}^{(i)}) \subseteq \mathcal{G}_n$, and by the tower property:

$$\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] = \mathbb{E}[\mathbb{E}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(i)})] | \mathcal{G}_n] = \mathbb{E}[0 | \mathcal{G}_n] = 0$$

For the reverse implication, assume that $\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] = 0$ and run the above argument backwards to conclude that $\mathbb{E}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(i)})] = 0$. ■

Economists prefer to write “ $|\mathcal{G}_n$ ” because it inspires notation like $\mathbb{E}(\varepsilon | \mathcal{X})$ (found in Hayashi [3]) which is read as “Fix the realised design matrix. Then the errors have mean zero.”

- When we combine assumptions 2 and 4, we get conditional homoskedasticity of our error terms:

Definition 3.3.5 The assumption of constant conditional variance

$$\text{Var}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})] = \sigma^2 > 0$$

of our errors given the predictors $\mathbf{X}$ is called **conditional homoskedasticity**.

Proof. For short, let $\sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) =: \mathcal{G}_n$. Then,

$$\begin{aligned} \text{Var}[\varepsilon^{(i)} | \mathcal{G}_n] &= \mathbb{E}\left[\left(\varepsilon^{(i)} - \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]\right)^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[\varepsilon^{(i)2} - \varepsilon^{(i)}\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] + \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[\varepsilon^{(i)2} | \mathcal{G}_n\right] - \underbrace{\mathbb{E}\left[\varepsilon^{(i)}\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n] | \mathcal{G}_n\right]}_{\mathbb{E}[g(\mathbf{X}) | \mathcal{G}_n] = g(\mathbf{X})} + \mathbb{E}\left[\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 | \mathcal{G}_n\right] \\ &= \mathbb{E}\left[\varepsilon^{(i)2} | \mathcal{G}_n\right] - 2(\mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n])^2 + \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 \\ &= \mathbb{E}\left[\varepsilon^{(i)2} | \mathcal{G}_n\right] - \mathbb{E}[\varepsilon^{(i)} | \mathcal{G}_n]^2 \\ &= \sigma^2 - 0 \end{aligned}$$

■

- Strict exogeneity implies $\mathbb{E}(\varepsilon^{(i)}) = 0$ for all $i = 1, \dots, n$.

Proof. By the law of total expectation, $\mathbb{E}(\varepsilon^{(i)}) = \mathbb{E}(\mathbb{E}[\varepsilon^{(i)} | \sigma(\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)})])$. ■

3.4 Ordinary Least-Squares (OLS)

Least-squares fitting is the name given to the mathematical procedure for finding the best-fitting curve to a collection of data by way of minimising the sum of squared residuals of the examples from the curve's respective predictions.

Ordinary¹¹ **least-squares** is a type of **linear** least-squares fitting method that estimates the unknown parameters $\boldsymbol{\theta}$ in a linear regression model by using a dataset $\mathcal{T}$. We denote the sum of squared residuals by $J(\boldsymbol{\theta})$ and define it by

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n (\epsilon^{(i)})^2 := \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2.$$

We can write this sum in matrix-form

$$J(\boldsymbol{\theta}) = \boldsymbol{\epsilon}^\top \boldsymbol{\epsilon} = (\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}).$$

Goal: We wish to find $\boldsymbol{\theta}$ that minimises our cost function $J(\boldsymbol{\theta})$ over $\boldsymbol{\theta} \in \mathbb{R}^{p+1}$.

To tackle this optimisation problem, I learned the basics of (convex) optimisation. I collect the build-up of the core results in **Chapter B**, and will make reference to them throughout.

3.4.1 NORMAL EQUATIONS

Theorem 3.4.1 Consider the minimisation problem

$$\begin{aligned} & \text{minimise} \quad \|\mathcal{X}\boldsymbol{\theta} - \mathbf{y}\|^2 \\ & \text{subject to} \quad \boldsymbol{\theta} \in \mathbb{R}^{p+1} \end{aligned} \tag{3.1}$$

where $\mathcal{X}$ is an n by $p+1$ matrix, and $\mathbf{y} \in \mathbb{R}^n$.

Under the assumption that the columns of $\mathcal{X}$ are linearly independent,

$$\boldsymbol{\theta}^* = (\mathcal{X}^\top \mathcal{X})^{-1} \mathcal{X}^\top \mathbf{y}$$

is the unique global minimiser of (3.1).

Proof. We fall back on the matrix form of the objective function $J(\boldsymbol{\theta}) = (\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y})$. According to **Corollary A.1.4**, calculating the gradient of J at $\boldsymbol{\theta}$ can be done by first finding the Fréchet derivative $dJ(\boldsymbol{\theta})$ of J at $\boldsymbol{\theta}$, and then transposing its coordinate representation (Jacobian).

Definition 3.4.2 Let $U \subseteq^{\text{open}} \mathbb{R}^n$ with $\mathbf{x} \in U$ and $f: U \rightarrow \mathbb{R}$. If there exists a bounded linear¹² map $L_{\mathbf{x}}: \mathbb{R}^n \rightarrow \mathbb{R}$ that satisfies

$$\lim_{\mathbf{v} \rightarrow \mathbf{0} \text{ in } \mathbb{R}^n} \frac{|f(\mathbf{x} + \mathbf{v}) - f(\mathbf{x}) - L_{\mathbf{x}}\mathbf{v}|}{\|\mathbf{v}\|} = 0$$

then f is said to be Fréchet differentiable at $\mathbf{x}$. The linear map $L_{\mathbf{x}}$ is called the Fréchet derivative of f at $\mathbf{x}$ and is denoted by $df(\mathbf{x})$.

¹¹There's some contention over what 'ordinary' actually means. The most sensible explanation I've found is from **a Stack Exchange comment** suggesting that it's there to contrast with the weight parameters in weighted least-squares regression. Namely, if the weights in WLS are all equal to 1, you get OLS.

¹²Other notation for this is $L_{\mathbf{x}} \in \mathcal{L}(\mathbb{R}^n; \mathbb{R})$. Also, one may recognise that $L_{\mathbf{x}}$ is also an element of the dual $(\mathbb{R}^n)^*$ of $\mathbb{R}^n$ — the vector space of linear functionals $f: \mathbb{R}^n \rightarrow \mathbb{R}$ on $\mathbb{R}^n$ that are continuous.

We first calculate the numerator of the difference quotient:

$$\begin{aligned}
J(\boldsymbol{\theta} + \mathbf{v}) - J(\boldsymbol{\theta}) &= (\mathcal{X}(\boldsymbol{\theta} + \mathbf{v}) - \mathbf{y})^\top (\mathcal{X}(\boldsymbol{\theta} + \mathbf{v}) - \mathbf{y}) - J(\boldsymbol{\theta}) \\
&= (\mathcal{X}\boldsymbol{\theta} - \mathbf{y} + \mathcal{X}\mathbf{v})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y} + \mathcal{X}\mathbf{v}) - J(\boldsymbol{\theta}) \\
&= J(\boldsymbol{\theta}) + (\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\mathbf{v}) + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}) + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\mathbf{v}) - J(\boldsymbol{\theta}) \\
&= \underbrace{(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\mathbf{v}) + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y})}_{\in \mathbb{R}} + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\mathbf{v}) \\
&= ((\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\mathbf{v}))^\top + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}) + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\mathbf{v}) \\
&= 2(\mathcal{X}\mathbf{v})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}) + (\mathcal{X}\mathbf{v})^\top (\mathcal{X}\mathbf{v})
\end{aligned}$$

Now we conjecture that the linear part of the remainder $L_{\boldsymbol{\theta}}\mathbf{v} = 2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}\mathbf{v}$ is our derivative candidate. The matrix representation of the map $L_{\boldsymbol{\theta}}$ is $2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}$. It's clearly linear because $L_{\boldsymbol{\theta}}(\mathbf{v} + \mathbf{w}) = L_{\boldsymbol{\theta}}\mathbf{v} + L_{\boldsymbol{\theta}}\mathbf{w}$, and $L_{\boldsymbol{\theta}}(\alpha\mathbf{v}) = \alpha L_{\boldsymbol{\theta}}\mathbf{v}$. Every linear map between normed vector spaces whose domain is finite dimensional is automatically bounded. Now we verify the limit in the definition of the Fréchet derivative. The difference quotient is given by

$$\begin{aligned}
\frac{|J(\boldsymbol{\theta} + \mathbf{v}) - J(\boldsymbol{\theta}) - L_{\boldsymbol{\theta}}\mathbf{v}|}{\|\mathbf{v}\|} &= \frac{|(\mathcal{X}\mathbf{v})^\top (\mathcal{X}\mathbf{v})|}{\|\mathbf{v}\|} = \frac{\|\mathcal{X}\mathbf{v}\|^2}{\|\mathbf{v}\|} \\
&\leq \frac{\|\mathcal{X}\|_{\text{op}} \|\mathbf{v}\|^2}{\|\mathbf{v}\|} \\
&= \|\mathcal{X}\|_{\text{op}} \|\mathbf{v}\| \longrightarrow 0 \text{ as } \mathbf{v} \rightarrow \mathbf{0} \text{ in } \mathbb{R}^n.
\end{aligned}$$

Therefore, $dJ(\boldsymbol{\theta}) = L_{\boldsymbol{\theta}}$ as linear maps, and so $2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}$ is the coordinate representation of $dJ(\boldsymbol{\theta})$ with respect to the standard bases of $\mathbb{R}^{p+1}$ and $\mathbb{R}$, from which it follows that $\nabla J(\boldsymbol{\theta}) = 2\mathcal{X}^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}) = 2\mathcal{X}^\top \mathcal{X}\boldsymbol{\theta} - 2\mathcal{X}^\top \mathbf{y}$.

In CS229, the lecture notes calculate the gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ using some matrix derivative facts which may be found in **Example A.2.2**.

$$\begin{aligned}
\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \left((\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathcal{X}\boldsymbol{\theta} - \mathbf{y}) \right) \\
&= \nabla_{\boldsymbol{\theta}} \left(\boldsymbol{\theta}^\top \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathcal{X}^\top \mathbf{y} - \mathbf{y}^\top \mathcal{X}\boldsymbol{\theta} + \mathbf{y}^\top \mathbf{y} \right) \\
&= \nabla_{\boldsymbol{\theta}} \text{tr} \left(\boldsymbol{\theta}^\top \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathcal{X}^\top \mathbf{y} - \mathbf{y}^\top \mathcal{X}\boldsymbol{\theta} + \mathbf{y}^\top \mathbf{y} \right) \\
&= \nabla_{\boldsymbol{\theta}} \left(\text{tr}(\boldsymbol{\theta}^\top \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta}) - \text{tr}(\mathbf{y}^\top \mathcal{X}\boldsymbol{\theta}) - \text{tr}(\mathbf{y}^\top \mathcal{X}\boldsymbol{\theta}) + \text{tr}(\mathbf{y}^\top \mathbf{y}) \right) \\
&= \nabla_{\boldsymbol{\theta}} \text{tr}(\boldsymbol{\theta}^\top \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta}) - 2\nabla_{\boldsymbol{\theta}} \text{tr}(\mathbf{y}^\top \mathcal{X}\boldsymbol{\theta}) \\
&= \nabla_{\boldsymbol{\theta}} \text{tr}(\boldsymbol{\theta}^\top \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta}) - 2\mathcal{X}^\top \mathbf{y} \\
&\stackrel{(5)}{=} ((\mathcal{X}^\top \mathcal{X})^\top \boldsymbol{\theta} + \mathcal{X}^\top \mathcal{X}\boldsymbol{\theta}) - 2\mathcal{X}^\top \mathbf{y} = 2\mathcal{X}^\top \mathcal{X}\boldsymbol{\theta} - 2\mathcal{X}^\top \mathbf{y}
\end{aligned}$$

In the third equality, note that $\text{tr}(a) = a$ for $a \in \mathbb{R}$. We introduced the trace for convenience of computation using 5 by setting $\mathcal{X}^\top = \boldsymbol{\theta}$, $B = \mathcal{X}^\top \mathcal{X}$, and $C = I$.

Note that J is differentiable. By the **First-Order Necessary Condition for Optimality**, if $\boldsymbol{\theta}^*$ is an unconstrained local minimiser for f , then

$$\begin{aligned}
\mathbf{0} &= \nabla f(\boldsymbol{\theta}^*) \\
&= 2\mathcal{X}^\top \mathcal{X}\boldsymbol{\theta}^* - 2\mathcal{X}^\top \mathbf{y}.
\end{aligned}$$

Re-arranging the above equality yields the system of equations $\mathcal{X}^\top \mathcal{X}\boldsymbol{\theta} = \mathcal{X}^\top \mathbf{y}$, and we call these the **normal equations**.

We wish to invert $\mathcal{X}^\top \mathcal{X}$ to get a solution $\boldsymbol{\theta}^*$. Since $\mathcal{X}$ is assumed to have linearly independent columns, it follows that $\mathcal{X}^\top \mathcal{X}$ is invertible by the following:

Lemma 3.4.3 Let A be an m by n matrix. If the columns of A are linearly independent, then $A^\top A$ is invertible.

Proof. A matrix A is invertible iff its nullspace is trivial $\{\mathbf{0}\}$. Observe that

$$\begin{aligned} (A^\top A)\mathbf{x} = \mathbf{0} &\implies \mathbf{x}^\top (A^\top A)\mathbf{x} = 0 \\ &\iff (A\mathbf{x})^\top (A\mathbf{x}) = 0 \\ &\iff \|A\mathbf{x}\|^2 = 0 \\ &\iff A\mathbf{x} = \mathbf{0} \end{aligned}$$

i.e. if $\mathbf{x}$ is in the nullspace of $A^\top A$, then $A\mathbf{x} = \mathbf{0}$. If we can demonstrate further that $\mathbf{x} = \mathbf{0}$, then the invertibility of $A^\top A$ follows.

Notice that

$$\begin{aligned} A\mathbf{x} &= \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \\ &= \begin{bmatrix} \sum_{j=1}^n a_{1j}x_j \\ \sum_{j=1}^n a_{2j}x_j \\ \vdots \\ \sum_{j=1}^n a_{mj}x_j \end{bmatrix} = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix} x_1 + \cdots + \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{mn} \end{bmatrix} x_n \end{aligned}$$

i.e. $A\mathbf{x}$ is a linear combination of the columns of A with weight x_j corresponding to column j . By the linear independence of the columns, this forces $x_j = 0$ for every j from 1 to n i.e. $\mathbf{x} = \mathbf{0}$. In summary, we've shown that $\mathbf{x} \in \text{nullspace}(A^\top A) \implies \mathbf{x} = \mathbf{0}$ which is equivalent to $A^\top A$ being invertible. †

Thus, an unconstrained local minimiser of J is $\boldsymbol{\theta}^* = (\mathcal{X}^\top \mathcal{X})^{-1} \mathcal{X}^\top \mathbf{y}$.

Now we wish to demonstrate that this local minimiser is a global minimiser, and also unique:

- Let's verify the final assumption of the theorem by calculating the Hessian matrix of J at $\boldsymbol{\theta}^* = (\mathcal{X}^\top \mathcal{X})^{-1} \mathcal{X}^\top \mathbf{y}$. According to **Remarks A.1.6**, we can calculate the Fréchet derivative of $dJ(\boldsymbol{\theta})$ i.e. the second-order Fréchet derivative of J , and then its coordinate representation with respect to the standard basis of $\mathbb{R}^{p+1}$ is the Hessian of J at $\boldsymbol{\theta}$. As before, we calculate the difference.

$$\begin{aligned} dJ(\boldsymbol{\theta} + \mathbf{v})\mathbf{h} - dJ(\boldsymbol{\theta})\mathbf{h} &= 2(\mathcal{X}(\boldsymbol{\theta} + \mathbf{v}) - \mathbf{y})^\top \mathcal{X}\mathbf{h} - 2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}\mathbf{h} \\ &= 2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}\mathbf{h} + 2(\mathcal{X}\mathbf{v})^\top \mathcal{X}\mathbf{h} - 2(\mathcal{X}\boldsymbol{\theta} - \mathbf{y})^\top \mathcal{X}\mathbf{h} \\ &= 2(\mathcal{X}\mathbf{v})^\top \mathcal{X}\mathbf{h} \\ &= \mathbf{v}^\top (2\mathcal{X}^\top \mathcal{X})\mathbf{h} \end{aligned}$$

Let $(L_{\boldsymbol{\theta}}\mathbf{v})\mathbf{h} = L_{\boldsymbol{\theta}}(\mathbf{v}, \mathbf{h}) = \mathbf{v}^\top (2\mathcal{X}^\top \mathcal{X})\mathbf{h}$. It's bilinear and automatically bounded. We don't need to verify the defining limit because $dJ(\boldsymbol{\theta} + \mathbf{v})\mathbf{h} - dJ(\boldsymbol{\theta})\mathbf{h} - L_{\boldsymbol{\theta}}(\mathbf{v}, \mathbf{h}) = 0$. Thus, the Hessian of J at $\boldsymbol{\theta}$ is

$$\nabla^2 J(\boldsymbol{\theta}) = 2\mathcal{X}^\top \mathcal{X}.$$

If we can show that $HJ(\boldsymbol{\theta}) \succ 0$ i.e. that $\boldsymbol{\theta}^\top (HJ(\boldsymbol{\theta}))\boldsymbol{\theta} > 0$ for all non-zero $\boldsymbol{\theta}$, then all the conditions of the theorem are satisfied, and we conclude that $\boldsymbol{\theta}^*$ is a strict local minimiser of J .

- Note that $\boldsymbol{\theta}^\top (\mathcal{X}^\top \mathcal{X})\boldsymbol{\theta} = (\mathcal{X}\boldsymbol{\theta})^\top (\mathcal{X}\boldsymbol{\theta}) = \|\mathcal{X}\boldsymbol{\theta}\|^2 \geq 0$.
- Now observe that $\|\mathcal{X}\boldsymbol{\theta}\|^2 \geq 0$ for all non-zero $\boldsymbol{\theta}$ iff $\mathcal{X}\boldsymbol{\theta} = \mathbf{0} \implies \boldsymbol{\theta} = \mathbf{0}$. This was demonstrated in the proof of **Lemma 3.4.3**.

Thus, $HJ(\boldsymbol{\theta}) \succ 0$.

By the **Second-Order Sufficient Condition for Strict Convexity**, we conclude that our J is strictly convex on $\mathbb{R}^{p+1}$, and therefore also convex on $\mathbb{R}^{p+1}$. By **Theorem B.3.3 (i)**, the unconstrained local minimiser for f is a global minimiser. Finally, we remark that the conditions of **Theorem B.5.2** are satisfied for $\Omega = \mathbb{R}^{p+1}$ so $\boldsymbol{\theta}^*$ is the unique global minimiser for J on $\mathbb{R}^{p+1}$. ■

The assumption that the columns of $\mathcal{X}$ are linearly independent means that there are no redundant features.

In summary, if the columns of the design matrix $\mathcal{X}$ are linearly independent, then $J(\boldsymbol{\theta})$ is strictly convex on $\mathbb{R}^{p+1}$, and *if an optimal solution exists*, then it's the unique global minimiser

$$\boldsymbol{\theta}^* = (\mathcal{X}^\top \mathcal{X})^{-1} \mathcal{X}^\top \mathbf{y}$$

that solves the normal equations $\mathcal{X}^\top \mathcal{X} \boldsymbol{\theta} = \mathcal{X}^\top \mathbf{y}$.

This conclusion came with the proviso that an optimal solution exists. We haven't (yet) shown this is the case.

3.4.2 IS THERE ALWAYS A SOLUTION TO THE NORMAL EQUATIONS?

Let's look more closely at the normal equations.

$$\mathcal{X}^\top \mathcal{X} \boldsymbol{\theta} = \mathcal{X}^\top \mathbf{y}$$

When does a solution exist? On the left, we have $\mathcal{X}^\top \mathcal{X}$ acting on $\boldsymbol{\theta}$, and outputting $\mathcal{X}^\top \mathbf{y}$. We can view this as the classical linear algebra problem of solving $A\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$. Equivalently¹³, this system tells us that $\mathbf{b}$ is in the column space of A so solving this system of equations means writing $\mathbf{b}$ as a linear combination of the columns of A . Thus, a solution to our normal equations exists iff $\mathcal{X}^\top \mathbf{y}$ is in the column space of $\mathcal{X}^\top \mathcal{X}$. This is indeed the case because:

Lemma 3.4.4 The column spaces of $\mathcal{X}^\top \mathcal{X}$ and $\mathcal{X}^\top$ are equal.

Proof.

⊆ This inclusion is straight-forward. Let $\mathbf{b} \in \text{colspace}(\mathcal{X}^\top \mathcal{X})$ i.e. there exists some $\mathbf{v} \in \mathbb{R}^{p+1}$ s.t. $\mathcal{X}^\top \mathcal{X} \mathbf{v} = \mathbf{b}$. Now let $\mathbf{w} = \mathcal{X} \mathbf{v} \in \mathbb{R}^{p+1}$, and so we've shown that there exists some $\mathbf{w} \in \mathbb{R}^n$ s.t. $\mathcal{X}^\top \mathbf{w} = \mathbf{b}$ i.e. $\mathbf{b} \in \text{colspace}(\mathcal{X}^\top)$. Thus, $\text{colspace}(\mathcal{X}^\top \mathcal{X}) \subseteq \text{colspace}(\mathcal{X}^\top)$.

⊇ For the reverse inclusion, let $\mathbf{b} \in \text{colspace}(\mathcal{X}^\top)$ i.e. $\exists \mathbf{u} \in \mathbb{R}^{n,1}$ s.t. $\mathcal{X}^\top \mathbf{u} = \mathbf{b}$. We wish to show that $\mathbf{b} \in \text{colspace}(\mathcal{X}^\top \mathcal{X})$. Observe¹⁴ that $\mathbb{R}^{n,1}$ admits the orthogonal decomposition

$$\mathbb{R}^{n,1} = \text{colspace}(\mathcal{X}) \oplus \text{nullspace}(\mathcal{X}^\top).$$

¹³As evidenced by our earlier calculation that

$$A\mathbf{x} = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix} x_1 + \cdots + \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{mn} \end{bmatrix} x_n.$$

¹⁴This fact is derived by considering the orthogonal projection of $\mathbb{R}^{n,1}$ onto $\text{nullspace}(\mathcal{X}^\top)$.

Decompose $\mathbf{u} = \mathbf{u}_1 + \mathbf{u}_2$ where $\mathbf{u}_1 \in \text{colspace}(\mathcal{X})$, $\mathbf{u}_2 \in \text{nullspace}(\mathcal{X}^\top)$. Then

$$\begin{aligned}
 \mathbf{b} &= \mathcal{X}^\top \mathbf{u} \\
 &= \mathcal{X}^\top (\mathbf{u}_1 + \mathbf{u}_2) \\
 &= \mathcal{X}^\top \mathbf{u}_1 + \mathcal{X}^\top \mathbf{u}_2 \\
 &= \mathcal{X}^\top \mathbf{u}_1 + \mathbf{0}_{\mathbb{R}^{p+1}} \\
 &= \mathcal{X}^\top (\mathcal{X} \mathbf{v}) \quad \text{for some } \mathbf{v} \in \mathbb{R}^{p+1} \text{ s.t. } \mathcal{X} \mathbf{v} = \mathbf{u}_1
 \end{aligned}$$

i.e. $\mathbf{b} \in \text{colspace}(\mathcal{X}^\top \mathcal{X})$.

■

Thus, there always exists a solution to the normal equations. In the case that $\mathcal{X}$ is of full-rank, there is a unique global solution to the optimisation problem. Otherwise, $\mathcal{X}$ is not of full-rank i.e. $\text{rank}(\mathcal{X}) < \min(n, p+1)$. Note that n is typically much larger than $p+1$ so $\min(n, p+1) = p+1$ and so $\mathcal{X}$ not being of full rank means that $\text{rank}(\mathcal{X}) < p+1$. By the rank-nullity theorem,

$$\text{rank}(\mathcal{X}) + \dim \text{nullspace}(\mathcal{X}) = p+1$$

so the dimension of the nullspace of $\mathcal{X}$ is strictly greater than 0 i.e. the nullspace of $\mathcal{X}$ is non-trivial. Let $\boldsymbol{\theta}^*$ be any optimal solution of the normal equations. Then we may take any non-zero vector $\boldsymbol{\varphi}$ in the nullspace of $\mathcal{X}$ and observe that $\boldsymbol{\theta}^* + \boldsymbol{\varphi}$ also satisfies the normal equations

$$\mathcal{X}^\top \mathcal{X}(\boldsymbol{\theta}^* + \boldsymbol{\varphi}) = \mathcal{X}^\top \mathcal{X} \boldsymbol{\theta}^* + \mathcal{X}^\top (\mathcal{X} \boldsymbol{\varphi}) = \mathcal{X}^\top \mathbf{y} + \mathcal{X}^\top \mathbf{0} = \mathcal{X}^\top \mathbf{y}.$$

Thus, there are infinitely many solutions to the normal equations.

Which solution does one typically choose out of the infinitely many? Possibly a solution with the minimum norm? I'm leaving a blank space for this to come back to because it feels important.

There are other methods to approximate $\hat{\boldsymbol{\theta}}$. The following two algorithms are *iterative*:

3.5 Iterative Methods

3.5.1 GRADIENT DESCENT

Gradient descent is an iterative searching algorithm that searches for an optimal solution (minimiser) $\hat{\theta}$ that minimises J . It begins by initialising some θ and repeatedly performs the *simultaneous* update

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad \text{for } j = 0, \dots, p.$$

We can re-write this update in vector form

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta).$$

Remarks 3.5.1

- I use the symbol $\leftarrow$ exclusively for assignment in the same sense as using $=$ when programming in Python e.g. `x = 2` assigns the integer 2 to the variable `x`.
- Each iteration of this algorithm moves every θ_j in the direction of steepest decrease of $J(\theta)$. This is because each partial derivative ∂_{θ_j} is the j^{th} coordinate of the gradient of J (with respect to θ) which is the direction of greatest increase of J .
- The constant α is called the **learning rate** and determines how “large” the step is in each iteration.
 - If all the features are normalised to $[-1, 1]$, then $\alpha = 0.01$ is a sensible value to start with.

The j^{th} coordinate of the gradient of J with respect to θ is

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{1}{2} \frac{\partial}{\partial \theta_j} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 \\ &= \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) \frac{\partial}{\partial \theta_j} \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) \\ &= \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}. \end{aligned}$$

Thus, the update rule looks at every example at each iteration and is called the **batch gradient descent update rule**:

$$\theta_j \leftarrow \theta_j - \alpha \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \quad \text{for } j = 0, \dots, p \quad \text{simultaneously.}$$

The batch gradient descent algorithm is as follows:

Algorithm 1 Batch gradient descent

```

1: procedure BATCH GRADIENT DESCENT
2:    $\theta \leftarrow \theta_0$  ▷ Initialise a guess for  $\theta$ 
3:   while convergence criterion do
4:      $\theta_j \leftarrow \theta_j - \alpha \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$  ▷ for every  $j$ 
5:   end while
6:   return  $\theta$ 
7: end procedure

```

The magnitude of the update of θ_j is proportional to the error term $h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}$. If the algorithm encounters an example for which the prediction nearly matches the actual value of $y^{(i)}$, then $(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \approx 0$ and it finds little need to change the parameters.

In general, gradient descent can be susceptible to converging to local minima. This is because the algorithm makes locally optimal choices at each step/iteration. However, the way this particular OLS optimisation problem was set up guarantees convergence to a unique global minimiser (assuming correct tuning of α so our updates don't consistently overshoot — α is typically too large in this case). J has a unique global minimiser.

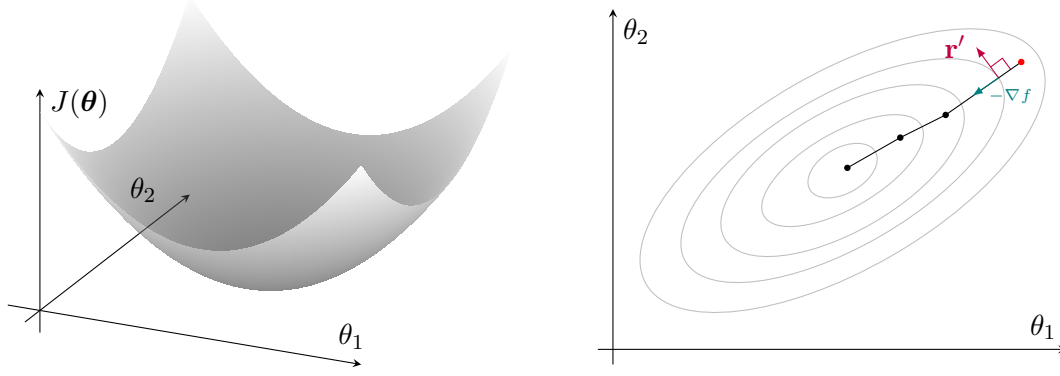


Figure 3.1: A paraboloid-shaped surface (left) with height $J(\theta)$ parameterised by $\theta = [\theta_0 \ \theta_1 \ \theta_2]^\top$ and its level sets/contours (right). The bias term θ_0 represents the offset from the θ_1 - θ_2 plane. The contours for constant values of $J(\theta)$ are ellipses.

The direction of steepest ascent of f (i.e. the gradient ∇f) is always orthogonal to the level set $f(\theta_1, \theta_2) = k$ of a surface i.e. for a parameterisation $\mathbf{r}$ of the level set, $\langle \nabla f, \mathbf{r}' \rangle = 0$. In the diagram, we follow $-\nabla f$ when updating our parameters. If α is too small, the algorithm will require far too many iterations. If the value of $J(\theta)$ increases then there's a very strong chance that α is too large and the minimiser has been overshoot. A disadvantage of batch gradient descent is that a single iteration can become very slow if the number n of training examples is very large.

3.5.2 STOCHASTIC GRADIENT DESCENT

Instead of scanning through all examples in a single iteration, a less computationally expensive option is to repeatedly loop over the training set $i = 1, \dots, n$, at each step using only the current example $(\mathbf{x}^{(i)}, y^{(i)})$ to update the parameters.

The update rule for a single example $(\mathbf{x}^{(i)}, y^{(i)})$ is called the Widrow-Hoff learning rule:

$$\theta_j \leftarrow \theta_j - \alpha \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

The stochastic gradient descent algorithm is as follows:

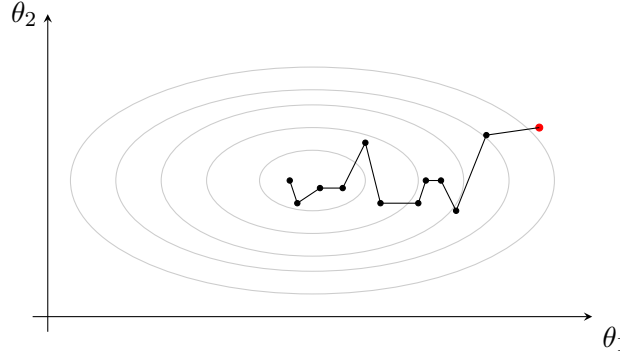
Algorithm 2 Stochastic Gradient Descent

```

1: procedure STOCHASTIC GRADIENT DESCENT
2:    $\theta \leftarrow \theta_0$  ▷ Initialise a guess for  $\theta$ 
3:   for  $i \leftarrow 1, n$  do
4:      $\theta_j \leftarrow \theta_j - \alpha \left( h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$  ▷ for every  $j$ 
5:   end for
6:   return  $\theta$ 
7: end procedure

```

Since the algorithm uses the derivative for one example in each iteration, the parameters θ are updated and improved slightly but may not be in the most direct direction downhill for J .



To this end, stochastic gradient descent takes a noisy route but on average our parameters θ get closer to a minimiser with each iteration. Convergence is not guaranteed for this algorithm and θ will often keep oscillating around a minimiser of $J(\theta)$. Decreasing the learning rate α in the later stages of the algorithm could yield good results.

3.6 Normality Assumptions (Ordinary Linear Regression)

A probabilistic assumption reveals that the ordinary least-squares cost function $J(\theta)$ is the one we should be minimising because it naturally arises through the method of maximum likelihood estimation. We add this assumption to the linear regression model:

5. Normality of errors:

The distribution of $\varepsilon = (\varepsilon^{(1)}, \dots, \varepsilon^{(n)})$ conditional on $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}$ is jointly normal $\mathcal{N}(\mathbf{0}, \sigma^2 I_n)$.

With the inclusion of joint normality, a linear regression model is termed a **Gaussian linear regression model**.

We always assume that our random variables map into nice enough spaces (e.g. Borel spaces) s.t. there exists a system of regular conditional probabilities of $\varepsilon \in \text{Meas}_{\mathcal{F}, \mathcal{B}_{\mathbb{R}^n}}(\Omega; \mathbb{R}^n)$ given $\mathbf{X} := (\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}) \in \text{Meas}_{\mathcal{F}, \mathcal{B}_{(\mathbb{R}^{(p+1)})^n}}(\Omega; (\mathbb{R}^{(p+1)})^n)$ i.e. a map $\kappa_{\mathbf{X}}^{\varepsilon}: \mathcal{B}_{\mathbb{R}^n} \times (\mathbb{R}^{(p+1)})^n \rightarrow [0, 1]$ s.t.

1. For every $\mathbf{x} = (x^{(1)}, \dots, x^{(n)}) \in (\mathbb{R}^{(p+1)})^n$, $\kappa_{\mathbf{x}}^{\varepsilon}(\cdot, \mathbf{x})$ is a probability measure on the codomain $(\mathbb{R}^n, \mathcal{B}_{\mathbb{R}^n})$ of ε .
2. For each $B \in \mathcal{B}_{\mathbb{R}^n}$, the mapping $\mathbf{x} \mapsto \kappa_{\mathbf{x}}^{\varepsilon}(B, \mathbf{x})$ is:

- an $\mathcal{B}_{(\mathbb{R}^{(p+1)})^n}$ -measurable function satisfying for $\mathbb{P}$ -a.e. $\omega \in \Omega$:

$$\mathbb{P}[\varepsilon^{-1}(B) \mid \sigma(\mathbf{X})](\omega) = \kappa_{\mathbf{X}(\omega)}^{\varepsilon}(B, \mathbf{X}(\omega)),$$

- and is $\mathbb{P}_{\mathbf{X}}$ -integrable, satisfying for all $B \in \mathcal{B}_{\mathbb{R}^n}$ and $D \in \mathcal{B}_{(\mathbb{R}^{(p+1)})^n}$:

$$\mathbb{P}(\varepsilon^{-1}(B) \cap \mathbf{X}^{-1}(D)) = \int_D \kappa_{\mathbf{x}}^{\varepsilon}(B, \mathbf{x}) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}).$$

The conditional normality means that $\kappa_{\mathbf{x}}^{\varepsilon}$ satisfies for every $B \in \mathcal{B}_{\mathbb{R}^n}$:

$$\kappa_{\mathbf{x}}^{\varepsilon}(B, \mathbf{x}) = \gamma_{n, \sigma^2}(B) \quad \text{for } \mathbb{P}_{\mathbf{X}}\text{-a.e. } \mathbf{x} \in (\mathbb{R}^{(p+1)})^n,$$

where φ_{n,σ^2} is a multivariate normal probability distribution. In other words, the conditional law of ε given $\mathbf{X}$ has a multivariate Gaussian density with mean $\mathbf{0}$ and covariance matrix $\sigma^2 I_n$. Also, since there's no dependence of φ_{n,σ^2} on $\mathbf{x}$, our disintegration formula becomes

$$\mathbb{P}(\varepsilon^{-1}(B) \cap \mathbf{X}^{-1}(D)) = \int_D \gamma_{n,\sigma^2}(B) d\mathbb{P}_{\mathbf{X}}(\mathbf{x}) = \gamma_{n,\sigma^2}(B) \mathbb{P}_{\mathbf{X}}(D)$$

and this factorisation demonstrates that ε and $\mathbf{X}$ are independent.

3.6.1 CONSEQUENCES OF JOINT CONDITIONAL NORMALITY OF OUR ERRORS

The assumption of joint conditional normality of our errors implies several statements:

Corollary 3.6.1 Each $\varepsilon^{(i)}$ is conditionally normally distributed given $\mathbf{X}^{(i)}$ with the same mean 0 and variance σ^2 .

Proof. The idea is to push the conditional law of ε given $\mathbf{X}$ forward via the natural coordinate projection map π_i . Indeed, for each $\mathbf{x} \in (\mathbb{R}^{(p+1)})^n$:

$$\kappa_{\mathbf{X}}^{\varepsilon^{(i)}}(\cdot, \mathbf{x}) := (\pi_i)_\# \kappa_{\mathbf{X}}^{\varepsilon}(\cdot, \mathbf{x}) = (\pi_i)_\# \gamma_{n,\sigma^2}(\cdot) = \gamma_{1,\sigma^2}(\cdot)$$

because the marginal distribution of a multivariate normal is univariate normal with variance the $(i, i)^{\text{th}}$ entry in the covariance matrix. ■

Corollary 3.6.2 The $\varepsilon^{(i)}$ are conditionally independent given $\mathbf{X}$.

Proof. This follows from the previous corollary by noting that for every $\mathbf{x}$ the joint conditional law of ε given $\mathbf{X}$ factorises into the following product measure of every marginal conditional distribution of $\varepsilon^{(i)}$ given $\mathbf{X}$ i.e.

$$\kappa_{\mathbf{X}}^{\varepsilon}(\cdot, \mathbf{x}) = \gamma_{n,\sigma^2} = \bigotimes_{i=1}^n \gamma_{1,\sigma^2}(\cdot) = \bigotimes_{i=1}^n \kappa_{\mathbf{X}}^{\varepsilon^{(i)}}(\cdot, \mathbf{x}).$$

■

Corollary 3.6.3 There are no conditional cross-variance terms.

Proof. I imagine this comes from the covariance matrix entries outside of the main-diagonal being 0. ■

Corollary 3.6.4 The distribution of $\mathbf{Y} = \Phi(\mathbf{X})\boldsymbol{\theta} + \varepsilon$ conditional on $\mathbf{X}$ is normal with mean $\Phi(\mathbf{X})\boldsymbol{\theta}$ and covariance matrix $\sigma^2 I_n$.

Proof. According to Section 18.3.2 from 103 [1, p. 205], we can in a sense “push-forward” the regular conditional probability of ε given $\mathbf{X}$ by a jointly-measurable map in order to obtain the conditional law of $\mathbf{Y}$.

Definition 3.6.5 Let $(E_X, \mathcal{E}_X)$ and $(E_Y, \mathcal{E}_Y)$ be measurable spaces. A **Markov (or transition) kernel from E_X to E_Y** is a map $\kappa: E_X \times \mathcal{E}_Y \rightarrow \mathbb{R}$ s.t.

- for every $x \in E_X$, $\kappa(\cdot, x)$ is a probability measure on $(E_Y, \mathcal{E}_Y)$, and
- for every $B \in \mathcal{E}_Y$, $\kappa(B, \cdot)$ is $\mathcal{E}_X$ -measurable.

Theorem 3.6.6 Given a jointly measurable map $Z: E_Y \times E_X \rightarrow E_Z$, the function $\kappa^Z: \mathcal{E}_Z \times E_X \rightarrow [0, 1]$ defined for $B \in \mathcal{E}_Z$ and $x \in E_X$ by

$$\kappa^Z(B, x) := \kappa(\{y \in E_Y : Z(y, x) \in B\}, x)$$

is a Markov kernel from E_X to E_Z .

It suffices to check that the map which takes ϵ and $\mathbf{X}$ to $\mathbf{Y}$ is jointly-measurable. Indeed, let¹⁵

$$Z: \underbrace{\mathbb{R}^n}_{\epsilon} \times \underbrace{(\mathbb{R}^{(p+1)})^n}_{\mathbf{X}} \rightarrow \underbrace{\mathbb{R}^n}_{\mathbf{Y}}$$

be the affine transformation

$$\mathbf{y} = Z(\epsilon, \mathbf{x}) = \Phi(\mathbf{x})\boldsymbol{\theta} + \epsilon.$$

This is clearly jointly measurable. Thus, the conditional law of $\mathbf{Y}$ given $\mathbf{X}$ is the regular conditional probability $\kappa_{\mathbf{X}}^{\mathbf{Y}}: \mathcal{B}_{\mathbb{R}^n} \times (\mathbb{R}^{(p+1)})^n \rightarrow [0, 1]$ defined for any $B \in \mathcal{B}_{\mathbb{R}^n}$ and $\mathbf{x} \in (\mathbb{R}^{(p+1)})^n$ by

$$\begin{aligned} \kappa_{\mathbf{X}}^{\mathbf{Y}}(B, \mathbf{x}) &:= \kappa_{\mathbf{X}}^{\epsilon}((Z \circ \iota_{\mathbf{x}})^{-1}(B), \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\epsilon}(\{\epsilon \in \mathbb{R}^n : (Z \circ \iota_{\mathbf{x}})(\epsilon) \in B\}, \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\epsilon}(\{\epsilon \in \mathbb{R}^n : Z(\epsilon, \mathbf{x}) \in B\}, \mathbf{x}) \\ &= \kappa_{\mathbf{X}}^{\epsilon}(\{\epsilon \in \mathbb{R}^n : \Phi(\mathbf{x})\boldsymbol{\theta} + \epsilon \in B\}, \mathbf{x}) \\ &= \gamma_{n, \sigma^2}(\{\epsilon \in \mathbb{R}^n : \Phi(\mathbf{x})\boldsymbol{\theta} + \epsilon \in B\}) \\ &= \int_{\{\epsilon \in \mathbb{R}^n : \Phi(\mathbf{x})\boldsymbol{\theta} + \epsilon \in B\}} \varphi_{n, \sigma^2}(\epsilon) d\lambda_{\mathbb{R}^n}(\epsilon) \\ &= \int_{h^{-1}(B)} \varphi_{n, \sigma^2}(\epsilon) d\lambda_{\mathbb{R}^n}(\epsilon) \quad \text{where } \mathbf{u} = h(\epsilon) := \Phi(\mathbf{x})\boldsymbol{\theta} + \epsilon \\ &= \int_B \varphi_{n, \sigma^2}(\mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}) |\det(J_{h^{-1}})(\mathbf{u})| d\lambda_{\mathbb{R}^n}(\mathbf{u}) \\ &= \int_B \varphi_{n, \sigma^2}(\mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}) d\lambda_{\mathbb{R}^n}(\mathbf{u}) \\ &= \mathcal{N}(\Phi(\mathbf{x})\boldsymbol{\theta}, \sigma^2 I_n)(B) \end{aligned}$$

The penultimate line's Jacobian determinant of $h^{-1}(\mathbf{u}) = \mathbf{u} - \Phi(\mathbf{x})\boldsymbol{\theta}$ is simply 1 because it's a translation. ■

The final piece of the puzzle follows.

Corollary 3.6.7 The $Y^{(i)}$ are conditionally independent given $\mathbf{X}$, and each $Y^{(i)}$ is conditionally normally distributed given $\mathbf{X}$ with mean $\Phi(\mathbf{x}^{(i)})\boldsymbol{\theta}$ and variance σ^2

Proof. The proof follows in an entirely analogous fashion to the proof of the independence of the $\epsilon^{(i)}$ conditional on $\mathbf{X}$. This observation is crucial for the following section. ■

We may also observe that

$$\Phi(\mathbf{x}^{(i)})\boldsymbol{\theta} = (\mathbf{x}^{(i)})^\top \boldsymbol{\theta} = \boldsymbol{\theta}^\top \mathbf{x}^{(i)}$$

so for notational simplicity we may simply write $\boldsymbol{\theta}^\top \mathbf{x}^{(i)}$.

¹⁵The underbraces denote which random elements these inputs and outputs correspond to.

By some leap of faith, I believe the conditional law of $Y^{(i)}$ given $\mathbf{X}^{(i)}$ is the same as that of $Y^{(i)}$ given $\mathbf{X}$. If this is true, then one has the disintegration formula for any $B \in \mathcal{B}_{\mathbb{R}}$ and $D \in \mathcal{B}_{\mathbb{R}^{(p+1)}}$:

$$\begin{aligned} \mathbb{P}\left((Y^{(i)})^{-1}(B) \cap (\mathbf{X}^{(i)})^{-1}(D)\right) &= \int_D \kappa_{\mathbf{X}^{(i)}}^{Y^{(i)}}(B, \mathbf{x}) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}) \\ &= \int_D \mathcal{N}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}, \sigma^2)(B) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}) \\ &= \int_D \int_B \varphi_{1, \boldsymbol{\theta}^\top \mathbf{x}^{(i)}, \sigma^2}(y) \, d\lambda(y) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}). \end{aligned}$$

3.6.2 MAXIMUM LIKELIHOOD ESTIMATION

We begin with the assumption that our data $\mathcal{T}$ has already been observed from a random sample

$$(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{\boldsymbol{\theta}}$$

whose distribution is a particular parametric distribution $\mathbb{P}_{\boldsymbol{\theta}}$ from a family $\{\mathbb{P}_{\boldsymbol{\theta}}\}_{\boldsymbol{\theta} \in \Theta}$, where Θ is some indexing set. For the purposes of what follows, we assume that this family of probability measures is dominated by a σ -finite measure ν . In the discrete case, ν will be the counting measure; and in the absolutely continuous case, ν is the Lebesgue measure. This domination assumption allows us to speak of the¹⁶ density of $\mathbb{P}_{\boldsymbol{\theta}}$, and therefore speak of the central object of this section.

The method of maximum likelihood estimation is a search for optimal parameters $\boldsymbol{\theta}' \in \Theta$ for which it was most likely that our data had been sampled from $\mathbb{P}_{\boldsymbol{\theta}'}$. We do this by working with a [surrogate](#) quantity called the joint likelihood L of our data. We view it as a function of $\boldsymbol{\theta}$ in order to carry out the aforementioned optimisation.

Definition 3.6.8 The **joint likelihood function for $\boldsymbol{\theta}$ given our data $\mathcal{T}$** is the density of the joint distribution of our sample (with respect to a dominating measure ν) evaluated at the observed value $\mathcal{T}$.

$$L(\boldsymbol{\theta}) = \left. \frac{d\mathbb{P}_{(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)})}}{d\nu} \right|_{\mathcal{T}}.$$

In our case, our random sample is i.i.d. and made up of (absolutely) continuous random vectors (with respect to λ), so we may take $\nu = \otimes^n \lambda$ and obtain¹⁷ the following expression:

$$L(\boldsymbol{\theta}) = \left. \frac{d\mathbb{P}_{(\mathbf{X}^{(1)}, Y^{(1)}), \dots, (\mathbf{X}^{(n)}, Y^{(n)})}}{d\otimes^n \lambda} \right|_{\mathcal{T}} \stackrel{\text{i.i.d.}}{=} \left. \left(\prod_1^n \frac{d\mathbb{P}_{\boldsymbol{\theta}}}{d\lambda} \right) \right|_{\mathcal{T}} = \prod_{i=1}^n f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, y^{(i)})$$

¹⁶Technically this is defined up to ν -null sets but it's not a massive abuse of language to say 'the' with the tacit assumption that we're referring to a representative of an equivalence class of functions that differ ν -a.e.!

¹⁷I'm leaving out some details that I expect are true but not quite sure e.g. if $\mu_1, \dots, \mu_n$ are measures, and $\nu_1, \dots, \nu_n$ is a collection of σ -finite measures s.t for every i we have $\mu_i \ll \nu_i$, then

$$\frac{d(\mu_1 \otimes \dots \otimes \mu_n)}{d(\nu_1 \otimes \dots \otimes \nu_n)} = \prod_{i=1}^n \frac{d\mu_i}{d\nu_i}.$$

I think it's important to emphasise that the likelihood is not a product of probabilities. Instead, it's a Radon-Nikodym derivative of the (joint) density of the random variable of interest with respect to a (product) dominating measure. In our case with a random sample, the independence implies that our derivative becomes a product of densities. In the case that the original random variable of interest is discrete, the density is indeed the probability mass function (and hence the probability itself). This means that the likelihood is the probability but simply viewed as a function of the parameters. In the general case, there is no such correspondence.



From the disintegration formula earlier, if we assume that $\mathbf{X}^{(i)}$ has density $f_{\mathbf{X}^{(i)}, \boldsymbol{\theta}}$, then we have the following equality for any $B \in \mathcal{B}_{\mathbb{R}}$ and $D \in \mathcal{B}_{\mathbb{R}^{(p+1)}}$:

$$\begin{aligned} \mathbb{P}\left((Y^{(i)})^{-1}(B) \cap (\mathbf{X}^{(i)})^{-1}(D)\right) &= \int_D \kappa_{\mathbf{X}^{(i)}}^{Y^{(i)}}(B, \mathbf{x}) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}) \\ &= \int_D \mathcal{N}(\Phi(\mathbf{x}^{(i)})\boldsymbol{\theta}, \sigma^2)(B) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}) \\ &= \int_D \int_B \varphi_{1, \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta}, \sigma^2}(y) \, d\lambda(y) \, d\mathbb{P}_{\mathbf{X}^{(i)}}(\mathbf{x}) \\ &= \int_D \int_B \varphi_{1, \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta}, \sigma^2}(y) \, d\lambda(y) f_{\mathbf{X}^{(i)}, \boldsymbol{\theta}}(\mathbf{x}) \, d\lambda(\mathbf{x}) \\ &= \int_D \int_B \varphi_{1, \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta}, \sigma^2}(y) f_{\mathbf{X}^{(i)}, \boldsymbol{\theta}}(\mathbf{x}) \, d\lambda(y) \, d\lambda(\mathbf{x}) \end{aligned}$$

which demonstrates the **joint density** $f_{\boldsymbol{\theta}}$ of $(\mathbf{X}^{(i)}, Y^{(i)})$ factorises into the product of the conditional density of $\kappa_{\mathbf{X}^{(i)}}^{Y^{(i)}}$ (which is the normal) and the marginal density of $\mathbf{X}^{(i)}$.

Therefore, the likelihood takes the form:

$$\begin{aligned} L(\boldsymbol{\theta}) &= \dots = \prod_{i=1}^n f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}, y^{(i)}) \\ &= \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta})^2}{2\sigma^2}\right) f_{\mathbf{X}^{(i)}, \boldsymbol{\theta}}(\mathbf{x}^{(i)}) \end{aligned}$$

This factorisation is useful for two reasons:

1. We've only made assumptions about the conditional distribution of our errors from which it follows that each $Y^{(i)}$ is conditionally normally distributed given $\mathbf{X}^{(i)}$.
2. If we assume the marginal density is independent of $\boldsymbol{\theta}$, then optimising $L(\boldsymbol{\theta})$ is an equivalent problem to optimising the **conditional likelihood** L_{cond} defined by

$$L_{\text{cond}}(\boldsymbol{\theta}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta})^2}{2\sigma^2}\right).$$

Densities often take small values, and the conditional likelihood above can be a product of many such small numbers. Multiplying many small numbers together in an algorithm could underflow¹⁸ the computer. We'd prefer to work with a sum instead of a product so need a suitable transformation T . By suitable, we'd like that if $\mathcal{L}(\boldsymbol{\theta})$ is maximised at $\hat{\boldsymbol{\theta}}$, then $T(\mathcal{L}(\boldsymbol{\theta}))$ is also maximised at $\hat{\boldsymbol{\theta}}$. A candidate for T which ticks all the boxes is the natural logarithm $T = \log$. It's monotonically increasing and transforms products into sums.

¹⁸When the result of a calculation is more precise than a computer can represent in memory.

Thus, we typically prefer to use the conditional log likelihood $\ell(\boldsymbol{\theta}) := \log(L_{\text{cond}}(\boldsymbol{\theta}))$:

$$\begin{aligned}\ell(\boldsymbol{\theta}) &= \log \left(\prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta})^2}{2\sigma^2} \right) \right) \\ &= \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta})^2}{2\sigma^2} \right) \right) \\ &= n \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right) - \frac{1}{2\sigma^2} \underbrace{\sum_{i=1}^n (y^{(i)} - \Phi(\mathbf{x}^{(i)})\boldsymbol{\theta})^2}_{= J(\boldsymbol{\theta})}\end{aligned}$$

This means that the problem of maximising $L_{\text{cond}}(\boldsymbol{\theta})$:

- is equivalent to maximising $\ell(\boldsymbol{\theta})$
- which is equivalent to minimising $J(\boldsymbol{\theta})$

which was precisely the ordinary least-squares linear regression problem.

In fact, the overall framework defined by the following two key points

1. Make a set of probabilistic assumptions about the conditional response.
2. Using the method of maximum likelihood estimation, derive a new type of supervised learning algorithm.

is something we'll re-use for fitting parameters for different types of problems. In particular, when the target variable $y \in \mathcal{Y}$ is now either 0 or 1 i.e. a classification problem.

When building learning algorithms, often we make assumptions about the world that we just know aren't 100% true because it leads to algorithms that are computationally efficient.

Andrew Ng

Classification

Let $\mathcal{X} = \mathbb{R}^p$, and $\mathcal{Y} = \{c_1, \dots, c_K\}$ for a finite integer $K \in \mathbb{N}$.

4.1 Linear Classifiers

A **classifier** is a function $h: \mathcal{X} \rightarrow \{c_1, \dots, c_K\}$, where K is the number of classes under consideration. Such functions partition the input feature space $\mathcal{X}$ into K cells¹ whose boundaries are called decision boundaries. If these decision boundaries are linear, we call h a **linear classifier**.

A linear classifier makes decisions by thresholding across linear decision boundaries so it can be thought of as the composition of first performing a linear transformation $T: \mathcal{X} \rightarrow \mathbb{R}^K$ of an input feature vector $\mathbf{x} \in \mathcal{X}$ (which is a way of scoring an input feature), followed by some (activation) thresholding function f to make the decision, i.e. $h = f \circ T$.

4.1.1 AGGREGATION/SCORING T

Theorem 4.1.1 (Theorem 1.3.2 [4, p. 12] **104**) Let U and V be vector spaces over $\mathbb{K}$ of dimensions n and m respectively. Then, for a given choice of bases of U and V , there is a one-to-one linear correspondence between the set $\text{Hom}_{\mathbb{K}}(U; V)$ of linear maps $U \rightarrow V$, and the set $\mathbb{K}^{m,n}$ of m by n matrices over $\mathbb{K}$.

$$\text{Hom}_{\mathbb{K}}(U; V) \cong \mathbb{K}^{m,n}$$

By the above theorem, the unique representative of $T: \mathcal{X} \rightarrow \mathbb{R}^K$ with respect to the standard bases of $\mathbb{R}^p$ and $\mathbb{R}^K$ is the matrix $W \in \mathbb{R}^{K,p}$ i.e. the coordinate representation of T acts as follows

$$T: \mathbf{x} \mapsto W\mathbf{x}.$$

Thus, $h(\mathbf{x}) = f(T(\mathbf{x})) = f(W\mathbf{x})$ and so we may think of h as a map that depends on the subscript W i.e. $h_W(\mathbf{x}) = f(W\mathbf{x})$. The entries $w_{i,j}$ of W are called weights. Each weight places an “importance” on the corresponding feature, and serves as a parameter that can be fine-tuned by an algorithm.

Focusing only on the linear transformation for now, if we think of each row $\mathbf{w}_i \in \mathbb{R}^{1,p}$ of W as the prototypical object/representation of the class c_i , then the i^{th} component of $T(\mathbf{x})$ is given by

$$\sum_{j=1}^p w_{i,j}x_j = \mathbf{w}_i\mathbf{x} = \langle \mathbf{x}, \mathbf{w}_i^\top \rangle.$$

Each inner product $\langle \mathbf{x}, \mathbf{w}_i^\top \rangle$ quantifies how “similar” our example’s feature representation $\mathbf{x}$ is to $\mathbf{w}_i$. Thus, $T(\mathbf{x})$ is a vector of “scores” representing how well $\mathbf{x}$ fits into each and every class.

Some books add a bias term $\mathbf{b} \in \mathbb{R}^K$ to the linear transformation $T(\mathbf{x}) = W\mathbf{x}$ to make an affine transformation $A(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$. Writ-

¹I use this term in the mathematical sense. A **partition** of a set $\mathcal{X}$ is a family of non-empty subsets of $\mathcal{X}$ such that each $x \in \mathcal{X}$ belongs to a unique subset. The subsets in a partition are called **cells**. We say that $\mathcal{X}$ is partitioned into $B_1, \dots, B_K$ if the B_i are such that for $i \neq j$, $B_i \cap B_j = \emptyset$ and

$$\mathcal{X} = \bigsqcup_{i=1}^K B_i.$$

ten explicitly:

$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,p} \\ w_{2,1} & w_{2,2} & \dots & w_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ w_{K,1} & w_{K,2} & \dots & w_{K,p} \end{bmatrix} \begin{bmatrix} x_1^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_K \end{bmatrix} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

Thankfully this can be viewed as a special case of $T(\mathbf{x}) = W\mathbf{x}$ by embedding the input feature vectors into $\mathbb{R}^{p+1}$ with $x_0 = 1$ and appending the bias vector to the left of W :

$$\begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & \dots & w_{1,p} \\ w_{2,0} & w_{2,1} & w_{2,2} & \dots & w_{2,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{K,0} & w_{K,1} & w_{K,2} & \dots & w_{K,p} \end{bmatrix} \begin{bmatrix} 1 \\ x_1^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix} = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

where $b_i = w_{i,0}$ for $i = 1, \dots, K$.

4.1.2 THRESHOLD f

We can take

$$f = \operatorname{argmax}_{i \in \{1, \dots, K\}} T_i(\mathbf{x})$$

as the subscript i of the class label c_i for our prediction. This follows the previous logic that such an i will correspond to the class c_i for which $\mathbf{x}$ is most similar to $\mathbf{w}_i$.

There are certainly other choices of f to define our linear classifier h . Probably something to do with how the decision boundaries are formed; one class vs the rest, pairwise (class vs class) comparisons etc.

The simplest case of classification is between $K = 2$ classes — a **binary classification** task. Our hypothesis $h: \mathcal{X} \rightarrow \{c_1, c_2\} := \{-1, 1\}$ is² called a **binary linear classifier** and takes the form

$$h(\mathbf{x}) = f(T(\mathbf{x})) = f(W\mathbf{x}) = f\left(\begin{bmatrix} \mathbf{w}_1\mathbf{x} \\ \mathbf{w}_2\mathbf{x} \end{bmatrix}\right).$$

The class labels inspire the following nomenclature:

- If $y^{(i)} = +1$, then $(\mathbf{x}^{(i)}, y^{(i)})$ is called a **positive example**.
- If $y^{(i)} = -1$, then $(\mathbf{x}^{(i)}, y^{(i)})$ is called a **negative example**.

Let f return the class label c_i for which $\mathbf{w}_i\mathbf{x}$ is maximised. Then our hypothesis is

$$h(\mathbf{x}) = f(T(\mathbf{x})) = \begin{cases} +1, & \text{if } \mathbf{w}_1\mathbf{x} \geq \mathbf{w}_2\mathbf{x} \\ -1, & \text{if } \mathbf{w}_1\mathbf{x} < \mathbf{w}_2\mathbf{x}. \end{cases}$$

The comparison between $\mathbf{w}_1\mathbf{x}$ and $\mathbf{w}_2\mathbf{x}$ can be written in terms of a single vector

$$\mathbf{w}_1\mathbf{x} - \mathbf{w}_2\mathbf{x} = \underbrace{(\mathbf{w}_1 - \mathbf{w}_2)}_{=: \boldsymbol{\theta}^\top} \mathbf{x},$$

where $\boldsymbol{\theta}$ is a column vector. This emphasises that in the case of $K = 2$ classes, the feature space $\mathcal{X}$ is partitioned into two cells, and the corresponding linear decision boundary is reflected by a single parameter $\boldsymbol{\theta}$.

²Up to re-labelling — we could equally have chosen $\{0, 1\}$ or any other two-element set for our labels $\mathcal{Y}$. This choice of course influences the form that f takes.

4.2 The Deceptron

I'm choosing to use the term '**deceptron**' instead of 'perceptron' in this section to distinguish it from the more modern usage of the term 'perceptron' that refers to any **non-linear activation function** in a single **neuron** of a neural network. This is because I really dislike it when the same word is used to refer to fundamentally different things. *These* are all buzzwords for the time being but will be explained in **Section 6.2**.



Let $K = 2$. Our hypothesis $h: \mathcal{X} \rightarrow \{-1, 1\}$ takes the form

$$h(\mathbf{x}) = f(T(\mathbf{x})) = f(\boldsymbol{\theta}^\top \mathbf{x}) =: h_{\boldsymbol{\theta}}(\mathbf{x}),$$

where we adopt the convention that $x_0 = 1$, and the bias term θ_0 has been incorporated into $\boldsymbol{\theta} \in \mathbb{R}^{p+1}$. Our training set is a collection of examples

$$\mathcal{T} = \left\{ (\mathbf{x}^{(i)}, y^{(i)}) \right\}_{i=1}^n \subseteq \mathbb{R}^p \times \{-1, +1\}.$$

We can plot the input feature vectors $\mathbf{x}^{(i)}$ as points in $\mathbb{R}^p$, and represent each point with a shape corresponding to its label $y^{(i)}$. Geometrically, we wish to learn a linear decision boundary that separates $\mathcal{X}$ into two subsets; one containing only inputs $\mathbf{x}^{(i)}$ with a positive label, and its complement containing only inputs $\mathbf{x}^{(i)}$ with a negative label. This linear decision boundary is a **hyperplane**³ of $\mathbb{R}^p$ i.e. a subset of $\mathbb{R}^p$ (with codimension 1) whose points satisfy the equation $\langle \mathbf{x}, \boldsymbol{\theta} \rangle := \boldsymbol{\theta}^\top \mathbf{x} = 0$ for $\mathbf{0} \neq \boldsymbol{\theta} \in \mathbb{R}^{p+1}$.

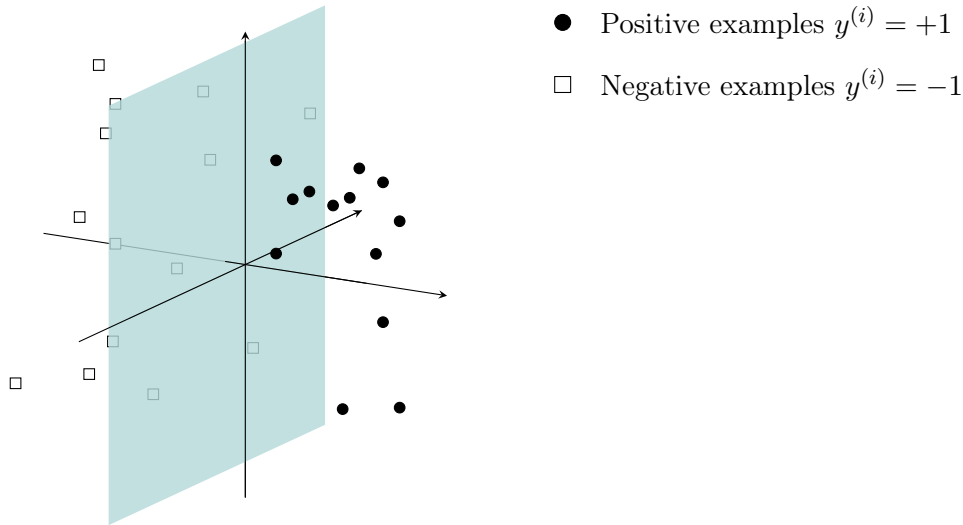


Figure 4.1: An example of a perfectly linearly separable dataset in $\mathbb{R}^3$ with a separating hyperplane decision boundary (teal).

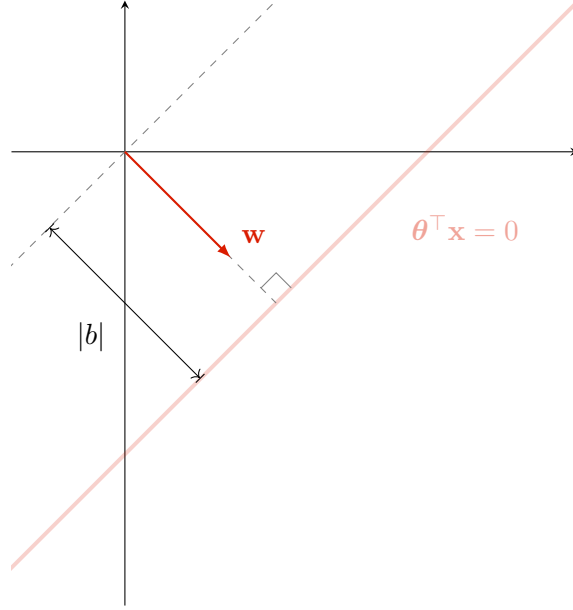
This means that our decision hyperplane (which passes through the origin) is the collection of points orthogonal to $\boldsymbol{\theta}$.

Some books make the geometry more explicit by separating $\boldsymbol{\theta} \in \mathbb{R}^{p+1}$ into

$$\text{weights } \mathbf{w} = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} \in \mathbb{R}^p \text{ and a bias (offset) term } b = \theta_0.$$

³In general a hyperplane has exactly two unit normals $\pm \hat{\mathbf{n}}$, and an **affine hyperplane** of $\mathbb{R}^p$ with normal vector $+\hat{\mathbf{n}}$ and origin translation $\mathbf{b} \in \mathbb{R}^n$ is defined to be the set of all $\mathbf{x} \in \mathbb{R}^p$ s.t. $\hat{\mathbf{n}} \cdot (\mathbf{x} - \mathbf{b}) = 0$.

The weights are a vector $\mathbf{w}$ that determines the “slope”/shape of the hyperplane (vectors on the hyperplane are orthogonal to $\mathbf{w}$), and b is the offset of the hyperplane from the origin in the direction opposite to $\mathbf{w}$. In $\mathbb{R}^2$ the picture is very clear:



This will be useful in the formulation of the support vector machine (see **Chapter 5**).

This divides $\mathbb{R}^p$ into two **half-spaces**.

- If an example $(\mathbf{x}^{(i)}, y^{(i)})$ satisfies $\theta^\top \mathbf{x}^{(i)} \geq 0$, then $\mathbf{x}^{(i)}$ is in the half-space in which our choice of θ directs into. This is because the inner product $\langle \mathbf{x}^{(i)}, \theta \rangle = \theta^\top \mathbf{x}^{(i)}$ is a measure of similarity between the two vectors. Assign the label $+1$ to such an $\mathbf{x}^{(i)}$.
- Otherwise, $\langle \mathbf{x}^{(i)}, \theta \rangle < 0$ and we shall assign it the value -1 .

These considerations define our hypothesis as

$$h_\theta(\mathbf{x}) = \text{sgn}(\theta^\top \mathbf{x}) = \begin{cases} +1, & \text{if } \theta^\top \mathbf{x} \geq 0 \\ -1, & \text{if } \theta^\top \mathbf{x} < 0. \end{cases}$$

This is the link between linear binary classifiers and decision hyperplanes.

4.2.1 SETUP

Given the last section, we know that finding an optimal decision boundary that minimises misclassifications over our training set is tantamount to finding a vector of optimal parameters θ . Searching for an appropriate θ will follow the same general structure as the previous chapters.

Algorithm 3 A single epoch of the training step in the single-layer perceptron algorithm

- 1: $\theta \leftarrow \mathbf{0}$ ▷ Initialise θ
 - 2: **for** $i = 1, \dots, n$ **do**
 - 3: Update $\theta \leftarrow \tilde{\theta}$
 - 4: **end for**
 - 5: **return** $\tilde{\theta}$
-

Consider a particular training example $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{T}$.

- If $(\mathbf{x}^{(i)}, y^{(i)})$ is correctly classified by $h_{\boldsymbol{\theta}}$, either:

- $y^{(i)} = +1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = +1$, or
- $y^{(i)} = -1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -1$.

i.e. $y^{(i)}h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = +1$.

- If $(\mathbf{x}^{(i)}, y^{(i)})$ is misclassified by $h_{\boldsymbol{\theta}}$, either:

- $y^{(i)} = +1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -1$, or
- $y^{(i)} = -1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = +1$.

i.e. $y^{(i)}h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -1$

It follows that our loss function is very straight-forward to define.

Definition 4.2.1

- For each example, we define the “**hard**” loss of our hypothesis $h_{\boldsymbol{\theta}}$ by

$$\mathcal{L}_{\text{hard}}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}) = \mathbb{1}\{h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \neq y^{(i)}\}.$$

- The associated empirical risk (which averages the hypothesis’ loss over $\mathcal{T}$) is given by

$$R_{\text{emp.}}(h_{\boldsymbol{\theta}}) := \frac{1}{n} \sum_{i \in \mathcal{M}} \mathcal{L}_{\text{hard}}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}\{h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \neq y^{(i)}\}.$$

That final term is precisely the misclassification rate of our hypothesis with respect to $\mathcal{L}_{\text{hard}}$. Naturally, we’d like to penalise misclassifications so everything checks out with the framework in Chapter 2 — the empirical risk is the quantity we wish to minimise by adjusting $\boldsymbol{\theta}$ in search of an optimal hypothesis $h_{\boldsymbol{\theta}}$ i.e. we want to find

$$\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^n \mathbb{1}\{h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \neq y^{(i)}\}.$$

Equivalently, finding such parameters $\boldsymbol{\theta}$ means that we maximise the number of correctly-classified (i.e. $y^{(i)}h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) > 0$) training examples.

How do we search for such parameters? The first thought is to spam gradient descent on line 3 in the vague general structure of **Algorithm 3**. The indicator function isn’t differentiable in any neighbourhood of its discontinuity. Too bad. No gradient descent for us!

Let’s conduct a little thought experiment to side-step this difficulty.

4.2.2 SURROGATE DECEPTRON PROBLEM

Suppose that we chose a differentiable function f instead of $\operatorname{sgn}(\mathbf{x})$ in our hypothesis, namely $f = \operatorname{id}_{\mathbb{R}}$. Then $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \boldsymbol{\theta}^{\top} \mathbf{x}^{(i)}$ would’ve been differentiable. We call this particular hypothesis the **unbounded linear unit**. Suppose further that instead of $\mathcal{L}_{\text{hard}}$, we chose a “softer” loss.

Definition 4.2.2 The **deceptron loss**⁴ is defined by

$$\mathcal{L}_{\text{soft}}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}) = \max(0, -y^{(i)}\boldsymbol{\theta}^{\top} \mathbf{x}^{(i)}).$$

⁴A variant $\max(1 - s, 0)$ is called the Hinge Loss.

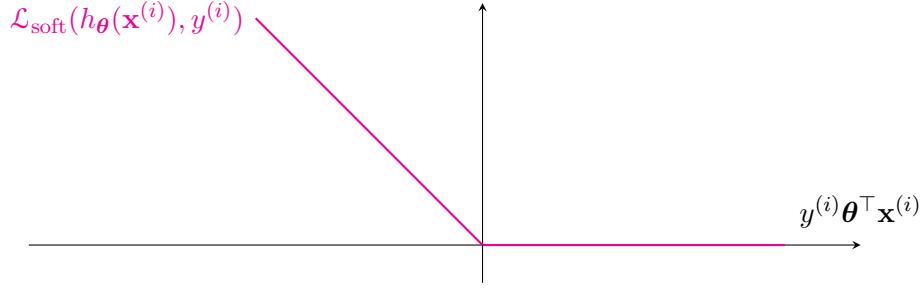


Figure 4.2: The decepton loss.

We get a bit more information from this than $\mathcal{L}_{\text{hard}}$ because a larger value of $-y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)}$ quantifies how off our prediction is (as opposed to the flat value of 1 for a misclassified example). Keep in mind that this loss function behaves the same as $\mathcal{L}_{\text{hard}}$ for correctly classified examples — they all get assigned a flat 0.

As before, the quantity we wish to minimise is the corresponding empirical risk function

$$\text{soft } R_{\text{emp.}}(h_{\boldsymbol{\theta}}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\text{soft}}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}).$$

Let $\mathcal{M} \subseteq \{1, \dots, n\}$ denote the set of indices corresponding to the examples misclassified by $h_{\boldsymbol{\theta}}$

$$\mathcal{M} := \{i: y^{(i)} h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) < 0\}.$$

It turns out that our soft empirical risk is equal to

$$\text{soft } R_{\text{emp.}}(h_{\boldsymbol{\theta}}) = \frac{1}{n} \sum_{i \in \mathcal{M}} -y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)}.$$

This sum is certainly a quantity we can differentiate and so the method of gradient descent is a viable approach for finding an optimal $\boldsymbol{\theta}$. Let's give that quantity a name

$$J(\boldsymbol{\theta}) := \sum_{i \in \mathcal{M}} -y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)}.$$

$J(\boldsymbol{\theta})$ is certainly positive. It's also convex:

Proof. We'll use two facts:

- $\max(0, -s)$ is convex.

Proof. $\max(x, y) = \frac{x+y}{2} + \frac{|x-y|}{2}$ and $|\cdot|$ is certainly convex (by the triangle inequality). ■

- If $f(\cdot)$ is convex, then $f(A(\cdot) + b)$ is also convex i.e. composition of a function with an affine transformation preserves convexity.

Now we simply note that $\max(0, -y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)})$ is the composition $(s \mapsto \max(0, -s)) \circ (\mathbf{x}^{(i)} \mapsto y^{(i)} \boldsymbol{\theta}^\top \mathbf{x}^{(i)})$. ■

Also note that $J(\boldsymbol{\theta})$ is convex in $\boldsymbol{\theta}$ with domain $\mathbb{R}^n$ and is bounded below by 0. The convexity implies that if there's a local minimiser, then it's ⁵ global minimiser at which J attains its global minimum.

The algorithm outline for the gradient descent goes as follows:

⁵Since J is not strictly convex (because $\max$ has flat parts), we can't conclude that such a global minimiser is unique.

Algorithm 4 Gradient descent on the soft empirical risk

```

1:  $\theta \leftarrow 0$  ▷ Initialise  $\theta$ 
2: while loop condition do
3:   update  $\theta \leftarrow \tilde{\theta}$  via the gradient descent update rule
4:   update  $\mathcal{M} \leftarrow \{i: -y^{(i)}(\tilde{\theta})^\top \mathbf{x}^{(i)} > 0\}$ 
5: end while
6: return  $\tilde{\theta}$ 

```

If the data are perfectly linearly separable into two classes, our loop condition can simply be (**while** $\mathcal{M} \neq \emptyset$) and the algorithm terminates when we find optimal parameters θ^* such that all examples have been correctly classified by h_{θ^*} i.e. the empirical risk is zero. Note that θ^* is not necessarily unique. There may be many values of the parameters for which the empirical risk is zero. This is illustrated in the following figure:

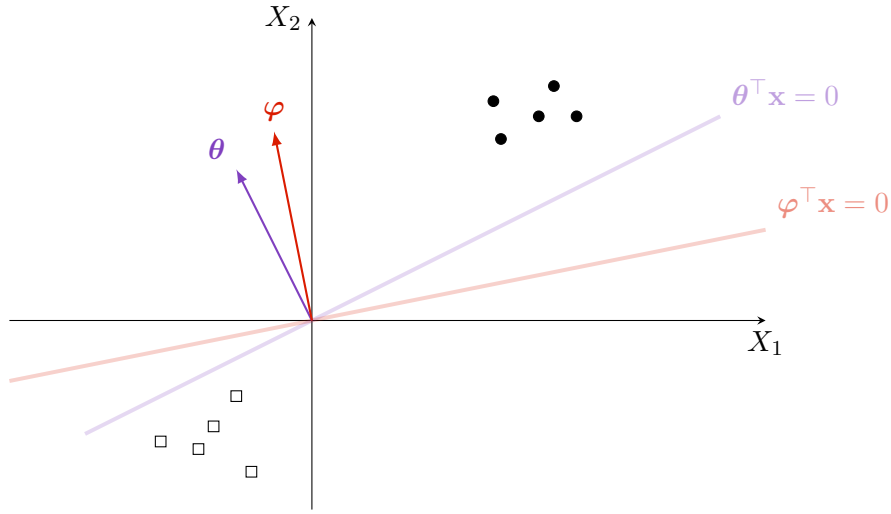


Figure 4.3: Even with perfectly linearly separable data, in $\mathbb{R}^2$ for illustrative convenience, there is no unique decision boundary for perfect classification. Note here that the bias/offset term $\theta_0 = 0$, so one should indeed note there's even more freedom than the diagram suggests for candidate decision boundaries.

If the data aren't perfectly linearly separable into two classes, a solution h_{θ} can be still be learned with $R_{\text{emp.}}(h_{\theta}) > 0$ but we'd have to implement a reasonable termination condition for our algorithm.

End of thought experiment. Let's return to the non-differentiable hypothesis with $\text{sgn}(\cdot)$ having a jump discontinuity:

4.2.3 CLASSICAL DECEPTRON

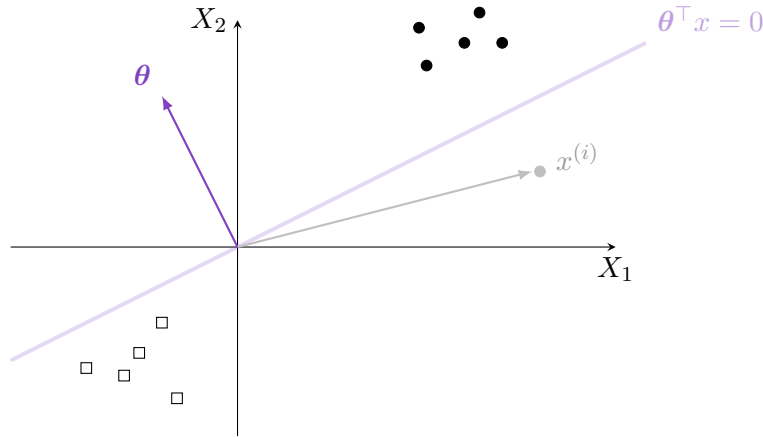
We shall make some geometric considerations to define an appropriate update rule for θ and *indirectly* minimise the misclassification rate $R_{\text{emp.}}(h_{\theta})$.

Suppose that we are working in $\mathbb{R}^2$ for visual convenience i.e. our feature vectors are of the form

$$\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \end{bmatrix} \in \mathcal{X} = \mathbb{R}^2.$$

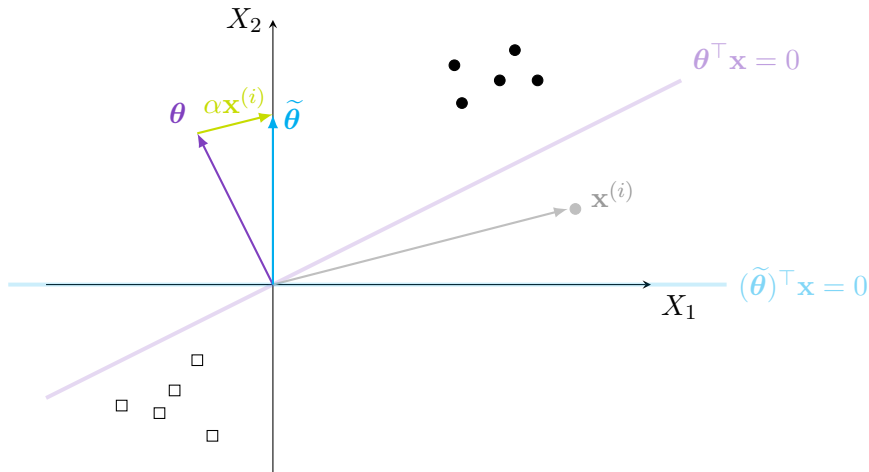
Suppose further that we have initialised θ and the corresponding hyperplane ~~miraculously~~ correctly classifies all but one training example.

- Suppose that the currently misclassified example $(\mathbf{x}^{(i)}, y^{(i)})$'s true label is positive. Highlight this by making it grey • i.e. $y^{(i)} = +1$ but $h_{\theta}(\mathbf{x}^{(i)}) = -1$.



To compensate for this example having been misclassified, a reasonable update of the parameters $\theta \leftarrow \tilde{\theta}$ would move (and in particular, for our hyperplane *passing through the origin*, rotate) the decision boundary in such a way that $\mathbf{x}^{(i)}$ would point more in the direction of $\tilde{\theta}$ i.e. to increase the value from $\langle \mathbf{x}^{(i)}, \theta \rangle < 0$ to $\langle \mathbf{x}^{(i)}, \tilde{\theta} \rangle \geq 0$. Since $\mathbf{x}^{(i)}$ is of course fixed (as part of the data $\mathcal{T}$ from which we're learning), we could add some non-negative scalar multiple α of $\mathbf{x}^{(i)}$ to θ i.e. $\theta \leftarrow \theta + \alpha \mathbf{x}^{(i)} =: \tilde{\theta}$ because

$$\langle \mathbf{x}^{(i)}, \tilde{\theta} \rangle := \langle \mathbf{x}^{(i)}, \theta + \alpha \mathbf{x}^{(i)} \rangle = \langle \mathbf{x}^{(i)}, \theta \rangle + \underbrace{\alpha \langle \mathbf{x}^{(i)}, \mathbf{x}^{(i)} \rangle}_{\geq 0} \geq \langle \mathbf{x}^{(i)}, \theta \rangle.$$

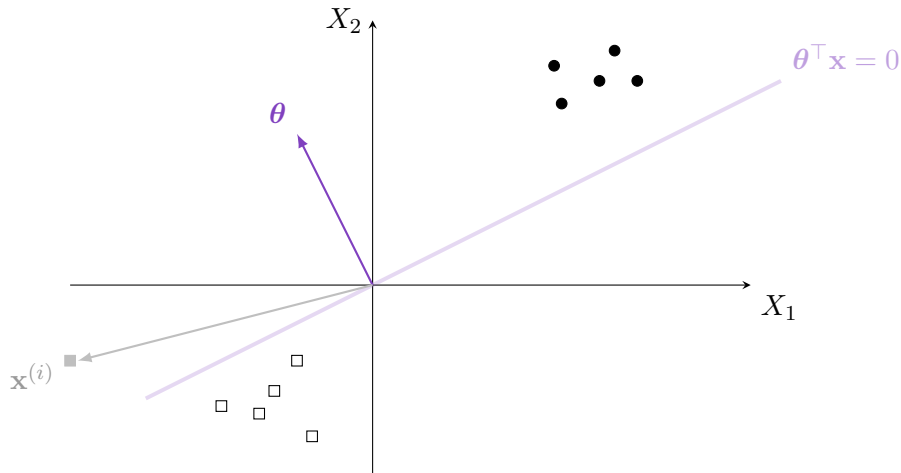


Therefore, our update of θ can be written as

$$\theta \leftarrow \theta + \alpha \mathbf{x}^{(i)} =: \theta + y^{(i)} \alpha \mathbf{x}^{(i)}.$$

That inclusion of $y^{(i)} = +1$ is non-obvious but the next example will highlight why we've done so.

- Now suppose instead that the misclassification is of a negative example ■ i.e. $y^{(i)} = -1$ but our hypothesis has assigned it the label $h_{\theta}(\mathbf{x}^{(i)}) = +1$.

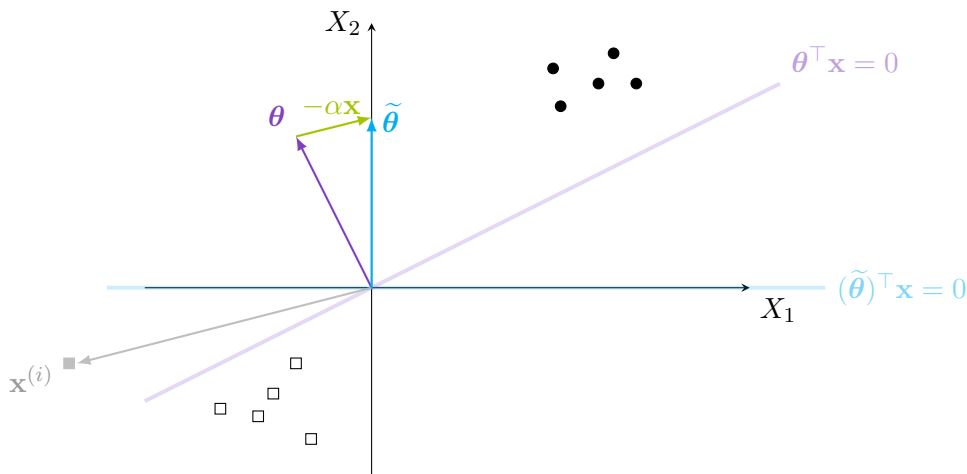


To compensate for this example having been misclassified, we want to achieve the opposite effect to before. Our decision boundary assigns our example a positive value $\langle \mathbf{x}^{(i)}, \boldsymbol{\theta} \rangle \geq 0$ but we wish to update $\boldsymbol{\theta} \leftarrow \tilde{\boldsymbol{\theta}}$ so that our example points in the opposite direction to $\tilde{\boldsymbol{\theta}}$ i.e. $\langle \mathbf{x}^{(i)}, \tilde{\boldsymbol{\theta}} \rangle < 0$. We can do this by subtracting some non-negative scalar multiple α of $\mathbf{x}^{(i)}$ to $\boldsymbol{\theta}$ i.e. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \mathbf{x}^{(i)} =: \tilde{\boldsymbol{\theta}}$ because

$$\langle \mathbf{x}^{(i)}, \tilde{\boldsymbol{\theta}} \rangle := \langle \mathbf{x}^{(i)}, \boldsymbol{\theta} - \alpha \mathbf{x}^{(i)} \rangle = \langle \mathbf{x}^{(i)}, \boldsymbol{\theta} \rangle - \underbrace{\alpha \langle \mathbf{x}^{(i)}, \mathbf{x}^{(i)} \rangle}_{\geq 0} \leq \langle \mathbf{x}^{(i)}, \boldsymbol{\theta} \rangle.$$

Note that

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \mathbf{x}^{(i)} =: \boldsymbol{\theta} + \underbrace{y^{(i)}}_{\text{yellow box}} \alpha \mathbf{x}^{(i)}.$$



To summarise, both misclassifications reduce to the update of parameters

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + y^{(i)} \alpha \mathbf{x}^{(i)}.$$

We didn't mention it before but if the example is correctly classified, we of course do not update $\boldsymbol{\theta}$ in the current iteration. Our total update rule is

$$\begin{cases} \text{Do nothing} & \text{if } (\mathbf{x}^{(i)}, y^{(i)}) \text{ is correctly classified,} \\ \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + y^{(i)} \alpha \mathbf{x}^{(i)} & \text{if } (\mathbf{x}^{(i)}, y^{(i)}) \text{ is misclassified.} \end{cases}$$

Equivalence to Rosenblatt's Perceptron (1962)

In the interest of historical preservation, the update rule derived above can be re-written in a way that serves as a special case of Rosenblatt's original 'error correction procedure' formulation introduced in [5, p. 110]. The actual update rule from [5, pp. 292–293] is written as the increment we update the weights by

$$\Delta \nu = a^* \cdot (\text{sgn}(R^* - r^*)) \cdot \epsilon.$$

In our notation:

- An asterisk superscript e.g. a^* denotes the current example being used, and any subsequent objects computed from it.
 - The current input feature vector is $a^* := \mathbf{x}^{(i)}$.
- The weights are $\nu := \boldsymbol{\theta}$.
- The *required response* is $R^* := y^{(i)}$.
- The *obtained response* is $r^* := h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$.
- The learning rate is $\epsilon := \alpha$.

Thus, the update rule is simply given by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \text{sgn}(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}.$$

How this is equivalent to our geometrically-derived update is simple. If our example has been correctly classified, Rosenblatt's rule also doesn't update the parameters for $\text{sgn}(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) = \text{sgn}(0) = 0$. For the misclassified examples:

- If $y^{(i)} = +1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -1$, then $\text{sgn}(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) = \text{sgn}(2) = 1 =: y^{(i)}$.
- Else, $y^{(i)} = -1$ and $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = +1$, and so $\text{sgn}(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) = \text{sgn}(-2) = -1 =: y^{(i)}$.

In both cases, $\text{sgn}(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) = y^{(i)}$ and so we recover our update rule.

The Algorithm and Comments

At last, we can state a naïve implementation of the **classical perceptron algorithm**.

Algorithm 5 A single epoch of the training step in the perceptron algorithm

```

1:  $\boldsymbol{\theta} \leftarrow \mathbf{0}$  ▷ Initialise  $\boldsymbol{\theta}$ 
2: while  $\mathcal{M} \neq \emptyset$  do
3:   for  $i = 1, \dots, n$  do
4:     if  $y^{(i)} \neq h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$  then
5:       pass
6:     else
7:       for  $i = 1, \dots, p$  do
8:          $\theta_j \leftarrow \theta_j + y^{(i)} \alpha x_j^{(i)}$ 
9:       end for
10:    end if
11:  end for
12: end while
13: return  $\boldsymbol{\theta}$ 

```

The above algorithm assumes that the data is perfectly linearly separable i.e. once all examples have been correctly classified, the `while` loop terminates. However, for training data that isn't perfectly linearly separable, we have a few options:

- We can fine-tune α as we go along, throttling the size of the updates until a threshold has been crossed (at which point we decide further updates aren't meaningful).
- We can impose a hard cap on the number of iterations.
- We can change our hypothesis from the non-differentiable $\text{sgn}(\cdot)$ function to $h_{\theta} = f \circ T$ for a differentiable f and follow the argument from earlier to minimise the corresponding “soft” loss function instead of the “hard” loss associated with $\text{sgn}(\cdot)$.

The classical perceptron learning algorithm isn't used much⁶ in practice; it doesn't have a probabilistic interpretation (as far as I can tell) but it's useful to have a geometric feel for what's happening with hyperplanes. It used to be pretty famous in the 50's and 60's when people thought it was a good model for the brain and how it works. However, a famous paper by Marvin Minsky demonstrated that the perceptron is limited because for some datasets, there does not exist a linear separating boundary.

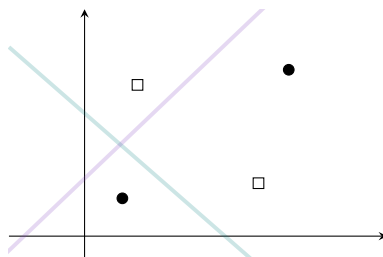


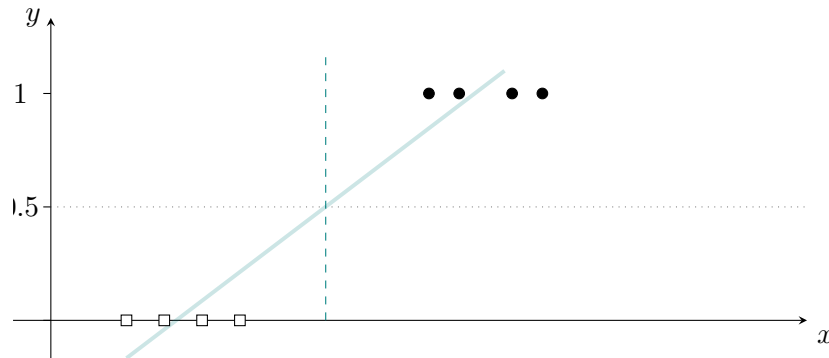
Figure 4.4: A dataset in $\mathbb{R}^2$ for which there does not exist a linear separating decision boundary. Two linear decision boundaries that fail to perfectly classify the dataset are depicted.

In the interest of inference, it would be great to have a model that quantifies how likely each example is to be a member of each class. It comes the modelling technique of logistic regression that uses a softer hypothesis to model the (conditional) probability of an unseen example's label (given its feature vector) — our hypothesis will be a so-called sigmoid function, and we shall see that the perceptron loss can be viewed as the limiting case of such a sigmoid.

⁶Hence the name ‘deceptron’ instead of ‘perceptron’!!

4.3 Logistic Regression

Consider the following dataset:



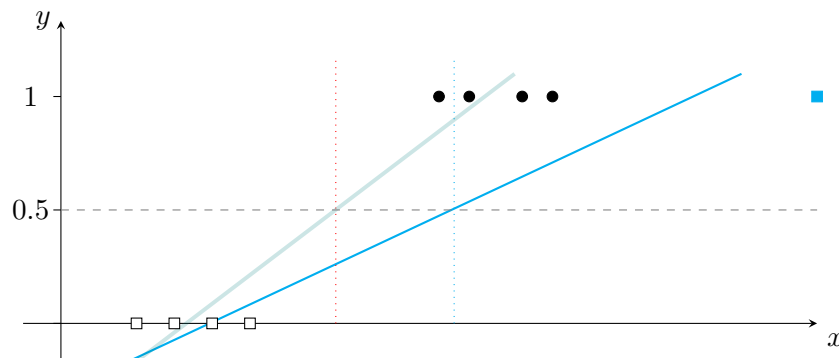
The data is clearly separated into two classes with labels $y^{(i)} \in \{0, 1\}$. Suppose that a linear regression hypothesis h_{θ} is fit to this data. Plotting the hypothesis gives a [line of best fit \$y = h_{\theta}\(x\)\$](#) . We can use this line to classify examples into two distinct classes in the following way:

- Set a value to threshold our prediction e.g. if $h_{\theta}(x) \geq 0.5$, we predict the value y to be 1, and 0 otherwise.
- On

The separating boundary can be set to $h_{\theta}(x) = 0.5$:

$$\text{label of } x = \begin{cases} 0, & h_{\theta}(x) \geq 0.5 \\ 1, & h_{\theta}(x) < 0.5 \end{cases}$$

Now say that we add a single example ■ to the dataset. With this extra example, which is clearly classified as positive and doesn't change the structure of the dataset very much



the [new straight line fit \$h_{\theta'}\$](#) is vastly different and thresholding at 0.5 gives a very different decision boundary. This suggests that the linear regression model doesn't generalise well to even similar training sets of this kind. **Model variance** refers to the amount by which the hypothesis h_{θ} would change if we estimated it using a different training data set. Thus, linear regression has high variance for this type of binary classification task and it's probably best to find another method that's not so sensitive to perturbing the input $\mathcal{T}$ in order to classify datasets of this nature.

4.3.1 THE MODEL

Without complete data on the population, there's often no perfect rule for classification. Instead, we resort to a stochastic approach that aims to model the conditional distribution of the response given the predictors. We do this, as described in **Section 1.2**:

- Assume that the data $\mathcal{T}$ has been generated by a true probability distribution $\mathbb{P}_{(\mathbf{X}, Y)}$ that lives in a family $\{\mathbb{P}_{\boldsymbol{\theta}}\}_{\boldsymbol{\theta} \in \Theta}$ of distributions parameterised by some $\boldsymbol{\theta} \in \Theta$.
- Search through this parameter space Θ by iterating over the training data in order to find an optimal parameter $\boldsymbol{\theta}^*$ that makes $\mathbb{P}_{\boldsymbol{\theta}^*}$ a good approximation for $\mathbb{P}_{(\mathbf{X}, Y)}$.

Probabilistic Assumptions

As always, we assume that the training examples are realisations of a collection of random vectors

$$(\mathbf{X}^{(i)}, Y^{(i)}) \stackrel{\text{i.i.d.}}{\sim} \mathbb{P}_{(\mathbf{X}, Y)}.$$

Since Y only assumes values in $\{0, 1\}$, it seems appropriate to model the response by a conditional Bernoulli distribution given $\mathbf{X}$. Since we always assume $\mathcal{X}$ to be a nice enough (Borel) space, there exists a system⁷ of regular conditional probabilities $\{\mathbb{P}^{\mathbf{x}}(\cdot) := \kappa_{\mathbf{X}}(\cdot, \mathbf{x})\}_{\mathbf{x} \in \mathcal{X}}$ on $\mathcal{F} = \sigma(\mathbf{X}, Y) = \sigma(\mathbf{X}) \vee \sigma(Y)$ given⁸ the sub- σ -algebra $\mathcal{G} = \sigma(\mathbf{X})$. The values of each conditional probability measure $\kappa_{\mathbf{X}}(\cdot, \mathbf{x})$ is completely determined⁹ by its values on $\{Y = 0\}$ and $\{Y = 1\}$, and these values must sum to unity

$$\kappa_{\mathbf{X}}(\{Y = 0\}, \mathbf{x}) + \kappa_{\mathbf{X}}(\{Y = 1\}, \mathbf{x}).$$

The goal is to model the conditional expectation of the response with a suitable hypothesis h so we can make predictions beyond the training data. In the case of our conditionally distributed Bernoulli random variable Y given $\mathbf{X}$, our conditional expectation is the function

$$\mathbf{x} \mapsto \int_{\Omega} Y(\omega') d\kappa_{\mathbf{X}}(\cdot, \mathbf{x})(\omega') =: \mathbb{E}[Y | \mathbf{X} = \mathbf{x}].$$

Since $Y^{-1}(\{0\}) \sqcup Y^{-1}(\{1\}) = \Omega$, our expression for $\mathbb{E}[Y | \mathbf{X} = \mathbf{x}]$ becomes

$$\begin{aligned} \mathbb{E}[Y | \mathbf{X} = \mathbf{x}] &= \int_{Y^{-1}(\{0\})} Y(\omega') d\kappa_{\mathbf{X}}(\cdot, \mathbf{x})(\omega') + \int_{Y^{-1}(\{1\})} Y(\omega') d\kappa_{\mathbf{X}}(\cdot, \mathbf{x})(\omega') \\ &= \int_{Y^{-1}(\{1\})} d\kappa_{\mathbf{X}}(\cdot, \mathbf{x})(\omega') \\ &= \kappa_{\mathbf{X}}(Y^{-1}(\{1\}), \mathbf{x}) \end{aligned}$$

which is the very formal way of writing the familiar expression

$$“\mathbb{E}[Y = 1 | \mathbf{X} = x] = \mathbb{P}_{Y|\mathbf{X}=x}(Y = 1 | \mathbf{X} = x).”$$

We first incorporate the predictors into the model as a linear combination $\boldsymbol{\theta}^{\top} \mathbf{X}$ like in linear regression. As before, $\boldsymbol{\theta}^{\top} \mathbf{X}$ can take any real value. Since Y assumes values in $\{0, 1\}$, and we're assuming that the conditional response is Bernoulli, we need to restrict the output of our hypothesis $[0, 1]$ in some meaningful way so as to interpret it as a probability. One such commonly used function is called the sigmoid/logistic function $g: \mathbb{R} \rightarrow [0, 1]$ defined by $z \mapsto (1 + e^{-z})^{-1}$. Thus, a candidate hypothesis to model the conditional expectation is $h_{\boldsymbol{\theta}}: \mathcal{X} \rightarrow [0, 1]$ defined by

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = g(\boldsymbol{\theta}^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^{\top} \mathbf{x}}}.$$

There are plenty of other functions g that could've been chosen to massage our prediction within $[0, 1]$. The logistic function was chosen in particular because it arises as a natural example of a broader class of algorithms derived from a broader set of probabilistic principles.¹⁰

⁷This is somewhat of an abuse of notation because we refer to a system of regular conditional probabilities on $\mathcal{F} = \sigma(\mathbf{X}, Y) = \sigma(\mathbf{X}) \vee \sigma(Y)$ generated by $\mathbf{X} \in \text{Meas}_{\mathcal{F}, \mathcal{E}_{\mathcal{X}}}(\Omega; \mathcal{X})$ both by the transition kernel $\kappa_{\mathbf{X}}^Y: \sigma(\mathbf{X}, Y) \times \mathcal{X} \rightarrow \mathbb{R}$ and the collection $\{\mathbb{P}^{\mathbf{x}}\}_{\mathbf{x} \in \mathcal{X}}$ where $\mathbb{P}^{\mathbf{x}}(\cdot)$ is the name we give every $\kappa_{\mathbf{X}}^Y(\cdot, \mathbf{x})$.

⁸Also said to be ‘generated by $\mathbf{X}$.’

⁹By a standard extension argument, first determining the values of every such measure on the generators of $\sigma(\mathbf{X}, Y)$.

¹⁰These are called generalised linear models.

All things above considered, the conditional distribution is completely specified by the condition

$$1 = \kappa_{\mathbf{X}}(\{Y = 0\}, \mathbf{x}) + \underbrace{\kappa_{\mathbf{X}}(\{Y = 1\}, \mathbf{x})}_{=: h_{\boldsymbol{\theta}}(\mathbf{x})}$$

which can be re-written for $y \in \{0, 1\}$ as

$$\kappa_{\mathbf{X}}(\{Y = y\}, \mathbf{x}) = (h_{\boldsymbol{\theta}}(\mathbf{x}))^y (1 - h_{\boldsymbol{\theta}}(\mathbf{x}))^{1-y}.$$

As usual, we don't have access to the actual conditional distribution of Y given $\mathbf{X}$ but we do have a training set $\mathcal{T}$ to analyse with the goal of learning "good" parameters $\boldsymbol{\theta}$ so that our model $h_{\boldsymbol{\theta}}$ best fits the data.

In exactly the same spirit as **Section 3.6.2**, we are interested in searching for parameters $\boldsymbol{\theta}$ that maximise the conditional likelihood given by

$$L_{\text{cond.}}(\boldsymbol{\theta}) = \prod_{i=1}^n (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^{y^{(i)}} (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^{1-y^{(i)}}$$

The conditional log-likelihood is simpler to maximise:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^n \left(y^{(i)} \log(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right).$$

An example of an algorithm to find an optimal value $\boldsymbol{\theta} = \boldsymbol{\theta}^*$ which maximises $\ell(\boldsymbol{\theta})$ is batch gradient ascent. The update rule for $j = 0, \dots, p$ simultaneously is:

$$\theta_j \leftarrow \theta_j + \alpha \frac{\partial}{\partial \theta_j} \ell(\boldsymbol{\theta}).$$

We use gradient ascent instead of descent because the log function is strictly concave. We could, equivalently, have used gradient descent on $-\ell(\boldsymbol{\theta})$.

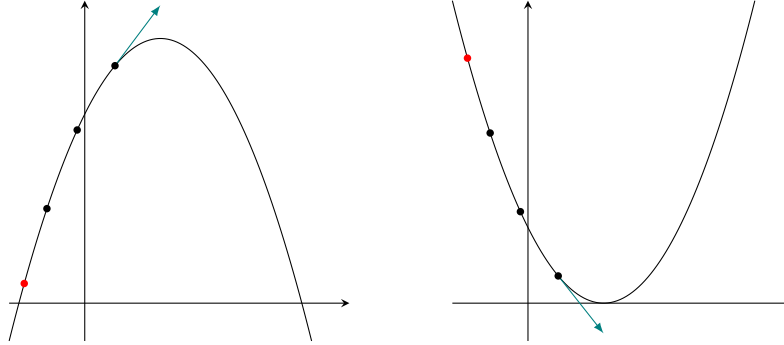


Figure 4.5: A visualisation of gradient ascent for $\ell(\theta_0, \theta_1)$ -concave (left) and gradient descent for ℓ -convex (right).

Plugging the definition of $\ell(\boldsymbol{\theta})$ into the update rule requires some algebra. The derivative of the logistic function is:

$$g'(z) = (g(z))(1 - g(z)) \quad (\text{d}\sigma)$$

For a single example $(\mathbf{x}^{(i)}, y^{(i)})$, the partial derivative of $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ w.r.t. θ_j is:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) &= \frac{\partial}{\partial \theta_j} g(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) \\ &= g'(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) \frac{\partial}{\partial \theta_j} (\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) \\ &= g'(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) x_j^{(i)} \\ &\stackrel{(\text{d}\sigma)}{=} g(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) (1 - g(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})) x_j^{(i)} \\ &= h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) x_j^{(i)} \end{aligned}$$

Thus, the expression for the partial derivative of the log-likelihood is:

$$\begin{aligned}\frac{\partial}{\partial \theta_j} \ell(\boldsymbol{\theta}) &= \frac{\partial}{\partial \theta_j} \sum_{i=1}^n \left(y^{(i)} \log(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right) \\ &= \sum_{i=1}^n \left(\frac{y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})}{h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))} \right) \frac{\partial}{\partial \theta_j} h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \\ &= \sum_{i=1}^n (y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) x_j^{(i)}\end{aligned}$$

and the update rule is given by the following formula

$$\theta_j \leftarrow \theta_j + \alpha \sum_{i=1}^n (y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) x_j^{(i)} \quad \text{for } j = 0, \dots, p \text{ simultaneously.}$$

This update rule looks identical (in form) to that of linear regression but the hypothesis function $h_{\boldsymbol{\theta}}$ is of course different ($g(\boldsymbol{\theta}^\top \mathbf{x})$ instead of $\Phi(\mathbf{x})\boldsymbol{\theta} = \boldsymbol{\theta}^\top \mathbf{x}$ for linear regression). This is no coincidence and will be re-visited later when GLMs are covered.

Logistic regression is really a method that, given an example x , outputs a probability $h_{\boldsymbol{\theta}}(x)$. It's up to the user to choose some kind of a threshold (e.g. 0.5) for these probabilities in order to define a classifier.



At the end of the perceptron algorithm, we can think of $\boldsymbol{\theta}$ as some linear combination of the misclassified examples during the training phase. With logistic regression, $\boldsymbol{\theta}$ is some linear combination of all examples with the weights being how correct or wrong our hypothesis was during training.

There are other methods we can use to search for a “good” $\boldsymbol{\theta}$:

4.4 Newton's Method

Gradient descent maximises a function using knowledge of its derivative.

An alternative method that can be used for maximising a function is known as Newton's method. Newton's method is a root-finding algorithm that constructs a sequence of real numbers that converge towards a root by leveraging second order Taylor approximations about the iterates. When applied to the derivative f' of a function f , Newton's method finds solutions to the equation $f' = 0$ i.e. the critical points (maxima, minima or saddle/inflexion). In this case, Newton's method on f' is a stronger version of gradient descent as it requires knowledge of f 's second derivative.

THE METHOD

Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and suppose that we wish to find a value θ^* of θ so that $f(\theta^*) = 0$. The goal is to pick an initial guess θ_0 and construct a sequence of iterates $\{\theta_k\}$ that converge towards a minimiser of f . The next iterate θ_{k+1} is found by minimising the second-order Taylor approximation about the prior iterate θ_k :

$$f(\theta_k + \varepsilon) \approx f(\theta_k) + f'(\theta_k)\varepsilon + \frac{f''(\theta_k)}{2}\varepsilon^2$$

by finding the critical point of the quadratic in ε

$$0 = \frac{\partial}{\partial \varepsilon} \left(f(\theta_k) + f'(\theta_k)\varepsilon + \frac{f''(\theta_k)}{2}\varepsilon^2 \right) \implies \varepsilon = \frac{-f'(\theta_k)}{f''(\theta_k)}$$

and setting $\theta_{k+1} = \theta_k + \varepsilon$. Thus, the update rule is

$$\theta_{k+1} \leftarrow \theta_k - \frac{f'(\theta_k)}{f''(\theta_k)}.$$

If θ is vector-valued, Newton's method can be generalised and the update rule becomes $\theta := \theta - H^{-1} \nabla_{\theta} f(\theta)$, where H is a $(p+1)$ by $(p+1)$ matrix of all mixed second-order partials called the Hessian and $\nabla_{\theta} f(\theta)$ is the usual vector of partial derivatives.

Newton's method enjoys a property called quadratic convergence and is typically faster than batch-gradient descent and requires fewer iterations to get a good approximation for a minimiser. However, each iteration is potentially more expensive because it requires finding and inverting a Hessian matrix (of mixed second order partials).

It definitely has its uses despite not being very popular for requiring more information (in the form of second order derivatives).

4.4.1 NEWTON-RAPHSON

The Newton-Raphson method is another method for root-finding and uses tangent lines to approximate a root. Initialise a guess $\theta^{(0)}$ and update the value of $\theta^{(i)}$ by choosing the point where the tangent line has a root

$$\theta^{(i+1)} \leftarrow \theta^{(i)} - \frac{f'(\theta^{(i)})}{f''(\theta^{(i)})}$$

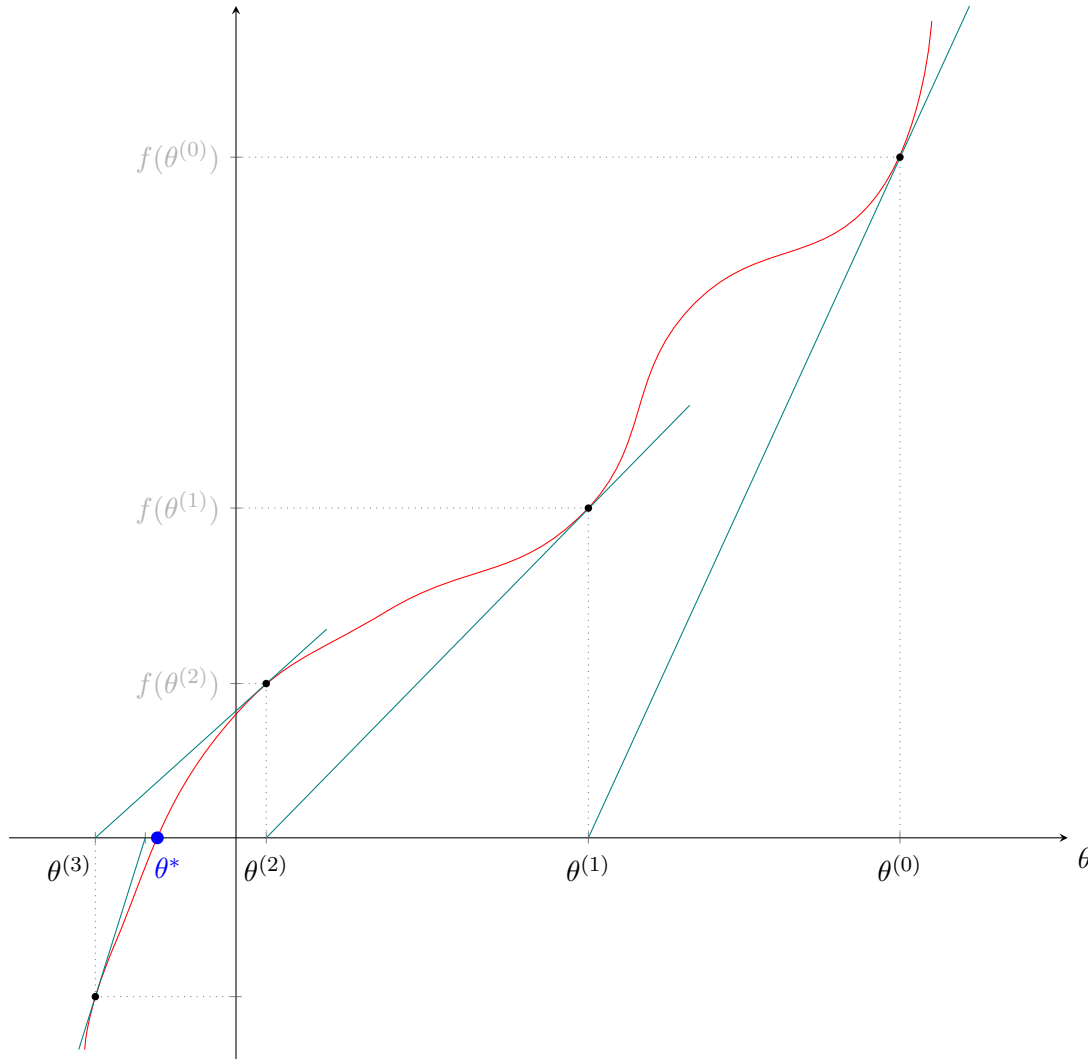
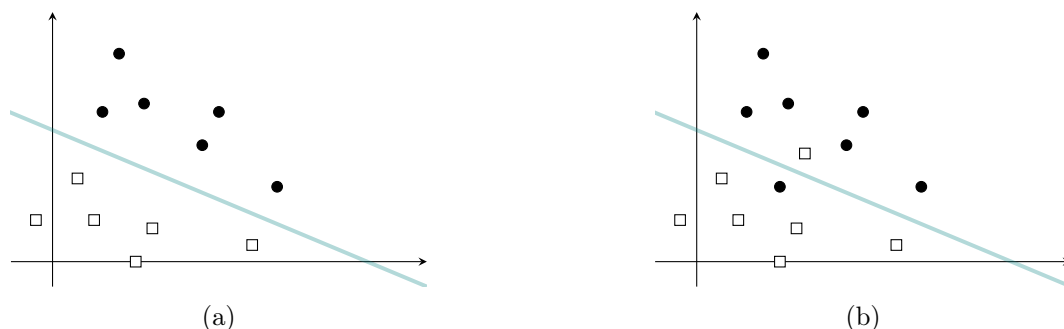


Figure 4.6: A visualisation of using the Newton-Raphson method to approximate the root θ^* .

Support Vector Machines

All the methods so far have generated linear classifiers. This is useful when the data is (i) linearly separable, and is still somewhat useful for data that is (ii) “almost” linearly separable (as illustrated below).



In **Figure 5.1b**, the best one can really do is fit a decision boundary that minimises the number of misclassified training examples. Such a model would have what is known as a **soft margin** which loosely means that we may separate most of the data (capturing the general label structure) while sacrificing a small (relative to the training set) number of examples to misclassification near the boundary.

5.1 The Plight of Linearly Inseparable Data

Example 5.1.1 The training data below could still be partially separated with a soft margin, but a decision boundary that curves looks more appropriate for the task of classification.

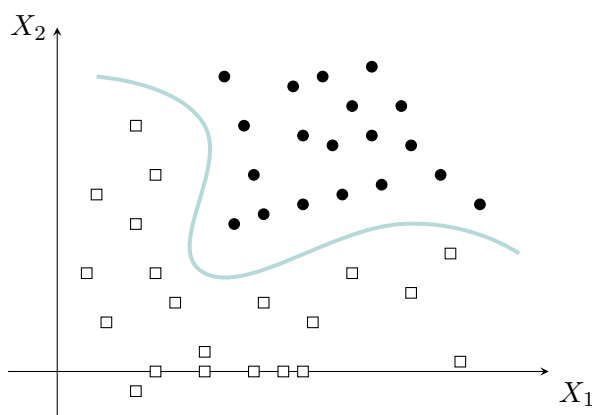


Figure 5.2: A visualisation of two-dimensional feature vectors (i.e. $\mathcal{X} = \mathbb{R}^2$) that fall under two class labels and are not perfectly separable by a hyperplane. ►

It turns out that we can obtain more complicated decision boundaries by first embedding (with some function ϕ) our input features into some sufficiently high-dimensional space $\phi(\mathcal{X})$ where the data *is* linearly separable, finding a decision hyperplane in that space, and then expanding the transformation to recover a non-linear decision boundary in our original space.

Some datasets will genuinely not allow for a separating hyperplane in any capacity. The following example is somewhat degenerate:

Example 5.1.2 The data below is genuinely linearly inseparable in $\mathbb{R}^2$. However, it becomes linearly separable when embedded into $\mathbb{R}^3$ via the **feature map** (or **representation**) $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ defined by

$$\phi: \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \mapsto \begin{bmatrix} x_1 \\ x_2 \\ x_1x_2 \end{bmatrix}.$$

The only point that is lifted from the $x_1 - x_2$ plane is $[1 \ 1]^\top$ and this is enough to separate the two classes:

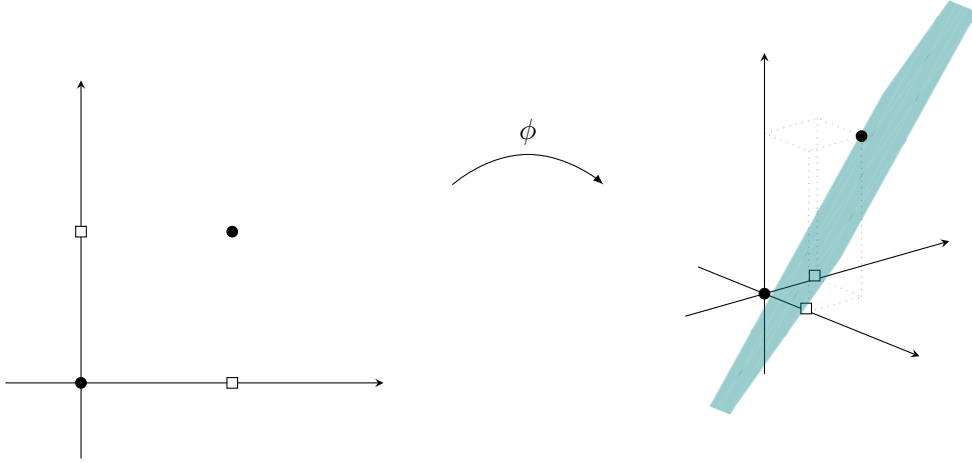


Figure 5.3: A linearly inseparable training set in $\mathbb{R}^2$ that becomes linearly separable once embedded into $\mathbb{R}^3$.

In the figure above, the decision hyperplane is given by $\mathbf{w}^\top \phi(\mathbf{x}) + b = 0$ where

$$\mathbf{w} = \begin{bmatrix} 1 \\ 1 \\ -2 \end{bmatrix} \quad \text{and} \quad b = -1/2.$$

We recover the non-linear decision boundary by substituting in our expression for $\phi(\mathbf{x})$ and obtain $x_1 + x_2 - 2x_1x_2 = 1/2$ which defines a degenerate hyperbola i.e. two intersecting lines $(x_1 - 1/2)(x_2 - 1/2) = 0$.

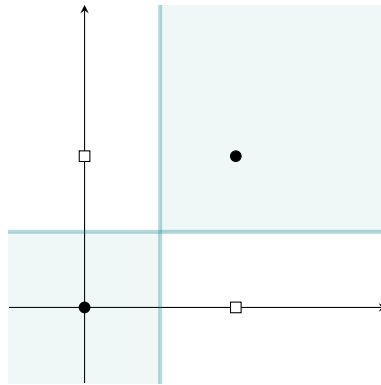


Figure 5.4: The non-linear decision boundary recovered from the embedding ϕ and decision hyperplane in $\phi(\mathcal{X}) = \mathbb{R}^3$. The shaded region is where $\mathbf{w}^\top \phi(\mathbf{x}) + b > 0$, and we can see that all positive examples (and consequently, negative examples) are correctly classified. ►

Example 5.1.3 Consider a dataset with two classes that find themselves arranged as follows:

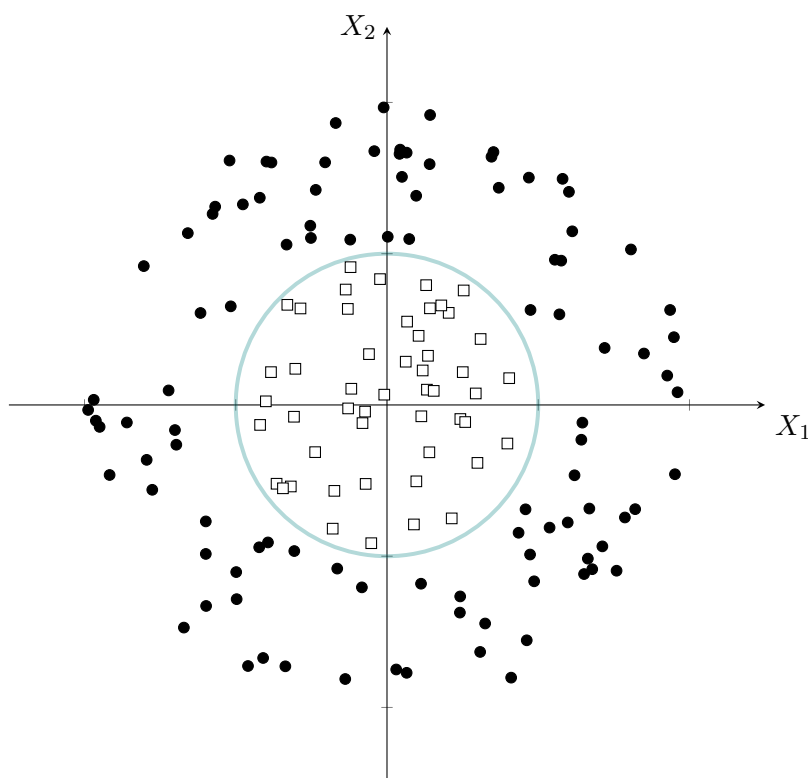


Figure 5.5: A dataset for which a radial decision boundary (a circle of radius 1) perfectly classifies all training examples.

Perhaps we could define a feature map that extracts the radial distance from the “centre” (in this case, the origin) of the data in $\mathcal{X}$ so that we may define a decision hyperplane in $\phi(\mathcal{X})$, and then revert back to $\mathcal{X}$ to obtain a “radial” classifier. ►

This method clearly requires careful consideration of the structure of the data being worked with, and what types of transformations ϕ would make the transformed data more amenable to linear classification downstream. The SVM model sidesteps this process (of embedding and then reversing the embedding) in a clever way by capitalising on the [kernel trick](#). Both of these will be defined in the upcoming sections. The material is, in my opinion, most cleanly presented in the following order:

1. Recall the perceptron learning algorithm for this will be the basis on which we build the rest of our theory.
2. Assume that our training data is linearly separable and define the SVM model.
3. Tackle the case where our training data is not linearly separable.

5.2 Separable Case (Optimal Margin Classifier)

The perceptron algorithm learns a set of “good” parameters $\theta_0, \dots, \theta_p$ that determine a hyperplane with which we classify (most of) the training examples correctly, and we hope for good performance on unseen data.¹

- The algorithm updates the parameters at each example so that the hyperplane makes a “more correct” prediction for the current example. This may make the predictions for other examples “more wrong”, so is it really a good algorithm?
- I’ve been told that it’s non-trivial to prove that if our data is linearly separable, then the perceptron algorithm terminates with a perfect classification of the training data.

I’ve been deliberately vague about “more wrong” and “more correct” predictions. We explain these by answering the following question:

ARE ALL SEPARATING HYPERPLANES EQUALLY “GOOD” FOR CLASSIFICATION?

Consider the following training data:

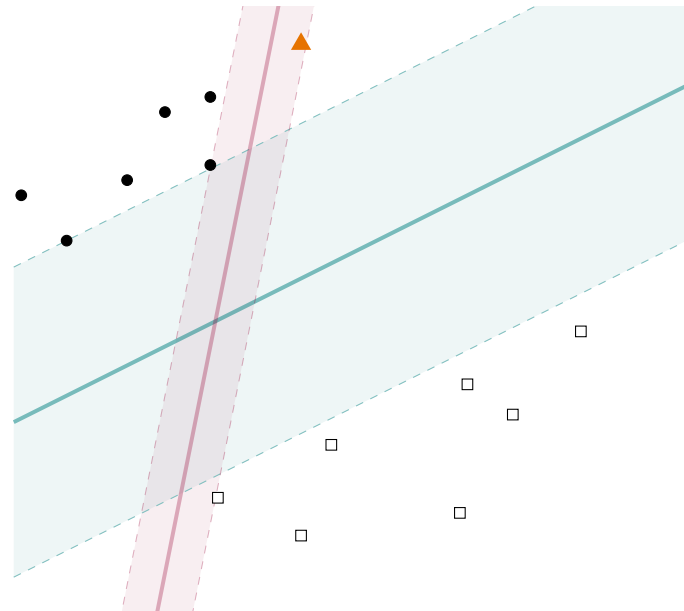


Figure 5.6: Two valid separating hyperplanes and their respective margins (shaded) that can be used to classify the training set (black circles and white squares). The **triangular point** represents an unseen example in the test set.

Simply by eyeing the data, we have a natural classification in mind — two clusters of input feature vectors. We visualise this by taking a separating hyperplane, finding its “margin” — the perpendicular distance from the hyperplane to its closest training example — and then looking for the hyperplane for which the margin is the greatest, using the shaded region defined by points within said margin from the hyperplane as a visual aid. Such a separating hyperplane will do the best job (geometrically in the margin sense) of separating our training examples into two classes.

Definition 5.2.1 A **support vector machine** (or **SVM**) is a supervised “maximum margin” learning model (used for classification) that has an associated learning algorithm.

¹I believe this has to do with the concept of the **VC dimension** of the data, the **complexity** of the hypothesis, and how it’s often the case that if we have enough training data, then good performance on the training data generalises to good performance on new data.

5.2.1 MARGINS

What makes one decision hyperplane better than another? Just by looking at the [unseen example](#)'s feature vector, we can see that it's most likely a circle. The blue decision boundary correctly classifies it as such, for it has quite a large perpendicular distance from the decision boundary. However, the purple decision boundary misclassifies it as a square. The small margin in this case means that slight perturbations of the input vector can cause a misclassification, even if the example is near the region in which like examples congregate. Bigger shaded region means less sensitive classification rule!

Also, from the perspective of logistic regression, we've already seen that points further away from the decision boundary are assigned a higher probability (by the sigmoid) of being in class. Thus, a large margin seems to be a good criterion.

Now suppose that the test example was part of the training set, and indeed is a negative example i.e. replace the [triangular point](#) in **Figure 5.6** with a white square. This example appears to be an outlier, and if we suppose that our data is linearly separable, and train a hyperplane to correctly classify all of our training examples, we can easily see that it's wildly different to the original hyperplane i.e. our model is very susceptible to outliers².

Ideally, we'd like to have a signed quantity that tells us both the perpendicular distance of an example from our decision boundary, and the side of the boundary on which it sits. One such candidate reared its head in the perceptron discussion. Multiply the label $y^{(i)}$ of an example by the signed quantity $\mathbf{w}^\top \mathbf{x}^{(i)} + b$ that is proportional to the signed distance of $\mathbf{x}^{(i)}$ from the hyperplane:

Definition 5.2.2 The **functional margin of an example** $(\mathbf{x}^{(i)}, y^{(i)})$ with respect to the decision hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is

$$\widehat{\gamma}^{(i)} := y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

If $\widehat{\gamma}^{(i)}$ is positive, our example is on the correct side of the decision hyperplane. Negative otherwise.

Definition 5.2.3 Then the **function margin of the training set** is the smallest among the functional margins of the training examples i.e.

$$\widehat{\gamma} := \min_i \widehat{\gamma}^{(i)}.$$

Then we can search for parameters which maximise $\widehat{\gamma}$. There is one glaring problem with this formulation. The equation defining a hyperplane is invariant under scaling all the parameters by the same real number. For if $\mathbf{w}^\top \mathbf{x} + b = 0$ defines a hyperplane parameterised by $(\mathbf{w}, b)$, then the scaled (with $c \neq 0$) parameters $(c\mathbf{w}, cb)$ define the same hyperplane:

$$\begin{aligned} (c\mathbf{w})^\top \mathbf{x} + (cb) &= c(\mathbf{w}^\top \mathbf{x} + b) \\ &= 0. \end{aligned}$$

Let's give the two hyperplanes different names:

$$\Pi: \mathbf{w}^\top \mathbf{x} + b = 0$$

$$\Pi_c: (c\mathbf{w})^\top \mathbf{x} + (cb) = 0$$

Why is this invariance under re-scaling a problem? The different parameterisations of the same hyperplane have different functional margins!! Take any example $(\mathbf{x}^{(i)}, y^{(i)})$. The functional margin with respect to Π_c is

$$\widehat{\gamma}_{\Pi_c}^{(i)} := y^{(i)}((c\mathbf{w})^\top \mathbf{x}^{(i)} + (cb)) = cy^{(i)}((\mathbf{w}^\top \mathbf{x}^{(i)} + b)) = c\widehat{\gamma}_{\Pi}^{(i)}.$$

²There is a method to relax the requirement of a hard margin between all examples, and we can fit a decision hyperplane with a soft margin like **Figure 5.1b**

If we're searching for optimal parameters to maximise the function margin, then we may simply rescale by larger and larger c . This creates a wild goose chase towards infinitely large parameters (weights and biases).

This surrogate quantity $\hat{\gamma}$ for the signed distance won't work in the sense of optimising our parameters but it's not of no use as we shall see shortly. Our problem was re-scaling so we instead use the signed distance:

The signed distance of a point $\mathbf{x}$ from a hyperplane Π can be calculated with the orthogonal projection of $\mathbf{x}$ onto Π . The signed distance from $\mathbf{x}$ to Π is the length of the scalar multiple $\delta\mathbf{w}$ of $\mathbf{w}$ s.t. $\mathbf{x} - \delta\mathbf{w}$ is equal to the orthogonal projection $\text{proj}_{\Pi}(\mathbf{x})$. Thus, $\mathbf{x} - \delta\mathbf{w}$ is on the hyperplane and satisfies

$$\begin{aligned} 0 &= \mathbf{w}^{\top}(\mathbf{x} - \delta\mathbf{w}) + b \\ &= \mathbf{w}^{\top}\mathbf{x} - \delta\|\mathbf{w}\|^2 + b \end{aligned}$$

and we may re-arrange to obtain

$$\begin{aligned} \delta\|\mathbf{w}\| &= \frac{\mathbf{w}^{\top}\mathbf{x} + b}{\|\mathbf{w}\|} \\ &= \left(\frac{\mathbf{w}}{\|\mathbf{w}\|} \right)^{\top} \mathbf{x} + \frac{b}{\|\mathbf{w}\|}. \end{aligned}$$

This quantity is $(\mathbf{w}, b) \mapsto (c\mathbf{w}, cb)$ scale-invariant.

Definition 5.2.4 The **geometric margin of an example** $(\mathbf{x}^{(i)}, y^{(i)})$ with respect to the decision hyperplane $\mathbf{w}^{\top}\mathbf{x} + b = 0$ is

$$\gamma^{(i)} := y^{(i)} \left(\left(\frac{\mathbf{w}}{\|\mathbf{w}\|} \right)^{\top} \mathbf{x} + \frac{b}{\|\mathbf{w}\|} \right).$$

The **geometric margin of the training set** is the smallest among the geometric margins of the training examples i.e.

$$\gamma := \min_i \gamma^{(i)}.$$

The relationship between the function and geometric margins is $\gamma = \frac{\hat{\gamma}}{\|\mathbf{w}\|}$.

In summary, we're interested in solving the following constrained optimisation problem:

$$\begin{aligned} &\text{maximise} && \gamma \\ &\text{among} && (\mathbf{w}, b, \gamma) \\ &\text{subject to} && \gamma^{(i)} \geq \gamma \quad \text{for } i = 1, \dots, n \end{aligned} \tag{5.1}$$

i.e. we'll consider all triples $(\mathbf{w}, b, \gamma)$ satisfying our constraints and choose the one whose third entry γ is the largest i.e. we're maximising the projection $(\mathbf{w}, b, \gamma) \mapsto \gamma$ onto the third entry.

Given the nature of the earlier discussion about scaling, we can attempt to remove any scaling ambiguity by including the constraint that $\|\mathbf{w}\| = 1$. This means that the function and geometric margins agree. Had we not included this constraint above, then if $(\mathbf{w}, b, \gamma)$ were feasible, so too would be $(c\mathbf{w}, cb, c\gamma)$ as discussed earlier. Our other constraint becomes $y^{(i)}(\mathbf{w}^{\top}\mathbf{x}^{(i)} + b) \geq \gamma = \hat{\gamma}$.

$$\begin{aligned} &\text{maximise} && \hat{\gamma} \\ &\text{among} && (\mathbf{w}, b, \hat{\gamma}) \\ &\text{subject to} && y^{(i)}(\mathbf{w}^{\top}\mathbf{x}^{(i)} + b) \geq \hat{\gamma} \quad \text{for } i = 1, \dots, n \\ &&& \|\mathbf{w}\| = 1 \end{aligned} \tag{5.2}$$

This is not a very easy optimisation problem. Non-convex problems are often difficult, and $\|\mathbf{w}\| = 1$ is not a convex constraint. We go back to our original problem and re-write the objective function and the constraint by using the relationship between the function and geometric margins:

$$\begin{aligned}
& \text{maximise} && \frac{\hat{\gamma}}{\|\mathbf{w}\|} \\
& \text{among} && (\mathbf{w}, b, \gamma) \\
& \text{subject to} && y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq \hat{\gamma} \quad \text{for } i = 1, \dots, n
\end{aligned} \tag{5.3}$$

This is an optimisation problem with linear constraints but our objective function is now a non-convex *fraction*. **However**, we may notice that by the scale-invariance of the geometric margin, we may re-scale any feasible $(\mathbf{w}, b, \hat{\gamma})$ to one with $\hat{\gamma} = 1$, leaving only $1/\|\mathbf{w}\|$ in the objective function.

Importantly, this tells us that we've now defined

$$\min_i |y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)| = 1.$$



Removing this degree of freedom with the function margin gives the optimisation problem

$$\begin{aligned}
& \text{maximise} && \frac{1}{\|\mathbf{w}\|} \\
& \text{among} && (\mathbf{w}, b) \\
& \text{subject to} && y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, \dots, n.
\end{aligned} \tag{5.4}$$

It's not immediately obvious that this problem has the same solutions to (5.3). If we re-scale the parameters $(\mathbf{w}, b) \mapsto (c\mathbf{w}, cb)$ in (5.4), then the objective function is

$$\frac{1}{\|c\mathbf{w}\|} = \frac{1}{|c|\|\mathbf{w}\|}.$$

We wish to choose c s.t. the objective is as large as possible i.e. we want c as small as possible under the given constraints:

$$y^{(i)} \left((c\mathbf{w})^\top \mathbf{x}^{(i)} + (cb) \right) \geq 1 \quad \text{for } i = 1, \dots, n$$

so we pick c s.t.

I believe this condition says that we push c as far down as possible until some training example satisfies $1 = cy^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$.

$$\min_i cy^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b \right) = 1$$

i.e.

$$\min_i y^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b \right) = \frac{1}{c}$$

the LHS of which is $\hat{\gamma}$ so we're picking c s.t. $c = 1/\hat{\gamma}$. Plugging this value of c back into the objective function recovers

$$\frac{1}{\|c\mathbf{w}\|} = \frac{1}{|c|\|\mathbf{w}\|} = \frac{|\hat{\gamma}|}{\|\mathbf{w}\|} = \frac{\hat{\gamma}}{\|\mathbf{w}\|}$$

the same objective function as (5.3).

Now it's plain sailing. Despite $1/\|\mathbf{w}\|$ being non-convex in (5.4), there is a standard equivalence of problems one sees very often in optimisation:

$$\begin{aligned}
\text{maximising } \frac{1}{\|\mathbf{w}\|} &\iff \text{minimising } \|\mathbf{w}\| \\
&\iff \text{minimising } \|\mathbf{w}\|^2
\end{aligned}$$

and it's easily verifiable that $\mathbf{w} \mapsto \|\mathbf{w}\|^2$ is a convex function e.g. by checking that its Hessian matrix is positive semi-definite.

According to **Definition B.3.6**, a general convex optimisation problem in function form (B.4) is written as follows:

$$\begin{aligned} & \text{minimise} && f(\mathbf{x}) \\ & \text{subject to} && g_i(\mathbf{x}) \leq 0 && \text{for } i = 1, \dots, m \\ & && h_j(\mathbf{x}) = 0 && \text{for } j = 1, \dots, k \end{aligned}$$

where $f, g_i: \mathbb{R}^n \rightarrow \mathbb{R}$ are convex functions, and $h_j: \mathbb{R}^n \rightarrow \mathbb{R}$ are affine functions.

Finally, we observe that our optimisation problem fits the general form of a (linearly constrained) convex optimisation problem with $f(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2$, and $g_i(\mathbf{w}, b) = 1 - y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$.

$$\begin{aligned} & \text{minimise} && \frac{1}{2}\|\mathbf{w}\|^2 \\ & \text{among} && (\mathbf{w}, b) \\ & \text{subject to} && y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 && \text{for } i = 1, \dots, n. \end{aligned} \tag{5.5}$$

A great benefit of converting optimisation problems into convex optimisation problems is that they're readily solvable with the plethora of algorithms out there.

However, we'll solve this one more directly by appealing to the theory of Lagrange multipliers and **dual problems**. See **Section B.6** for a quick recap of the method of Lagrange multipliers with *equality constraints*.

5.2.2 PRIMAL SETUP

Our optimisation problem (5.5) is convex but only has inequality constraints.

This section will side-bar (in gray) the general discussion for any (non-convex) optimisation problem, and we will apply this theory to our particular optimisation problem.

The most general non-convex optimisation problem is

$$\begin{aligned} & \text{minimise} && f(\mathbf{x}) \\ & \text{subject to} && g_i(\mathbf{x}) \leq 0 && \text{for } i = 1, \dots, m \\ & && h_j(\mathbf{x}) = 0 && \text{for } j = 1, \dots, k. \end{aligned}$$

We introduce the **generalised Lagrangian**

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) := f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{j=1}^k \nu_j h_j(\mathbf{x}).$$

This quantity is designed to penalise violations of constraints.

Since we have no equality constraints, the final summation in the general case doesn't appear in our Lagrangian. Thus, the generalised Lagrangian corresponding to (5.5) is

$$L(\mathbf{w}, b, \boldsymbol{\lambda}) = f(\mathbf{w}, b) + \sum_{i=1}^n \lambda_i g_i(\mathbf{w}, b).$$

One important distinction is that to violate an equality constraint (say $h_j(\mathbf{x}) = 0$), the violation may be positive or negative, and so there is no consistent sign with which one can prepend the associated multiplier in the Lagrangian that would penalise violations in both directions. However, an inequality constraint of the form $g_i(\mathbf{x}) \leq 0$ can only be violated when $g_i(\mathbf{x}) > 0$ is positive. The associated multiplier λ must then have non-negative entries $\lambda_i \geq 0$ to increase the Lagrangian (which we're trying to minimise) and penalise the violation.

We can re-write our problem in a different but equivalent form. I believe the following observation came after studying what is known as the dual problem to an optimisation problem, but I'm not too sure of the history. Consider the supremum of our Lagrangian over $\boldsymbol{\lambda}$ with non-negative entries:

$$\sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} L(\mathbf{w}, b, \boldsymbol{\lambda}) = \sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} \left(f(\mathbf{w}, b) + \sum_{i=1}^n \lambda_i g_i(\mathbf{w}, b) \right)$$

- If $\mathbf{w}, b$ are such that all constraints involving g_i are satisfied, then for every i , we have that $\lambda_i g_i(\mathbf{w}, b) \leq 0$, and so the supremum is attained at $\lambda_i = 0$ with value $f(\mathbf{w}, b)$.
- Otherwise, $\mathbf{w}, b$ violate at least one constraint i.e. $\exists i_0 \in \{1, \dots, n\}$ s.t. $g_{i_0}(\mathbf{w}, b) > 0$. In this case, $\lambda_{i_0} g_{i_0}(\mathbf{w}, b) > 0$, and the supremum of L over $\boldsymbol{\lambda}$ s.t. $\lambda_i \geq 0$ for all i is obtained by letting $\lambda_{i_0} \rightarrow +\infty$. Therefore, the supremum is $+\infty$.

In summary, we observe that the supremum and f agree when $\mathbf{w}, b$ are feasible, and the supremum is $+\infty$ otherwise i.e.

$$\sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} L(\mathbf{w}, b, \boldsymbol{\lambda}) = \begin{cases} f(\mathbf{w}, b) & \text{if } \mathbf{w}, b \text{ are feasible} \\ +\infty & \text{otherwise.} \end{cases}$$

The constraints are absorbed into the supremum because it encodes the feasibility constraints i.e.

$$\begin{aligned} \sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} L(\mathbf{w}, b, \boldsymbol{\lambda}) &= \begin{cases} f(\mathbf{w}, b) & \text{if } \mathbf{w}, b \text{ are feasible} \\ +\infty & \text{otherwise.} \end{cases} \\ &= f(\mathbf{w}, b) + \begin{cases} 0 & \text{if } \mathbf{w}, b \text{ are feasible} \\ +\infty & \text{otherwise.} \end{cases} \end{aligned}$$

and minimising this is the same as minimising $f(\mathbf{w}, b)$ over the feasible set.

Therefore, (5.5) is equivalent to the **primal Lagrange optimisation problem**:

$$\begin{aligned} &\text{minimise} \quad \sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} L(\mathbf{w}, b, \boldsymbol{\lambda}) \\ &\text{among} \quad (\mathbf{w}, b) \end{aligned} \tag{5.6}$$

Our problem (5.6) has the upside of an unconstrained optimisation problem, but the downside of a more complicated objective.

5.2.3 DUAL SETUP

Alternatively, one may approach the problem in the reverse order, looking first at a lower bound for the objective, and then varying the multipliers to tighten the bound.

Continuing the general case in this sidebar, note that for any $\tilde{\mathbf{x}}$ in the feasible set, we have $L(\tilde{\mathbf{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f(\tilde{\mathbf{x}})$ which requires that $\lambda_i \geq 0$ for every $i = 1, \dots, m$. Furthermore,

$$q(\boldsymbol{\lambda}, \boldsymbol{\nu}) := \inf_{\mathbf{x} \in \mathcal{D}} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq L(\tilde{\mathbf{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f(\tilde{\mathbf{x}}).$$

Since this holds for every feasible point $\tilde{\mathbf{x}}$, we conclude that $q(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f^*$. This inequality holds trivially when $q(\boldsymbol{\lambda}, \boldsymbol{\nu}) = -\infty$, and offers a non-trivial lower bound when $q(\boldsymbol{\lambda}, \boldsymbol{\nu}) > -\infty$ i.e. when the pair $\boldsymbol{\lambda}, \boldsymbol{\nu}$ with $\lambda_i \geq 0$ is in the domain of q . Such a pair is called **dual feasible**. The collection of all such pairs is called the **dual feasible set**.

Now that we've established that $q(\boldsymbol{\lambda}, \boldsymbol{\nu})$ is a lower bound, we take the supremum over $\boldsymbol{\lambda}$ with $\lambda_i \geq 0$ and $\boldsymbol{\nu} \in \mathbb{R}^k$ to obtain the tightest lower bound for the optimal value f^* i.e.

$$\begin{aligned} & \text{maximise} && \inf_{\mathbf{x} \in \mathcal{D}} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \\ & \text{among} && \boldsymbol{\lambda}, \boldsymbol{\nu} \\ & \text{subject to} && \lambda_i \geq 0 \text{ for } i = 1, \dots, m. \end{aligned} \tag{5.7}$$

The side-barred discussion motivates the formulation of the **dual Lagrange optimisation problem**:

$$\begin{aligned} & \text{maximise} && \inf_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\lambda}) \\ & \text{among} && \boldsymbol{\lambda} \\ & \text{subject to} && \lambda_i \geq 0 \text{ for } i = 1, \dots, n \end{aligned} \tag{5.8}$$

Remarks 5.2.5

- We call it the *dual* problem because we swap the order of the supremum and infimum.
- One calls

$$p^* := \inf_{\mathbf{w}, b} \sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} L(\mathbf{w}, b, \boldsymbol{\lambda}) \quad \text{and} \quad d^* := \sup_{\boldsymbol{\lambda}: \lambda_i \geq 0} \inf_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\lambda})$$

the **primal** and **dual value**, respectively.

This is an abridged version of the relevant theory:

- It is a known theorem that $d^* \leq p^*$ for every optimisation problem. This inequality is referred to as the **weak duality** of the primal and dual optimisation problems. The **duality gap** is the difference $p^* - d^*$.
- **Slater's condition** is the existence of a strictly feasible point for the primal problem i.e. a feasible point in the **relative interior** of $\mathcal{D}$ such that the inequality constraint is instead a strict inequality $g_i(\mathbf{x}) < 0$, and the equality constraint $h_j(\mathbf{x})$ holds.
- **Slater's Theorem** says that for a convex optimisation problem, if Slater's regularity condition holds, then the primal and dual problems are **strongly dual** i.e. $d^* = p^*$.
- If we assume strong duality, then a collection of conditions called the **KKT conditions** hold.
- For a convex problem, a feasible point $\tilde{\mathbf{x}}$ and its corresponding dual feasible pair $(\tilde{\boldsymbol{\lambda}}, \tilde{\boldsymbol{\nu}})$ satisfying the KKT conditions implies they are optimal (primal and dual values, respectively).
 - In summary, for a convex problem, a feasible point $\tilde{\mathbf{x}}$ and its corresponding dual feasible pair $(\tilde{\boldsymbol{\lambda}}, \tilde{\boldsymbol{\nu}})$ satisfying the KKT conditions is equivalently optimal.

5.2.4 DUALITY AND KKT CONDITIONS

Suppose that $\mathbf{x}^*$ and $(\boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ are optimisers of the primal and dual Lagrange optimisation problems, respectively. Suppose further that the duality gap is zero i.e. that the problems are strongly dual. Then

$$\begin{aligned} f(\mathbf{x}^*) &= \inf_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*) && \text{because zero duality gap} \\ &\leq L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*) && \text{since the infimum is less than or equal to the function at any point} \\ &= f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* g_i(\mathbf{x}^*) + \sum_{j=1}^k \nu_j^* h_j(\mathbf{x}^*) \\ &\leq f(\mathbf{x}^*) \end{aligned}$$

where the final inequality comes from the feasibility of $\mathbf{x}^*$ which tells us that:

- $g_i(\mathbf{x}^*) \leq 0$ for $i = 1, \dots, m$,
- $h_j(\mathbf{x}^*) = 0$ for $j = 1, \dots, k$,

and the dual feasibility of $\boldsymbol{\lambda}^*$ which tells us that $\lambda_i^* \geq 0$ for $i = 1, \dots, m$. This means that all the inequalities above are in fact equalities. From this, we conclude two things:

1. The first is that

$$\sum_{i=1}^m \lambda_i^* g_i(\mathbf{x}^*) = 0$$

and because $g_i(\mathbf{x}^*) \leq 0$ and $\lambda_i^* \geq 0$ for $i = 1, \dots, m$, we have that each

$$\lambda_i^* g_i(\mathbf{x}^*) = 0.$$

This condition^a is called the **complementary slackness condition**.

2. Note that $\mathbf{x}^*$ minimises $L(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ over $\mathbf{x}$.

If we assume further that f , g_i , and h_j are all differentiable functions (and so their domains are open sets), and suppose that $\mathbf{x}^*$ and $(\boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ are any primal and dual optimisers with zero duality gap, then $\mathbf{x}^*$ minimising $L(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ over $\mathbf{x}$ implies that $\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*) = \mathbf{0}$ i.e. that

$$\nabla_{\mathbf{x}} f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla_{\mathbf{x}} g_i(\mathbf{x}^*) + \sum_{j=1}^k \nu_j^* \nabla_{\mathbf{x}} h_j(\mathbf{x}^*) = \mathbf{0}.$$

We can nicely package all these conditions together

$$\begin{cases} g_i(\mathbf{x}^*) \leq 0 & i = 1, \dots, m \\ h_j(\mathbf{x}^*) = 0 & j = 1, \dots, k \\ \lambda_i^* \geq 0 & i = 1, \dots, m \\ \lambda_i^* g_i(\mathbf{x}^*) = 0 & i = 1, \dots, m \\ \nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*) = \mathbf{0} \end{cases}$$

called the **Karush-Kuhn-Tucker (KKT) conditions**.

^aWe could equivalently write it as $\lambda_i > 0 \implies g_i(\mathbf{x}^*) = 0$, or equivalently $g_i(\mathbf{x}^*) < 0 \implies \lambda_i^* = 0$.

There hasn't been any appeal to convexity in this section.



For a convex problem, with g_i convex and h_j affine, the KKT conditions are sufficient for points $\tilde{\mathbf{x}}$ and $(\tilde{\boldsymbol{\lambda}}, \tilde{\boldsymbol{\nu}})$ that satisfy them to be primal and dual optimisers, with zero duality gap.

5.2.5 SOLVING THE DUAL PROBLEM

Back to our dual optimisation problem:

$$\begin{aligned} & \text{maximise} && \inf_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\lambda}) \\ & \text{among} && \boldsymbol{\lambda} \\ & \text{subject to} && \lambda_i \geq 0 \text{ for } i = 1, \dots, n \end{aligned} \quad (5.9)$$

The Lagrangian is convex in $(\mathbf{w}, b)$. This is easiest to verify by showing that for fixed $\boldsymbol{\lambda}$, the Hessian of the map $L_{\boldsymbol{\lambda}}(\mathbf{w}, b) := L(\mathbf{w}, b, \boldsymbol{\lambda})$ is positive semi-definite. The Lagrangian is also differentiable, and the objective represents an unconstrained minimisation problem. This means that we can find the stationary point to find any global minimiser. Recall that

$$\begin{aligned} L(\mathbf{w}, b, \boldsymbol{\lambda}) &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \lambda_i \left(1 - y^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b \right) \right) \\ &= \frac{1}{2} \mathbf{w}^\top \mathbf{w} + \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \lambda_i y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)} - \sum_{i=1}^n \lambda_i y^{(i)} b \end{aligned}$$

∂_{w_k}

$$\begin{aligned} \frac{\partial L}{\partial w_k}(\mathbf{w}, b, \boldsymbol{\lambda}) &= \frac{\partial}{\partial w_k} \left(\frac{1}{2} \sum_{j=1}^p (w_j)^2 - \sum_{i=1}^n \lambda_i \left(1 - y^{(i)} \left(\sum_{j=1}^p w_j x_j^{(i)} + b \right) \right) \right) \\ &= w_k - \sum_{i=1}^n \lambda_i y^{(i)} x_k^{(i)} \\ \therefore 0 &= \frac{\partial L}{\partial w_k}(\mathbf{w}, b, \boldsymbol{\lambda}) \iff w_k = \sum_{i=1}^n \lambda_i y^{(i)} x_k^{(i)} \text{ for } k = 1, \dots, p. \end{aligned}$$

We gather these in vector-form for an optimal solution:

$$\mathbf{w}^* = \sum_{i=1}^n \lambda_i y^{(i)} \mathbf{x}^{(i)}.$$

This solution is actually unique because $L(\mathbf{w}, b, \boldsymbol{\lambda})$ is strictly convex in $\mathbf{w}$ if we hold b and $\boldsymbol{\lambda}$ fixed.

∂_b

$$0 = \frac{\partial L}{\partial b}(\mathbf{w}, b, \boldsymbol{\lambda}) = - \sum_{i=1}^n \lambda_i y^{(i)}$$

This doesn't solve for an optimal b^* but it does give us a constraint for the dual problem, for if the given sum were non-zero, then one could send b to the appropriately signed ∞ to drive the infimum to $-\infty$. Thus, we're forced to consider multipliers λ_i

Now we can substitute this value for $\mathbf{w}^*$ back into the objective and add the condition given by stationarity w.r.t b at b^* to the dual problem.

$$\begin{aligned} & \inf_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\lambda}) \\ &= L(\mathbf{w}^*, b, \boldsymbol{\lambda}) \\ &= \frac{1}{2} (\mathbf{w}^*)^\top \mathbf{w}^* + \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \lambda_i y^{(i)} (\mathbf{w}^*)^\top \mathbf{x}^{(i)} - \sum_{i=1}^n \lambda_i y^{(i)} b \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{i'=1}^n \lambda_i \lambda_{i'} y^{(i)} y^{(i')} (\mathbf{x}^{(i')})^\top \mathbf{x}^{(i)} + \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \lambda_i y^{(i)} \left(\sum_{i'=1}^n \lambda_{i'} y^{(i')} \mathbf{x}^{(i')} \right)^\top \mathbf{x}^{(i)} - \sum_{i=1}^n \lambda_i y^{(i)} b \\ &= -\frac{1}{2} \sum_{i=1}^n \sum_{i'=1}^n \lambda_i \lambda_{i'} y^{(i)} y^{(i')} (\mathbf{x}^{(i')})^\top \mathbf{x}^{(i)} + \sum_{i=1}^n \lambda_i - b \sum_{i=1}^n \lambda_i y^{(i)} \end{aligned}$$

Thus, our dual problem is

$$\begin{aligned}
& \text{maximise} && -\frac{1}{2} \sum_{i=1}^n \sum_{i'=1}^n \lambda_i \lambda_{i'} y^{(i)} y^{(i')} (\mathbf{x}^{(i')})^\top \mathbf{x}^{(i)} + \sum_{i=1}^n \lambda_i - b \left(\sum_{i=1}^n \lambda_i y^{(i)} \right) \\
& \text{among } \boldsymbol{\lambda} && \\
& \text{subject to} && \lambda_i \geq 0 \text{ for } i = 1, \dots, n \\
& && \sum_{i=1}^n \lambda_i y^{(i)} = 0
\end{aligned} \tag{5.10}$$

(The final term of the objective disappears because of the constraint.)

If we can solve this for some (not necessarily unique and) optimal $\boldsymbol{\lambda}^*$, then we can obtain the optimal

$$\mathbf{w}^* = \sum_{i=1}^n \lambda_i^* y^{(i)} \mathbf{x}^{(i)}$$

which gives the “slope” of our hyperplane. What remains is to find the optimal offset b^* . This is actually quite straight-forward. Recall that we removed the degree of freedom in our optimisation earlier by defining

$$\min_i |y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)| = 1.$$

The decision boundary should be equally far away from the closest point from each class i.e. the smallest functional margins for each label should be equal to 1. For the positive class,

$$1 = \min_{i: y^{(i)}=1} \hat{\gamma} = \min_{i: y^{(i)}=1} ((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b),$$

and for the negative class we have

$$\begin{aligned}
1 = \min_{i: y^{(i)}=-1} \hat{\gamma} &= - \min_{i: y^{(i)}=-1} ((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b) \iff -1 = - \min_{i: y^{(i)}=-1} -((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b) \\
&= \max_{i: y^{(i)}=-1} ((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b).
\end{aligned}$$

Thus,

$$\min_{i: y^{(i)}=1} ((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b) = - \max_{i: y^{(i)}=-1} ((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b)$$

and we conclude that

$$b^* = \frac{-1}{2} \left(\min_{i: y^{(i)}=1} (\mathbf{w}^*)^\top \mathbf{x}^{(i)} + \max_{i: y^{(i)}=-1} (\mathbf{w}^*)^\top \mathbf{x}^{(i)} \right)$$

is unique.

Thus, $(\mathbf{w}^*, b^*)$ is the unique solution of the primal problem. This does not imply uniqueness of the multipliers in the dual problem. The KKT conditions tell us a bit more about this solution. In particular, the primal feasibility, dual feasibility, and complementary slackness condition say

$$\begin{cases} (1 - y^{(i)}((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b)) & \text{for } i = 1, \dots, n \\ \lambda_i^* \geq 0 & \text{for } i = 1, \dots, n \\ \lambda_i^* (1 - y^{(i)}((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b)) = 0 & \text{for } i = 1, \dots, n \end{cases}$$

This tells us that $\lambda_i^* > 0 \iff y^{(i)}((\mathbf{w}^*)^\top \mathbf{x}^{(i)} + b^*) = 1$ which is precisely the condition that the training example $(\mathbf{x}^{(i)}, y^{(i)})$ is one of the points that defines the margin. Such an $\mathbf{x}^{(i)}$ is called a **support vector**. By definition of there always being at least one closest point, there always exists a support vector.

Consequently, $\mathbf{w}^*$ is a linear combination of **only** the support vectors. Moving any other non-support vectors as long as they don't cross into the margin will not affect the optimal decision hyperplane.

Therefore, the optimal weights and offset

$$\mathbf{w}^* = \sum_{i: \lambda_i^* > 0} \lambda_i^* y^{(i)} \mathbf{x}^{(i)} \quad \text{and} \quad b^* = \frac{-1}{2} \left(\min_{i: y^{(i)}=1} (\mathbf{w}^*)^\top \mathbf{x}^{(i)} + \max_{i: y^{(i)}=-1} (\mathbf{w}^*)^\top \mathbf{x}^{(i)} \right)$$

solve the primal problem and are the parameters of a maximum-margin separating hyperplane for our (linearly separable) training data.

Thus concludes the SVM model for linearly separable data. As outlined in a few examples at the beginning of this chapter, SVMs are also used to perform classification with non-linear decision boundaries.

5.3 Linearly Inseparable Case

If we look at the objective function in the final form of the dual problem (5.10)

$$\sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{i'=1}^n \lambda_i \lambda_{i'} y^{(i)} y^{(i')} (\mathbf{x}^{(i')})^\top \mathbf{x}^{(i)},$$

and the output label associated with our maximum-margin decision hyperplane

$$g(\mathbf{x}) = \begin{cases} +1 & \text{if } 0 \leq (\mathbf{w}^*)^\top \mathbf{x} + b^* = \sum_{i: \lambda_i^* > 0} \lambda_i^* y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{x} + b^* \\ -1 & \text{otherwise,} \end{cases}$$

the feature vectors present themselves in a very specific form³ — as an **inner product**.

Suppose that our training data is linearly inseparable in $\mathcal{X}$. Recall the discussion in **Section 5.1** that goes through how we may use some feature map ϕ to embed our input feature vectors (in $\mathcal{X}$) into a higher-dimensional space $\phi(\mathcal{X})$ where the transformed data is likely to be linearly separable. It stands to reason that we may use the SVM model with a hard margin to obtain a maximum margin decision hyperplane $(\mathbf{w}^*)^\top \phi(\mathbf{x}) + b^* = 0$ that perfectly classifies our transformed data in $\phi(\mathcal{X})$. One big problem: our transformed feature vectors $\phi(\mathbf{x}^{(i)})$ could potentially be very high-dimensional, and by our **key observation** they present themselves in inner products which would be quite costly⁴ to compute.

For the purposes of learning/optimisation, we're only interested in the number $\langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(i')}) \rangle$, and if we could find a less expensive way to compute it, that would be fantastic. We can! Compute the algebraic expression of the inner product

$$\langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(i')}) \rangle = K(\mathbf{x}^{(i)}, \mathbf{x}^{(i')})$$

and now we have a proxy quantity given by $K: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, some known, and less computationally expensive function to evaluate. In essence, we've side-stepped the computation of each $\phi(\mathbf{x}^{(i)})$ and then the inner product, and instead can compute the inner product using only K evaluated at the input features $\mathbf{x}^{(i)}$. We call such a function K a **kernel**, and this side-stepping is called the **kernel trick**.

³I've suppressed their appearance in b^* but they do appear there too once one expands the $\mathbf{w}^*$.

⁴A small saving grace is that our optimal $\mathbf{w}^*$ is *only* a linear combination of the support vectors in $\phi(\mathcal{X})$ but the situation is still not ideal.

Example 5.3.1 This is a poor example because the transformed space is very simple but we'll do it anyway. From **Example 5.1.2**, we have for any $\mathbf{x}, \mathbf{z} \in \mathcal{X} = \mathbb{R}^2$ that

$$\begin{aligned}\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle &= \left\langle \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{bmatrix}, \begin{bmatrix} z_1 \\ z_2 \\ z_1 z_2 \end{bmatrix} \right\rangle \\ &= x_1 z_1 + x_2 z_2 + x_1 x_2 z_1 z_2 \\ &=: K(\mathbf{x}, \mathbf{z}).\end{aligned}$$



The moral of the story is that for an incredibly high-dimensional codomain of ϕ , computing the kernel will be more efficient than computing each transformed vector and then finding the inner product. But how do we do this in a practical sense? I imagine over time people have discovered kernels that work very well for particular types of data, and some kernels tend to generally work quite well.

Example 5.3.2 Let $\mathbf{x}, \mathbf{z} \in \mathbb{R}^n$ and $r, \gamma, \sigma \in \mathbb{R}$. Some common kernels include:

1. A linear kernel $\langle \mathbf{x}, \mathbf{z} \rangle$
2. A polynomial kernel $(\gamma \langle \mathbf{x}, \mathbf{z} \rangle + r)^d$
3. Sigmoid: $\tanh(\gamma \langle \mathbf{x}, \mathbf{z} \rangle + r)$
4. Radial basis function (also called an rbf or Gaussian kernel):

$$e^{-\|\mathbf{x}-\mathbf{z}\|/\sigma} \quad \text{where} \quad \|\mathbf{x}-\mathbf{z}\| = \sqrt{\langle \mathbf{x}-\mathbf{z}, \mathbf{x}-\mathbf{z} \rangle}.$$



Some pros and cons of SVMs are:

- + They are useful for potentially highly non-linear decision boundaries.
- + The algorithmic implementation of the SVM is of **turn-key type** i.e. there aren't many parameters to fiddle with (like the learning rate α in gradient descent).
- They aren't as useful as neural networks nowadays.

PART II

Deep Learning

Neural Networks

6.1 The Limitations of Linear Classifiers

Geometrically, linear classifiers draw hyperplanes to partition Euclidean space into two complementary subsets.

Example 6.1.1 Consider the problem of classifying whether or not a .jpeg image of size say 64x64 pixels contains a horse. The label 1 corresponds to the presence of a horse, and 0 for the absence thereof. Our training set $\mathcal{T}$ is therefore a collection of images of such dimension paired with a label in $\{0, 1\}$. We need $64 \cdot 64 \cdot 3$ numbers to represent¹ the image because there are 3 colour channels (R, G, and B) associated with every pixel. We flatten the image into a column vector of length 12,288.

Linear classifiers learn one template per class, and so a half-space would represent the presence of a horse. This has the glaring shortcoming of the decision hyperplane being unable to represent multiple modes of the same object category i.e. a linear classifier would learn parameters that define a template for the presence of a horse which blends images of, for example, horses facing to the right border of the image with those that face the left border with those that are dark with those that are black etc. ►

This representational shortcoming can be addressed by constructing maps $f_i: \mathcal{X} \rightarrow \mathbb{R}^{d_i}$ that extract particular features of the image. Then we may concatenate these different feature representations to obtain a single, long, high-dimensional feature vector

$$\phi(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{d_1 + \dots + d_m}$$

that describes an image where different parts of the vector now capture different types of information from the image e.g. $f_1(\mathbf{x})$ could represent how much red is in the image, $f_2(\mathbf{x})$ how many vertical edges there are etc.

There's an analogous kernel trick for to bypass the explicit construction and use of such a feature map i.e. there exists some kernel K s.t. for any $\mathbf{x}, \mathbf{z} \in \mathbb{R}^{d_1 + \dots + d_m}$ we have

$$K(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^m f_i(\mathbf{x}) f_i(\mathbf{z}).$$

I believe that some feature maps have infinite dimensional codomain, say F , and so the corresponding kernel would be a closed-form expression that coincides with an infinite sum.

Instead of manually constructing the maps that extract features and then learning the parameters of a linear classifier, perhaps we could also learn the feature maps too. Such a model would be a single end-to-end pipeline that takes in raw input of pixels of an image, and outputs a prediction. All parts of the model would be simultaneously optimisable with a view to maximise its performance on the image classification task. We've loosely described a **neural network**.

¹Had the image been a .png, there would be a 4th value for the opacity (alpha) of the image.

For a linear classifier, each collection of parameters is interpreted as a single template per class, and the output scores are the similarity measures (represented by an inner product) between the input image vector and each of the learned templates. A “neural network” generalises this by having different “layers” to represent varying levels of abstraction of our input data, with:

- early layers comprising of templates that are typically not very interpretable — known as a **distributed representation**,
- and later layers recombining (in a weighted fashion) earlier layers to represent multiple modes of an object category.

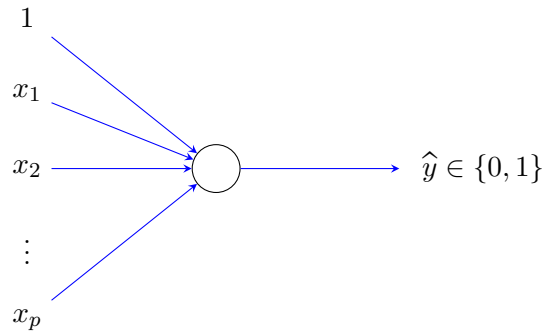
6.2 What is a Neural Network, Roughly?

Definition 6.2.1 **Neural networks** are a class of functions (usually obtained via composition) that are used to approximate other functions. They are often visualised as a collection of nodes and directed edges, with each edge representing the output of a prior node being fed as input into the current node, and each node representing the application of an affine transformation of its inputs followed by a non-linear “**activation**” function.

Importantly, neural networks are simple enough to be efficiently trained by gradient-based methods.

6.2.1 THE SINGLE-LAYER PERCEPTRON

Example 6.2.2 Recall the perceptron discussed in **Section 4.2**. We can visually represent the classifier $h_{\theta}(\mathbf{x}) := \text{sgn}(\theta^{\top} \mathbf{x})$ by a node that receives several inputs (the entries of $\mathbf{x}$), and outputs a corresponding label $h_{\theta}(\mathbf{x}) =: \hat{y} \in \{0, 1\}$:



The node is called a **neuron** because of the biological analogue that “fires” once a certain stimulus is received — the stimulus in our case being a linear combination of inputs that exceeds the threshold 0. Equivalently,

$$\theta^{\top} \mathbf{x} \geq 0 \iff \theta_0 \cdot 1 + \sum_{j=1}^p \theta_j x_j \geq 0 \iff \sum_{j=1}^p \theta_j x_j \geq -\theta_0$$

and so the bias term is the negative threshold. This is the simplest possible example of a neural network — one with a single node. We need some terminology to describe the “linear part” of this neuron:

Definition 6.2.3 An **affine linear functional** is an affine map from a vector space to the field over which it’s defined.

Such a functional differs from the usual linear functional by an offset. In our case, T is the

affine linear functional defined by

$$T: \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} \mapsto \langle \mathbf{x}, \boldsymbol{\theta} \rangle + \theta_0 = \sum_{j=1}^p \theta_j x_j + \theta_0$$

and so this offset is the bias term. The non-linear activation function is simply the sign function $\text{sgn}(\cdot)$. Thus, our neural network (comprising of a single neuron) is the composition

$$(\text{sgn} \circ T)(\mathbf{x}),$$

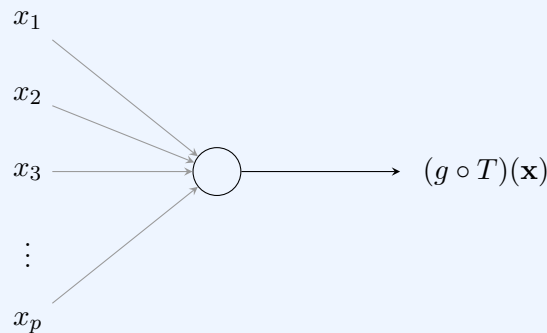
and we're approximating the hypothetical true binary function that correctly classifies all training and unseen examples.

Other books use a different labelling convention $\{0, 1\}$ instead of our $\{-1, +1\}$ for examples and so their activation will typically be something of the form $\mathbb{1}_{[0, +\infty)}(z)$ so their single-layer neural network will be the composition $(\mathbb{1}_{[0, +\infty)} \circ T)(\mathbf{x})$.



We're free to change the activation function $\text{sgn}(\cdot)$ (or $\mathbb{1}_{[0, +\infty)}$) with something else for the purpose of approximating other functions. This gives rise to the true definition of a (single-layer) perceptron:

Definition 6.2.4 A **perceptron** is a single-layer neural network with one neuron that first applies an affine linear functional T to the input feature vector x , followed by an *arbitrary* activation function g i.e. $\mathbf{x} \mapsto (g \circ T)(\mathbf{x})$. Visually, we represent it as follows:



Definition 6.2.5 The (visual) configuration of a neural network is informally called its **architecture**.

The next example replaces the activation of the previous example's neural network with the smooth sigmoid function, and the overall goal of the neural net is to approximate the conditional expectation, and as a consequence the resulting linear decision boundary approximates the true binary function that classifies the appropriate object (in this case, images):

6.2.2 LOGISTIC REGRESSION AS A NEURAL NETWORK

Example 6.2.6 Say we wish to classify a .jpeg image of size 64x64 pixels depending on whether it contains a cat. Label the image with the number 1 if it does, or 0 if it doesn't. Our training set $\mathcal{T}$ is therefore a collection of images flattened into column vectors in $\mathbb{R}^{12,288}$, each paired with a label in $\{0, 1\}$.

The method of logistic regression aims to compute the probability that an unseen image (represented by a feature vector $\mathbf{x}$) has the label 1. We denote by $\boldsymbol{\theta}$ the vector of parameters we wish

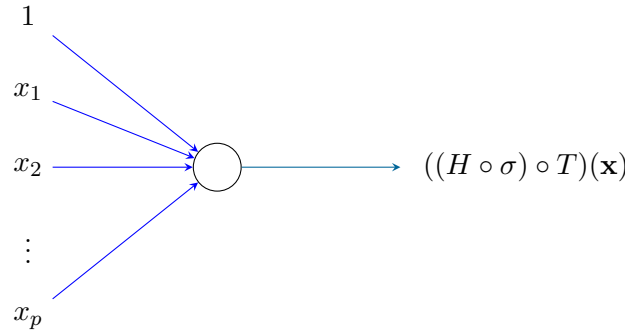
to learn that will (hopefully, by the end of the training step) represent the prototypical image that contains a cat (and therefore has the label 1).

In this case, θ will have the same number of parameters as each input example's feature vector $\mathbf{x}^{(i)}$, plus a bias/intercept term i.e. the dimension of θ scales with $\mathbf{x}^{(i)}$. Ideally, we'd like to avoid the scenario where $\mathbf{x}^{(i)}$ is a prohibitively high-dimensional vector, for that would mean we have a very large number of parameters to learn.

First, we compute the unbounded linear unit $\langle \mathbf{x}^{(i)}, \theta \rangle = \theta^\top \mathbf{x}^{(i)}$, which serves as a measure of similarity between the input and θ . Then we compose this with the sigmoid function to output the probability $\sigma(\langle \mathbf{x}^{(i)}, \theta \rangle)$ that the image contains a cat. Then we employ an algorithm that iterates over the training data $\mathcal{T} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_i$, minimising an associated loss function that penalises misclassification, finally outputting a vector $\hat{\theta}$ of learned parameters. Using these parameters, we threshold the output probability $\sigma(\langle \mathbf{x}, \hat{\theta} \rangle)$ at a certain value (say 0.5) to obtain the predicted label $\hat{y}$ for $\mathbf{x}$ e.g.

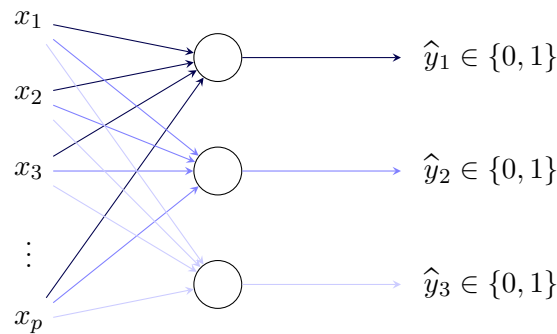
$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(\langle \mathbf{x}, \hat{\theta} \rangle) \geq 0.5, \\ 0 & \text{otherwise.} \end{cases}$$

We can visualise the logistic regression model as a neural network with a single layer, and whose non-linear activation is the composition $g = H \circ \sigma$ of a thresholding function H with the sigmoid:



6.2.3 MULTI-CLASS CLASSIFICATION

This example will offer good insight into the notation used when we begin to add more nodes to a neural network. Suppose that we wish to detect a cat, lion, or iguana in an image. This is a multi-class classification problem. We can approach this by modifying the prior example to include two more “logistic regressions” as nodes, each with parameters corresponding to the other two classes (lion and iguana).



For this example, the neural network just above has only a single layer:

Definition 6.2.7 A **layer** denotes a collection of neurons that are not connected to each other.

- The **input layer** takes as input the feature vector of data.

- The **output layer** outputs our prediction(s).
- Any layers between the two are called **hidden layers** because they only see as input what the prior layer (an abstraction of the input data) gave it, and the output $\hat{y}$ is hidden from it too.

In general, a neural network with a more complicated architecture may have multiple layers.

Square parentheses in a superscript $\dots^{[\ell]}$ denote the object belongs to or describes the ℓ^{th} layer of the architecture.

Definition 6.2.8

- The **depth** of a neural network is the number of layers in its architecture.
- Let $d^{[\ell]}$ denote the **width of the ℓ^{th} layer** — the number of neurons in said layer.

Example 6.2.9 In the above example, the neural network has a single layer, so depth 1, and the width of this layer is $d^{[1]} = 3$. ►

The top neuron is responsible for detecting the presence of a cat, the middle a lion, and the bottom an iguana. The training data for this modified network should have as its labels $y^{(i)}$ column vectors of length 3 whose entries are in $\{0, 1\}$. This network has 3 times the parameters of the prior linear regression example. Indeed, these nodes will train independently since there is no interdependence of the sigmoids. An appropriate loss function would simply be the sum of each logistic regression's loss. This is technically a neural network but a trivial one. Note that $\hat{y}$ is a column vector of length 3. The network still fits the purpose for labelling different animals in the same image. We can modify this by adding the constraint that there is a unique animal in each image. How do we appropriately modify the architecture?

- Assume that every image must contain at least one of the animals so the corresponding output labels must be one of $[1 \ 0 \ 0]^\top$, $[0 \ 1 \ 0]^\top$, or $[0 \ 0 \ 1]^\top$.
- Since the outputs of the respective sigmoids represent *independent* probabilities, we can replace them with the appropriate softmax functions e.g.

$$\hat{y}_2 = \frac{(\theta^{[2]})^\top \mathbf{x}}{\sum_{k=1}^3 \exp((\theta^{[k]})^\top \mathbf{x})}$$

so that the outputs of the neurons (before thresholding) sum to 1 i.e. they *interact* with each other. For example, if two of the neurons output very low probabilities, then this forces the third to fire i.e. the image has that animal.

- I believe something called the **cross-entropy** loss is used here: $-\sum_{k=1}^3 y_k \log(\hat{y}_k)$.

Remarks 6.2.10 When considering a neural network, keep in mind that one should typically have as many neurons in the final layer as one wishes to predict classes (or just 1 in the case of a regression).

6.3 What is a Neural Network, Precisely?

We restrict our attention to a particular subclass of neural networks:

Definition 6.3.1 If a neural network is arranged by layers, every edge is directed from layer ℓ to layer $\ell + 1$, and every node in layer ℓ connects to every node in layer $\ell + 1$, then the neural network is called a **multi-layer perceptron** or **feedforward^a neural network**.

^aFeedforward for the obvious reason — the input is fed only forward through the network up until prediction. “There are no **feedback** connections in which outputs of the model are fed back into itself.” — [6, p. 168].

This will simplify notation greatly. The architecture of such a feedforward neural network will have L layers in total and $d^{[\ell]}$ neurons in each layer $\ell = 1, \dots, L$.

6.3.1 A SINGLE NEURON

- Denote by $\mathbf{a}_k^{[\ell-1]}$ the output of the k^{th} neuron in the $(\ell - 1)^{\text{th}}$ layer.
 - Though it may seem trivial, define the feature vector input by $\mathbf{x} =: \mathbf{a}^{[0]} \in \mathbb{R}^{d^{[0]}}$.
- Denote by $\mathbf{a}^{[\ell-1]}$ the vector whose components are the outputs all the neurons in $(\ell - 1)^{\text{th}}$ layer:

$$\mathbf{a}^{[\ell-1]} = \begin{bmatrix} a_1^{[\ell-1]} \\ \vdots \\ a_{d^{[\ell-1]}}^{[\ell-1]} \end{bmatrix}.$$

- Denote by $z_k^{[\ell]}$ the number we obtain by applying an affine linear functional $T_k^{[\ell]}$ on $\mathbb{R}^{d^{[\ell-1]}+1}$ to $\mathbf{a}^{[\ell-1]}$. Namely,

$$T_k^{[\ell]} : \mathbf{a}^{[\ell-1]} \mapsto \left\langle \mathbf{a}^{[\ell-1]}, \left(\mathbf{w}_k^{[\ell]} \right)^\top \right\rangle + \mathbf{b}_k^{[\ell]}$$

where $\mathbf{b}_k^{[\ell]}$ is the bias/intercept term of this neuron’s “linear part”.

- Finally, the output of the k^{th} neuron in the ℓ^{th} layer is

$$a_k^{[\ell]} := g_k^{[\ell]} \left(\mathbf{z}_k^{[\ell]} \right)$$

where $g_k^{[\ell]}$ is the non-linear activation function of said neuron.

6.3.2 A SINGLE LAYER

For the ℓ^{th} layer:

- We can collect the maps $T_1^{[\ell]}, \dots, T_{d^{[\ell]}}^{[\ell]}$ into the following vector-to-vector map:

$$T^{[\ell]} : \mathbf{a}^{[\ell-1]} \mapsto \begin{bmatrix} T_1^{[\ell]}(\mathbf{a}^{[\ell-1]}) \\ \vdots \\ T_{d^{[\ell]}}^{[\ell]}(\mathbf{a}^{[\ell-1]}) \end{bmatrix} = \begin{bmatrix} z_1^{[\ell]} \\ \vdots \\ z_{d^{[\ell]}}^{[\ell]} \end{bmatrix} =: \mathbf{z}^{[\ell]}$$

- Equivalently, collecting the maps can be viewed as collecting the row vectors $\mathbf{w}_k^{[\ell]}$ into a matrix $W^{[\ell]}$ that represents the linear functional part of $T^{[\ell]}$ and a vector of biases because

$$\begin{aligned}
T(\mathbf{a}^{[\ell-1]}) = \mathbf{z}^{[\ell]} &= \begin{bmatrix} z_1^{[\ell]} \\ \vdots \\ z_{d^{[\ell]}}^{[\ell]} \end{bmatrix} = \begin{bmatrix} \left\langle \mathbf{a}^{[\ell-1]}, (\mathbf{w}_1^{[\ell]})^\top \right\rangle + \mathbf{b}_1^{[\ell]} \\ \vdots \\ \left\langle \mathbf{a}^{[\ell-1]}, (\mathbf{w}_{d^{[\ell]}}^{[\ell]})^\top \right\rangle + \mathbf{b}_{d^{[\ell]}}^{[\ell]} \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^{[\ell]} \mathbf{a}^{[\ell-1]} \\ \vdots \\ \mathbf{w}_{d^{[\ell]}}^{[\ell]} \mathbf{a}^{[\ell-1]} \end{bmatrix} + \begin{bmatrix} b_1^{[\ell]} \\ \vdots \\ b_{d^{[\ell]}}^{[\ell]} \end{bmatrix} \\
&= \begin{bmatrix} -\mathbf{w}_1^{[\ell]} - \\ \vdots \\ -\mathbf{w}_{d^{[\ell]}}^{[\ell]} - \end{bmatrix} \mathbf{a}^{[\ell-1]} + \begin{bmatrix} b_1^{[\ell]} \\ \vdots \\ b_{d^{[\ell]}}^{[\ell]} \end{bmatrix} \\
&=: W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}
\end{aligned}$$

- We can also collect the maps $g_1^{[\ell]}, \dots, g_{d^{[\ell]}}^{[\ell]}$ into the following vector-to-vector map that acts component-wise

$$g^{[\ell]}: \mathbf{z}^{[\ell]} \mapsto \begin{bmatrix} g_1^{[\ell]}(z_1^{[\ell]}) \\ \vdots \\ g_{d^{[\ell]}}^{[\ell]}(z_{d^{[\ell]}}^{[\ell]}) \end{bmatrix} = \begin{bmatrix} a_1^{[\ell]} \\ \vdots \\ a_{d^{[\ell]}}^{[\ell]} \end{bmatrix} =: \mathbf{a}^{[\ell]}$$

i.e. $\mathbf{a}^{[\ell]} := g^{[\ell]}(\mathbf{z}^{[\ell]})$.

In summary, at each layer ℓ , the following equations

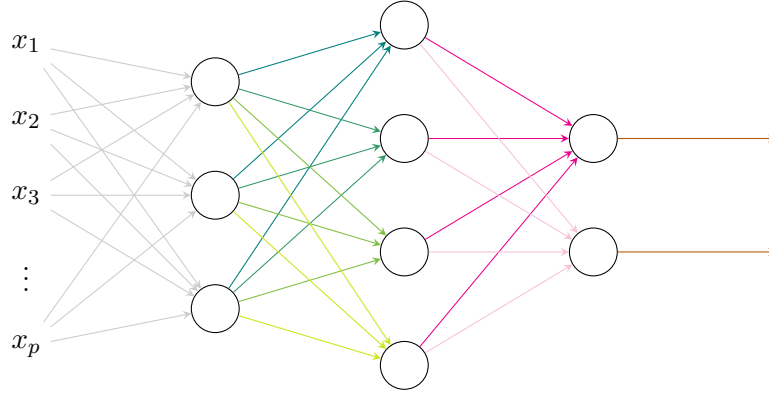
- $\mathbf{z}^{[\ell]} = W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}$
- $\mathbf{a}^{[\ell]} = g^{[\ell]}(\mathbf{z}^{[\ell]})$,

starting with $\mathbf{a}^{[0]} = \mathbf{x}$, describe the flow of information through the neural network and are known as the **forward propagation equations**.

Example 6.3.2 Say we modify the logistic regression neural network in **Example 6.2.6** by adding more layers and nodes.

In the case of this binary classification task of detecting a cat, the first layers of a deeper neural network would typically understand the fundamental concepts of the image e.g. the edges. Later (hidden) layers will use these edges to figure out more secondary information like the presence of ears, a mouth etc. and they'll communicate this information through the network to the output layer which will 'construct' the face of the animal based on what it received, and be able to tell if there's a cat or not. This idea of "communicating" the information manifests in the idea that all the nodes will be connected and push information forward (see **Definition 6.3.1**).

To this end, when the architecture is particularly deep/wide/complex we don't really know what the hidden layers are going to figure out **but** with enough data, the neural network should 'understand' very complex information about the data. The deeper you go, the more complex information the neurons are able to 'understand'. This is the reason why neural networks are often referred to as **black-box models**, and the employment of a neural network for this kind of learning is called **end-to-end learning** — we have a ground truth (predictions), and we aren't constraining the network in the hidden layers. We simply let it learn what it's going to learn.



Henceforth, I will say $\mathbf{z}^{[\ell]}$ is of size $(m, 1)$ when $\mathbf{z}^{[\ell]} \in \mathbb{R}^{m,1}$. Similarly, $W^{[\ell]}$ is of size (m, n) when $W^{[\ell]} \in \mathbb{R}^{m,n}$.

The forward propagation equations are:

- $\mathbf{z}^{[1]} = W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$

There are 3 neurons in layer 1 i.e. $\mathbf{z}^{[1]}$ is of size $(3, 1)$. This means that $\mathbf{b}^{[1]}$ is also of size $(3, 1)$, and because $\mathbf{x}$ is of size $(p, 1)$, we conclude that $W^{[1]}$ is a matrix of size $(3, p)$.

- $\mathbf{a}^{[1]} = g^{[1]}(\mathbf{z}^{[1]})$

Since $g^{[1]}$ acts component-wise, it preserves the dimensions of $\mathbf{z}^{[1]}$ when mapping it to $\mathbf{a}^{[1]}$. Thus, $\mathbf{a}^{[1]}$ is of size $(3, 1)$.

- $\mathbf{z}^{[2]} = W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$

There are 4 neurons in layer 2 i.e. $\mathbf{z}^{[2]}$ is of size $(4, 1)$ so $\mathbf{b}^{[2]}$ has size $(4, 1)$, and $\mathbf{a}^{[1]}$ is of size $(3, 1)$ so $W^{[2]}$ must be of size $(4, 3)$.

- $\mathbf{a}^{[2]} = g^{[2]}(\mathbf{z}^{[2]})$

As before, $\mathbf{a}^{[2]}$ is of the same size as $\mathbf{z}^{[2]}$ i.e. $(4, 1)$.

- $\mathbf{z}^{[3]} = W^{[3]}\mathbf{a}^{[2]} + \mathbf{b}^{[3]}$

There are two neurons in layer 3 i.e. $\mathbf{z}^{[3]}$ is of size $(2, 1)$ so $\mathbf{b}^{[3]}$ has size $(2, 1)$, and $\mathbf{a}^{[2]}$ is of size $(4, 1)$ so $W^{[3]}$ must be of size $(2, 4)$.

- $\mathbf{a}^{[3]} = g^{[3]}(\mathbf{z}^{[3]})$

Finally, $\mathbf{a}^{[3]}$ is of size $(2, 1)$ so there are 2 outputs from this neural network.



In general, the dimensions of the objects appearing at each layer in the forward-propagation equations are:

- $\mathbf{z}^{[\ell]}, \mathbf{a}^{[\ell]} \in \mathbb{R}^{d^{[\ell]}, 1}$
- $W^{[\ell]} \in \mathbb{R}^{d^{[\ell]}, d^{[\ell-1]}}$
- $\mathbf{b}^{[\ell]} \in \mathbb{R}^{d^{[\ell]}, 1}$

and the maps have domains and codomains

- $T^{[\ell]}: \mathbb{R}^{d^{[\ell-1]}} \rightarrow \mathbb{R}^{d^{[\ell]}}$
- $g^{[\ell]}: \mathbb{R}^{d^{[\ell]}} \rightarrow \mathbb{R}^{d^{[\ell]}}$

6.3.3 THE WHOLE NETWORK

Definition 6.3.3 Given the above notational convention, we may represent a feedforward neural network of depth L , denoted by $F^{[L]}(\mathbf{x})$, by the composition

$$\mathbf{x} = \mathbf{a}^{[0]} \xrightarrow{T^{[1]}} \mathbf{z}^{[1]} \xrightarrow{g^{[1]}} \mathbf{a}^{[1]} \xrightarrow{T^{[2]}} \dots \xrightarrow{g^{[L-1]}} \mathbf{a}^{[L-1]} \xrightarrow{T^{[L]}} \mathbf{z}^{[L]} \xrightarrow{g^{[L]}} \mathbf{a}^{[L]} =: F^{[L]}(\mathbf{x}).$$

Definition 6.3.4 Equivalently, we may define $F^{[L]}(\mathbf{x})$ recursively by

- $F^{[1]}(\mathbf{x}) = (g^{[1]} \circ T^{[1]})(\mathbf{x})$
- $F^{[\ell]}(\mathbf{x}) = (g^{[\ell]} \circ T^{[\ell]})(F^{[\ell-1]}(\mathbf{x}))$ for any $1 < \ell \leq L$.

6.3.4 FORWARD BATCH PROPAGATION

Continuing on with **Example 6.3.2**, we can go one step further and vectorise once more to capture the idea of training on a batch of examples $\mathcal{T}$ simultaneously. For every training example, we have a set of forward propagation equations. For example, we can write out the first propagation equation for each example:

$$\begin{aligned} \mathbf{z}^{1} &= W^{[1]}\mathbf{x}^{(1)} + \mathbf{b}^{[1]}, \\ &\vdots \\ \mathbf{z}^{[1](n)} &= W^{[1]}\mathbf{x}^{(n)} + \mathbf{b}^{[1]}. \end{aligned}$$

If we stack the $\mathbf{z}^{[1](i)}$ of size $(3, 1)$ side-by-side into a matrix $Z^{[1]}$ of size $(3, n)$, and we stack the examples side-by-side into a matrix X of size (p, n) like so

$$Z^{[1]} = \begin{bmatrix} | & & | \\ \mathbf{z}^{1} & \dots & \mathbf{z}^{[1](n)} \\ | & & | \end{bmatrix} \quad \text{and} \quad X = \begin{bmatrix} | & & | \\ \mathbf{x}^{(1)} & \dots & \mathbf{x}^{(n)} \\ | & & | \end{bmatrix},$$

then we can immediately see that $W^{[1]}$ being of size $(3, p)$ matches the dimensions of X for matrix multiplication. All that remains is to collect the bias terms into a single matrix $B^{[1]}$ that has the same dimensions as $Z^{[1]}$ so we may write $Z^{[1]} = W^{[1]}X + B^{[1]}$. The number of parameters should remain the same so we simply **broadcast** the bias $\mathbf{b}^{[1]}$ along the columns of $B^{[1]}$

$$B^{[1]} = \begin{bmatrix} | & & | \\ \mathbf{b}^{[1]} & \dots & \mathbf{b}^{[1]} \\ | & & | \end{bmatrix}.$$

Therefore, we may write the vectorised equation

$$Z^{[1]} = W^{[1]}X + B^{[1]}.$$

Again, we collate the activations $g^{[1]}$ into a map $G^{[1]}$ that acts component-wise on the columns of the matrix being fed to it

$$G^{[1]}: Z^{[1]} = \begin{bmatrix} | & & | \\ \mathbf{z}^{1} & \dots & \mathbf{z}^{[1](n)} \\ | & & | \end{bmatrix} \mapsto \begin{bmatrix} | & & | \\ g^{[1]}(\mathbf{z}^{1}) & \dots & g^{[1]}(\mathbf{z}^{[1](n)}) \\ | & & | \end{bmatrix}$$

and so the other first forward propagation equation is $A^{[1]} := G^{[1]}(Z^{[1]})$.

Thus, at each layer ℓ , we have the following **forward batch propagation equations**

- $Z^{[\ell]} = W^{[\ell]}A^{[\ell-1]} + B^{[\ell]}$
- $A^{[\ell]} = G^{[\ell]}(Z^{[\ell]})$

where $A^{[0]} = X$.

Deciding how many neurons/layers there are in an architecture can be done by training, say 10, different architectures on a training set, looking at the validation set accuracy of them all, and deciding which is best. Experience helps too, for each problem has its own complexity. The more complicated the problem, the deeper the network ought to be and the more data one should ostensibly use in order to capture more salient features.

6.4 Neural Network Loss

The toy example offers some good insight but the calculations themselves really are no more complicated than just working out the general layer-by-layer expressions themselves.

Consider a general feedforward neural network with L layers, and $d^{[\ell]}$ neurons in the ℓ^{th} layer, whose purpose it is to approximate a function that classifies a cat in an image as before. Each neuron uses its parameters to compute a similarity between the input and a template (represented by the parameters) for each kind of salient feature one can locate within the input image. Earlier layers may correspond to edge-recognition, and later hidden layers to a combination of edges that identify certain shapes like a mouth, or ears, or eyes. The further down the network we go, these simpler templates (e.g. $W^{[1]}$, $\mathbf{b}^{[1]}$, $W^{[2]}$, $\mathbf{b}^{[2]}$) are combined/propagated through the network to form a clearer “image” of the input as it’s processed through. Our training method should, as we encounter more input examples, update these parameters to more closely reflect the true underlying template. This was the guiding intuition for the perceptron, and it’s no different here.

We want to find these optimal parameters by gradient descent. First off, we need a loss function to quantify how good our predictions are. Recall that, for a supervised learning problem, we define the loss of a hypothesis h on a single example $(\mathbf{x}^{(i)}, y^{(i)})$ by the map $\mathcal{L}_h: \mathcal{X} \times \mathcal{Y} \rightarrow [0, +\infty)$ defined by

$$\mathcal{L}: (\mathbf{x}^{(i)}, y^{(i)}) \mapsto \mathcal{L}_h(\mathbf{x}^{(i)}, y^{(i)}) := \mathcal{L}(h(\mathbf{x}^{(i)}), y^{(i)}),$$

where $\mathcal{L}: \mathcal{Y} \times \mathcal{Y} \rightarrow [0, +\infty)$. In our case, the hypothesis h is our neural network $F^{[L]}$. We can collect the parameters on which our neural network depends on into the vector

$$\tilde{\theta} = \begin{bmatrix} W^{[1]} \\ \mathbf{b}^{[1]} \\ \vdots \\ W^{[L]} \\ \mathbf{b}^{[L]} \end{bmatrix} \text{ which lives in the parameter space } \tilde{\Theta} = \prod_{\ell=1}^L \left(\mathbb{R}^{d^{[\ell]}, d^{[\ell-1]}} \times \mathbb{R}^{d^{[\ell]}} \right).$$

Important technical point:

The method we'll employ to optimise these parameters is gradient descent. We need our loss functions to be scalar functions i.e. maps from $\mathbb{R}^P \rightarrow \mathbb{R}$ for some positive integer P . Our parameter space $\tilde{\Theta}$ is canonically isomorphic to $\mathbb{R}^P =: \Theta$ via the map $\Phi: \tilde{\Theta} \rightarrow \mathbb{R}^P = \Theta$ defined by

$$\Phi: \tilde{\theta} = \begin{bmatrix} W^{[1]} \\ \mathbf{b}^{[1]} \\ \vdots \\ W^{[L]} \\ \mathbf{b}^{[L]} \end{bmatrix} \mapsto \begin{bmatrix} \text{vec}(W^{[1]}) \\ \mathbf{b}^{[1]} \\ \vdots \\ \text{vec}(W^{[L]}) \\ \mathbf{b}^{[L]} \end{bmatrix} =: \theta,$$

where vec is a map that flattens a matrix into a column vector, and the dimension of the codomain

$$P = \sum_{\ell=1}^L (d^{[\ell]} d^{[\ell-1]} + d^{[\ell]})$$

is the total number of parameters in θ . This map is said to “flatten” the parameter space into $\mathbb{R}^P$.



I'm not sure which of stacking columns, or transposing rows and then stacking those is appropriate. I don't imagine there's much difference.

Now we may write the explicit dependence of our network on the parameters θ (viewed as a vector in $\mathbb{R}^P$) as

$$F^{[L]}(\mathbf{x}) = F_{\theta}^{[L]}(\mathbf{x}).$$

Definition 6.4.1

- The loss of our neural network on a fixed training example $(\mathbf{x}^{(i)}, y^{(i)})$ is defined by the composition

$$\mathcal{L}^{(i)}: \Theta \xrightarrow{\theta \mapsto F_{\theta}^{[L]}(\mathbf{x}^{(i)})} \mathcal{Y} \xrightarrow{\mathcal{L}(\cdot, y^{(i)})} \mathbb{R},$$

where $\mathcal{L}^{(i)}: \Theta \rightarrow \mathbb{R}$ is a scalar function.

- For the entire training set, we define the cost function $\mathcal{J}: \Theta \rightarrow \mathbb{R}$ by²

$$\mathcal{J}(\theta) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}^{(i)}(\theta).$$

6.5 Back-propagation Equations

Since differentiation is a linear map, we can save ourselves some ink³ and work with the derivatives of $\mathcal{L}^{(i)}$ instead of $\mathcal{J}$. The Fréchet derivative of $\mathcal{L}^{(i)}$ at θ is the linear map which acts as follows

$$d\mathcal{L}^{(i)}(\theta)\mathbf{h} = \sum_{\ell,j,k} \frac{\partial \mathcal{L}^{(i)}}{\partial W_{jk}^{[\ell]}}(\theta) \mathbf{h}_{jk}^{W^{[\ell]}} + \sum_{\ell,j} \frac{\partial \mathcal{L}^{(i)}}{\partial b_j^{[\ell]}}(\theta) \mathbf{h}_j^{\mathbf{b}^{[\ell]}}$$

²The normalisation $1/n$ isn't mandatory but it helps to compare different models.

³Maybe the disclaimer uses more ink than would've been used to write the summations every time...

where

$$\mathbf{h} = \begin{bmatrix} \text{vec}(\mathbf{h}^{W^{[1]}}) \\ \mathbf{h}^{\mathbf{b}^{[1]}} \\ \vdots \\ \text{vec}(\mathbf{h}^{W^{[L]}}) \\ \mathbf{h}^{\mathbf{b}^{[L]}} \end{bmatrix} \in \mathbb{R}^P,$$

each $\mathbf{h}^{W^{[\ell]}}$ is a matrix with dimension matching the corresponding $W^{[\ell]}$ in $\tilde{\boldsymbol{\theta}}$, and each $\mathbf{h}^{\mathbf{b}^{[\ell]}}$ is a vector of the same length as the corresponding $\mathbf{b}^{[\ell]}$ in $\tilde{\boldsymbol{\theta}}$. By **Corollary A.1.4**, recall that the gradient is the transpose of the coordinate representation of the Fréchet derivative of a scalar function. Let

- $\frac{\partial \mathcal{L}^{(i)}}{\partial W^{[\ell]}}$ denote the matrix whose $(j, k)^{\text{th}}$ entry is the partial derivative $\frac{\partial \mathcal{L}^{(i)}}{\partial W_{jk}^{[\ell]}}(\boldsymbol{\theta})$, and
- $\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[\ell]}}$ denote the column vector whose j^{th} entry is the partial derivative $\frac{\partial \mathcal{L}^{(i)}}{\partial b_j^{[\ell]}}(\boldsymbol{\theta})$.

Note that my notation omits the variable. This is deliberate, and I do so to distinguish this object (a matrix/vector of partials) from an honest-to-goodness derivative.

Then our expression for the gradient of $\mathcal{L}^{(i)}$ becomes

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) = \begin{bmatrix} \text{vec} \left(\frac{\partial \mathcal{L}^{(i)}}{\partial W^{[1]}} \right) \\ \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[1]}} \\ \vdots \\ \text{vec} \left(\frac{\partial \mathcal{L}^{(i)}}{\partial W^{[L]}} \right) \\ \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[L]}} \end{bmatrix} = \Phi \begin{pmatrix} \left[\frac{\partial \mathcal{L}^{(i)}}{\partial W^{[1]}} \right] \\ \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[1]}} \\ \vdots \\ \left[\frac{\partial \mathcal{L}^{(i)}}{\partial W^{[L]}} \right] \\ \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[L]}} \end{pmatrix}.$$

The term acted on by Φ is often conflated with the gradient of the scalar function $\mathcal{L}^{(i)}: \mathbb{R}^P \rightarrow \mathbb{R}$ but this is a type error. Never sacrifice notational convenience for lack of clarity. However, it is very convenient to collect the gradient descent update rules into matrix-form for weights, and vector-form for biases as follows:

$$\begin{cases} W^{[\ell]} \leftarrow W^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial W^{[\ell]}}, \\ \mathbf{b}^{[\ell]} \leftarrow \mathbf{b}^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[\ell]}}. \end{cases}$$

6.5.1 UPDATE DERIVATIVES

The key observation to calculating the derivatives with respect to the parameters $W_{jk}^{[\ell]}$ and $b_j^{[\ell]}$ is that $\mathcal{L}^{(i)}$ depends on them through $\mathbf{z}^{[\ell(i)]}$. To this end, we want to investigate what happens to $\mathcal{L}^{(i)}$ when we perturb $\mathbf{z}^{[\ell]}$ about $\mathbf{z}^{[\ell(i)]}$ but so far we only have that $\mathcal{L}^{(i)}$ maps $\Theta \rightarrow \mathbb{R}$. Clearly we need to factor $\mathcal{L}^{(i)}$ by some function that takes $\mathbf{z}^{[\ell]}$ as input in order to investigate (by the chain rule) the effect of changing each $z_j^{[\ell]}$. We can leverage the forward-propagation equations (from end to beginning) to achieve this:

$$\begin{aligned}
\mathcal{L}^{(i)}(\boldsymbol{\theta}) &= L(F_{\boldsymbol{\theta}}^{[L]}(\mathbf{x}^{(i)}), y^{(i)}) = \mathcal{L}(a^{[L](i)}, y^{(i)}) \\
&= \mathcal{L}(g^{[L]}(\mathbf{z}^{[L](i)}), y^{(i)}) \\
&= \mathcal{L}(g^{[L]}(T^{[L]}(\mathbf{a}^{[L-1](i)})), y^{(i)}) \\
&= \mathcal{L}(g^{[L]}(T^{[L]}(g^{[L-1]}(\mathbf{z}^{[L-1](i)}))), y^{(i)}) \\
&= \dots \\
&= \mathcal{L}((g^{[L]} \circ T^{[L]} \circ g^{[L-1]} \circ \dots \circ T^{[\ell+1]} \circ g^{[\ell]})(\mathbf{z}^{[\ell](i)}), y^{(i)})
\end{aligned}$$

Define $\Phi^{[\ell]} := \mathcal{L}(\cdot, y^{(i)}) \circ (g^{[L]} \circ T^{[L]} \circ g^{[L-1]} \circ \dots \circ T^{[\ell+1]} \circ g^{[\ell]})$. Note that as a map $\Phi^{[\ell]}: \mathbb{R}^{d^{[\ell]}} \rightarrow \mathbb{R}$, and upon taking $\mathbf{z}^{[\ell](i)}$ as input, outputs $\Phi^{[\ell]}(\mathbf{z}^{[\ell](i)}) = \mathcal{L}(\mathbf{a}^{[L](i)}, y^{(i)}) = \mathcal{L}^{(i)}(\boldsymbol{\theta})$. The nature of $\Phi^{[\ell]}$'s definition is recursive

$$\begin{aligned}
\Phi^{[\ell]} &= \mathcal{L}(\cdot, y^{(i)}) \circ (g^{[L]} \circ T^{[L]} \circ g^{[L-1]} \circ \dots \circ g^{[\ell+1]} \circ T^{[\ell+1]} \circ g^{[\ell]}) \\
&= \mathcal{L}(\cdot, y^{(i)}) \circ (g^{[L]} \circ T^{[L]} \circ g^{[L-1]} \circ \dots \circ g^{[\ell+1]}) \circ T^{[\ell+1]} \circ g^{[\ell]} \\
&= \Phi^{[\ell+1]} \circ T^{[\ell+1]} \circ g^{[\ell]}
\end{aligned}$$

and we'll use this to propagate the error derivatives through the network. The complementary function to $\Phi^{[\ell]}$ is $\mathcal{Z}^{[\ell](i)}$ which summarises the first ℓ layers of the network (minus the activation at layer ℓ). Thus, $\mathcal{Z}^{[\ell](i)}(\boldsymbol{\theta}) = \mathbf{z}^{[\ell](i)}$, and so $\mathcal{L}^{(i)}(\boldsymbol{\theta}) = (\Phi^{[\ell]} \circ \mathcal{Z}^{[\ell](i)})(\boldsymbol{\theta})$. By the chain rule,

$$\begin{aligned}
d\mathcal{L}^{(i)}(\boldsymbol{\theta}) &= d\Phi^{[\ell]}(\mathcal{Z}^{[\ell](i)}(\boldsymbol{\theta})) \circ d\mathcal{Z}^{[\ell](i)}(\boldsymbol{\theta}) \\
&= d\Phi^{[\ell]}(\mathbf{z}^{[\ell](i)}) \circ d\mathcal{Z}^{[\ell](i)}(\boldsymbol{\theta})
\end{aligned}$$

We'll use the above equality to extract the partial derivative of $\Phi^{[\ell]}$ with respect to $z_j^{[\ell]}$ at $\mathbf{z}^{[\ell](i)}$. Indeed, if we let $\mathbf{h}$ be s.t. $\boldsymbol{\epsilon}_j = d\mathcal{Z}^{[\ell](i)}(\boldsymbol{\theta})\mathbf{h}$, then the above equality becomes

$$d\mathcal{L}^{(i)}(\boldsymbol{\theta})\mathbf{h} = d\Phi^{[\ell]}(\mathbf{z}^{[\ell](i)})\boldsymbol{\epsilon}_j = \frac{\partial \Phi^{[\ell]}}{(\mathbf{z}^{[\ell]})_j}(\mathbf{z}^{[\ell]}) = \frac{\partial \Phi^{[\ell]}}{z_j^{[\ell]}}(\mathbf{z}^{[\ell]}),$$

where the penultimate equality is simply the definition of the partial derivative as the Fréchet derivative at a point in the direction of the j^{th} standard basis vector (and the final equality is just noting that $(\mathbf{z}^{[\ell]})_j = z_j^{[\ell]}$).

Since we know that $\Phi^{[\ell]}(\mathbf{z}^{[\ell](i)}) = \mathcal{L}^{(i)}(\boldsymbol{\theta})$, we define

$$\frac{\partial \Phi^{[\ell]}}{\partial z_j^{[\ell]}}(\mathbf{z}^{[\ell](i)}) =: \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}).$$

We can deal with the derivative of $\Phi^{[\ell]}$ at any $\mathbf{z}^{[\ell]} \in \mathbb{R}^{d^{[\ell]}}$ straight-forwardly using its recursive nature. By the chain rule applied twice,

$$\begin{aligned}
d\Phi^{[\ell]}(\mathbf{z}^{[\ell]}) &= d(\Phi^{[\ell+1]} \circ T^{[\ell+1]})(g^{[\ell]}(\mathbf{z}^{[\ell]})) \circ dg^{[\ell]}(\mathbf{z}^{[\ell]}) \\
&= d\Phi^{[\ell+1]}(T^{[\ell+1]}(g^{[\ell]}(\mathbf{z}^{[\ell]}))) \circ dT^{[\ell+1]}(g^{[\ell]}(\mathbf{z}^{[\ell]})) \circ dg^{[\ell]}(\mathbf{z}^{[\ell]}) \\
&= d\Phi^{[\ell+1]}(T^{[\ell+1]}(g^{[\ell]}(\mathbf{z}^{[\ell]}))) \circ L_{W^{[\ell+1]}} \circ \text{diag} \left(\frac{dg_1^{[\ell]}}{dz_1^{[\ell]}}(z_1^{[\ell]}), \dots, \frac{dg_{d^{[\ell]}}^{[\ell]}}{dz_{d^{[\ell]}}^{[\ell]}}(z_{d^{[\ell]}}^{[\ell]}) \right)
\end{aligned}$$

where $L_A \mathbf{v} = A\mathbf{v}$ denotes the linear map corresponding to left-multiplication of its matrix representative with respect to the appropriate standard bases.

At $\mathbf{z}^{[\ell]} = \mathbf{z}^{[\ell](i)}$, when applied to the standard basis vector $\mathbf{e}_j$, we recover an expression for our partial derivative as follows:

$$\begin{aligned}
\frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) &= \frac{\partial \Phi^{[\ell]}}{\partial z_j^{[\ell]}}(\mathbf{z}^{[\ell](i)}) = d\Phi^{[\ell]}(\mathbf{z}^{[\ell](i)})\mathbf{e}_j \\
&= d\Phi^{[\ell+1]}(\mathbf{z}^{[\ell+1](i)}) \left(W^{[\ell+1]} \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \mathbf{e}_j \right) \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) d\Phi^{[\ell+1]}(\mathbf{z}^{[\ell+1](i)})(W^{[\ell+1]}\mathbf{e}_j) \quad \text{by linearity} \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) d\Phi^{[\ell+1](i)}(\mathbf{z}^{[\ell+1](i)}) \left(\begin{bmatrix} W_{1j}^{[\ell+1]} \\ \vdots \\ W_{d^{[\ell+1]}j}^{[\ell+1]} \end{bmatrix} \right) \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) d\Phi^{[\ell+1]}(\mathbf{z}^{[\ell+1](i)}) \left(\sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} \mathbf{e}_m \right) \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} d\Phi^{[\ell+1]}(\mathbf{z}^{[\ell+1](i)})\mathbf{e}_m \quad \text{by linearity} \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} \frac{\partial \Phi^{[\ell+1]}}{\partial (\mathbf{z}^{[\ell+1]})_m}(\mathbf{z}^{[\ell+1](i)}) \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} \frac{\partial \Phi^{[\ell+1]}}{\partial z_m^{[\ell+1]}}(\mathbf{z}^{[\ell+1](i)}) \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} \frac{\partial \mathcal{L}^{(i)}}{\partial z_m^{[\ell+1]}}(\boldsymbol{\theta}).
\end{aligned}$$

$$\therefore \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) = \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \sum_{m=1}^{d^{[\ell+1]}} W_{mj}^{[\ell+1]} \frac{\partial \mathcal{L}^{(i)}}{\partial z_m^{[\ell+1]}}(\boldsymbol{\theta})$$

We can collect the above equations in vector/matrix form.

$$\begin{aligned}
\frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) &= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \begin{bmatrix} W_{1j}^{[\ell+1]} & \dots & W_{d^{[\ell+1]}j}^{[\ell+1]} \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}^{(i)}}{\partial z_1^{[\ell+1]}}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial \mathcal{L}^{(i)}}{\partial z_{d^{[\ell+1]}}^{[\ell+1]}}(\boldsymbol{\theta}) \end{bmatrix} \\
&= \frac{dg_j^{[\ell]}}{dz_j^{[\ell]}}(z_j^{[\ell](i)}) \left\langle \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}), (W^{[\ell+1]})_j \right\rangle
\end{aligned}$$

where $(W^{[\ell+1]})_j$ denotes the j^{th} column of the $\mathbb{R}^{d^{[\ell+1]}, d^{[\ell]}}$ matrix $W^{[\ell+1]}$ so there are $d^{[\ell]}$ columns of $W^{[\ell+1]}$.

If we collect the derivatives of the $g_j^{[\ell]}$ into a vector

$$(g^{[\ell]})'(\mathbf{z}^{[\ell](i)}) := \begin{bmatrix} \frac{dg_1^{[\ell]}}{dz_1^{[\ell]}}(z_1^{[\ell](i)}) \\ \vdots \\ \frac{dg_{d^{[\ell]}}^{[\ell]}}{dz_{d^{[\ell]}}^{[\ell]}}(z_{d^{[\ell]}}^{[\ell](i)}) \end{bmatrix},$$

then we may stack our partial derivatives of $\mathcal{L}^{(i)}(\boldsymbol{\theta})$ with respect to the $z_j^{[\ell](i)}$ into the following gradient:

$$\begin{aligned} \nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) &= \begin{bmatrix} \frac{dg_1^{[\ell]}}{dz_1^{[\ell]}}(z_1^{[\ell](i)}) \\ \vdots \\ \frac{dg_{d^{[\ell]}}^{[\ell]}}{dz_{d^{[\ell]}}^{[\ell]}}(z_{d^{[\ell]}}^{[\ell](i)}) \end{bmatrix} \odot \begin{bmatrix} \langle \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}), (W^{[\ell+1]})_1 \rangle \\ \vdots \\ \langle \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}), (W^{[\ell+1]})_{d^{[\ell]}} \rangle \end{bmatrix} \\ &= (g^{[\ell]})'(\mathbf{z}^{[\ell](i)}) \odot \left((W^{[\ell+1]})^\top \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \right), \end{aligned}$$

where $\odot$ is the component-wise multiplication of vectors, known as the **Hadamard product**.

Definition 6.5.1 The system of equations

$$\nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) = (g^{[\ell]})'(\mathbf{z}^{[\ell](i)}) \odot \left((W^{[\ell+1]})^\top \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \right)$$

are known as the **back-propagation equations** for our feedforward neural network.

Recall that our update rules are

$$\begin{cases} W^{[\ell]} \leftarrow W^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial W^{[\ell]}}, \\ \mathbf{b}^{[\ell]} \leftarrow \mathbf{b}^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{b}^{[\ell]}}, \end{cases}$$

where the terms multiplied by the learning rates are matrices of partial derivatives with respect to the $W_{jk}^{[\ell]}$ and $b_j^{[\ell]}$, respectively. By the chain rule from multivariable calculus (i.e. in coordinates), we observe that

$$\begin{aligned} \bullet \frac{\partial \mathcal{L}^{(i)}}{\partial W_{jk}^{[\ell]}}(\boldsymbol{\theta}) &= \sum_{m=1}^{d^{[\ell]}} \frac{\partial \Phi^{[\ell]}}{\partial z_m^{[\ell]}}(\mathbf{z}^{[\ell](i)}(\boldsymbol{\theta})) \frac{\partial z_m^{[\ell]}}{\partial W_{jk}^{[\ell]}}(\boldsymbol{\theta}) \\ \bullet \frac{\partial \mathcal{L}^{(i)}}{\partial b_j^{[\ell]}}(\boldsymbol{\theta}) &= \sum_{m=1}^{d^{[\ell]}} \frac{\partial \Phi^{[\ell]}}{\partial z_m^{[\ell]}}(\mathbf{z}^{[\ell](i)}(\boldsymbol{\theta})) \frac{\partial z_m^{[\ell]}}{\partial b_j^{[\ell]}}(\boldsymbol{\theta}) \end{aligned}$$

and notice that we've computed the term in common $\frac{\partial \Phi^{[\ell]}}{\partial z_m^{[\ell]}}(\mathbf{z}^{[\ell](i)}(\boldsymbol{\theta}))$ that we defined as $\frac{\partial \mathcal{L}^{(i)}}{\partial z_m^{[\ell]}}(\boldsymbol{\theta})$.

6.5.2 INPUT INTO NEURON

The forward-propagation equation $\mathbf{z}^{[\ell](i)} = W^{[\ell]} \mathbf{a}^{[\ell-1](i)} + \mathbf{b}^{[\ell]}$ tells us, in coordinates, that:

$$\begin{aligned} z_m^{[\ell](i)} &= \sum_{k=1}^{d^{[\ell-1]}} \left(\mathbf{w}_m^{[\ell]} \right)_k a_k^{[\ell-1](i)} + b_m^{[\ell]} \\ &= \sum_{k=1}^{d^{[\ell-1]}} W_{mk}^{[\ell]} a_k^{[\ell-1](i)} + b_m^{[\ell]}. \end{aligned}$$

Thus, the partial derivative of $z_m^{[\ell](i)}$ with respect to $W_{jk}^{[\ell]}$ is

$$\frac{\partial z_m^{[\ell](i)}}{\partial W_{jk}^{[\ell]}}(\boldsymbol{\theta}) = \frac{\partial}{\partial W_{jk}^{[\ell]}} \left(\sum_{k=1}^{d^{[\ell-1]}} W_{mk}^{[\ell]} a_k^{[\ell-1](i)} + b_m^{[\ell]} \right) = \begin{cases} a_k^{[\ell-1](i)} & \text{if } m = j \\ 0 & \text{otherwise.} \end{cases}$$

Also, the partial derivative of $z_m^{[\ell](i)}$ with respect to $b_j^{[\ell]}$ is

$$\frac{\partial z_m^{[\ell](i)}}{\partial b_j^{[\ell]}}(\boldsymbol{\theta}) = \frac{\partial}{\partial b_j^{[\ell]}} \left(\sum_{k=1}^{d^{[\ell-1]}} W_{mk}^{[\ell]} a_k^{[\ell-1](i)} + b_m^{[\ell]} \right) = \begin{cases} 1 & \text{if } m = j \\ 0 & \text{otherwise.} \end{cases}$$

Therefore, our update derivatives collapse to

- $\frac{\partial \mathcal{L}^{(i)}}{\partial W_{jk}^{[\ell]}}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) a_k^{[\ell-1](i)}$ and
- $\frac{\partial \mathcal{L}^{(i)}}{\partial b_j^{[\ell]}}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}).$

Definition 6.5.2 The **gradient descent update formulae** for our feedforward neural network are:

$$\begin{cases} W_{jk}^{[\ell]} \leftarrow W_{jk}^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) a_k^{[\ell-1](i)} \\ b_j^{[\ell]} \leftarrow b_j^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial z_j^{[\ell]}}(\boldsymbol{\theta}) \end{cases}$$

We may easily stack the above update rules for the biases $b_j^{[\ell]}$ into a column vector:

$$\mathbf{b}^{[\ell]} \leftarrow \mathbf{b}^{[\ell]} - \alpha \nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}).$$

For the weights, the form is slightly different because we have a matrix of parameter updates. Our goal is to collect the terms into the matrix update rule

$$W^{[\ell]} \leftarrow W^{[\ell]} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial W^{[\ell]}}.$$

The matrix of partial derivatives can be written as the following outer product:

$$\begin{aligned}
 \frac{\partial \mathcal{L}^{(i)}}{\partial W^{[\ell]}} &= \begin{bmatrix} \frac{\partial \mathcal{L}^{(i)}}{\partial z_1^{[\ell]}}(\boldsymbol{\theta}) a_1^{[\ell-1](i)} & \dots & \frac{\partial \mathcal{L}^{(i)}}{\partial z_1^{[\ell]}}(\boldsymbol{\theta}) a_{d^{[\ell-1]}}^{[\ell-1](i)} \\ \vdots & & \vdots \\ \frac{\partial \mathcal{L}^{(i)}}{\partial z_{d^{[\ell]}}^{[\ell]}}(\boldsymbol{\theta}) a_1^{[\ell-1](i)} & \dots & \frac{\partial \mathcal{L}^{(i)}}{\partial z_{d^{[\ell]}}^{[\ell]}}(\boldsymbol{\theta}) a_{d^{[\ell-1]}}^{[\ell-1](i)} \end{bmatrix} \in \mathbb{R}^{d^{[\ell]}, d^{[\ell-1]}} \\
 &= \begin{bmatrix} \frac{\partial \mathcal{L}^{(i)}}{\partial z_1^{[\ell]}}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial \mathcal{L}^{(i)}}{\partial z_{d^{[\ell]}}^{[\ell]}}(\boldsymbol{\theta}) \end{bmatrix} \begin{bmatrix} a_1^{[\ell-1](i)} & \dots & a_{d^{[\ell-1]}}^{[\ell-1](i)} \end{bmatrix} \\
 &= \left(\nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \right) (\mathbf{a}^{[\ell-1](i)})^\top.
 \end{aligned}$$

Definition 6.5.3 Therefore, the **matrix-form** of our **gradient descent update formulae** for our weight matrices and bias vectors are:

$$\begin{cases} W^{[\ell]} \leftarrow W^{[\ell]} - \alpha \left(\nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \right) (\mathbf{a}^{[\ell-1](i)})^\top \\ \mathbf{b}^{[\ell]} \leftarrow \mathbf{b}^{[\ell]} - \alpha \nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \end{cases}$$

where

$$\nabla_{\mathbf{z}^{[\ell]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) = (g^{[\ell]})'(\mathbf{z}^{[\ell](i)}) \odot \left((W^{[\ell+1]})^\top \nabla_{\mathbf{z}^{[\ell+1]}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) \right).$$

Appendices

Differentiation

A.1 Fréchet Derivative

The most general (and incredibly useful!!) definition of derivative that I'm aware of is that of the Fréchet derivative of a function between normed vector spaces.

Definition A.1.1 Let $(\mathbb{X}, \|\cdot\|_{\mathbb{X}})$ and $(\mathbb{Y}, \|\cdot\|_{\mathbb{Y}})$ be normed spaces, $U \subseteq \mathbb{X}$ be open, $\mathbf{x} \in U$, and $f: U \rightarrow \mathbb{Y}$. If there is a bounded linear map $A_{\mathbf{x}}: \mathbb{X} \rightarrow \mathbb{Y}$ that satisfies

$$\lim_{\mathbf{v} \rightarrow \mathbf{0} \text{ in } \mathbb{X}} \frac{\|f(\mathbf{x} + \mathbf{v}) - f(\mathbf{x}) - A_{\mathbf{x}}\mathbf{v}\|_{\mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}} = 0 \quad (\text{A.1})$$

then f is said to be **Fréchet differentiable at $\mathbf{x}$** . The linear map $A_{\mathbf{x}}$ is then called the **Fréchet derivative of f at $\mathbf{x}$** and is denoted $df(\mathbf{x})$.

Corollary A.1.2 Let $\mathbb{X} = \mathbb{R}^n$ with the standard Euclidean basis $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$, $\mathbb{Y} = \mathbb{R}^m$ with similar remarks. Let $U \subseteq \mathbb{R}^n$ be open, and let $f: U \rightarrow \mathbb{R}^m$ be Fréchet differentiable at $\mathbf{x} \in U$. Then the Jacobian matrix $Jf(\mathbf{x})$ of partial derivatives of f at $\mathbf{x}$ exists and is the matrix of the Fréchet derivative $df(\mathbf{x})$ with respect to the standard Euclidean bases of $\mathbb{R}^n$ and $\mathbb{R}^m$ i.e

$$df(\mathbf{x})\mathbf{v} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}) & \frac{\partial f_1}{\partial x_2}(\mathbf{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(\mathbf{x}) & \frac{\partial f_m}{\partial x_2}(\mathbf{x}) & \cdots & \frac{\partial f_m}{\partial x_n}(\mathbf{x}) \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix}$$

for all $\mathbf{v} = \sum_{i=1}^n v_i \mathbf{e}_i \in \mathbb{R}^n$.

For the following, let $(\mathbb{X}, \|\cdot\|_{\mathbb{X}}) = (\mathbb{R}^n, \|\cdot\|)$, and $(\mathbb{Y}, \|\cdot\|_{\mathbb{Y}}) = (\mathbb{R}, |\cdot|)$.

Definition A.1.3 The **gradient of $f: \mathbb{R}^n \rightarrow \mathbb{R}$ at $\mathbf{x} \in \mathbb{R}^n$** is defined as the vector

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^n$$

of partial derivatives of f .

Corollary A.1.4 The coordinate representation $Jf(\mathbf{x})$ of the Fréchet derivative $df(\mathbf{x})$ of $f: \mathbb{R}^n \rightarrow \mathbb{R}$, and the gradient $\nabla f(\mathbf{x})$ of f at $\mathbf{x}$ are related by the transpose operation

$$Jf(\mathbf{x}) = (\nabla f(\mathbf{x}))^{\top}.$$

Proof. The coordinate representation of the Fréchet derivative of $f: \mathbb{R}^n \rightarrow \mathbb{R}$ with respect to the standard basis $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$ of $\mathbb{R}^n$ is a $1 \times n$ matrix i.e. a linear functional on $\mathbb{R}^n$, taking vectors to scalars i.e. a row of partial derivatives

$$Jf(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}(\mathbf{x}) \quad \cdots \quad \frac{\partial f}{\partial x_n}(\mathbf{x}) \right].$$

The claim follows. ■

Definition A.1.5 Let $(\mathbb{X}, \|\cdot\|_{\mathbb{X}})$ and $(\mathbb{Y}, \|\cdot\|_{\mathbb{Y}})$ be normed spaces, $U \subseteq \mathbb{X}$ be open, $\mathbf{x} \in U$, and $f: U \rightarrow \mathbb{Y}$. If there exists a bounded-linear-map-valued bounded linear map $A_{\mathbf{x}}: \mathbb{X} \rightarrow \mathcal{L}(\mathbb{X}; \mathbb{Y})$ that satisfies

$$\lim_{\mathbf{v} \rightarrow \mathbf{0} \text{ in } \mathbb{X}} \frac{\|df(\mathbf{x} + \mathbf{v}) - df(\mathbf{x}) - A_{\mathbf{x}}\mathbf{v}\|_{\mathbb{X} \rightarrow \mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}} = 0$$

then f is said to be twice **Fréchet differentiable at $\mathbf{x}$** . The linear map $A_{\mathbf{x}}$ is then called the second-order **Fréchet derivative of f at $\mathbf{x}$** and is denoted $d^2f(\mathbf{x})$.

Remarks A.1.6

- We can view an element of $\mathcal{L}(\mathbb{X}, \mathcal{L}(\mathbb{X}; \mathbb{Y}))$ as a bounded bilinear map $A_{\mathbf{x}}: \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{Y}$, treating $A_{\mathbf{x}}(\mathbf{v})\mathbf{w}$ as being the same as $A_{\mathbf{x}}(\mathbf{v}, \mathbf{w})$.
- The norm $\|\cdot\|_{\mathbb{X} \rightarrow \mathbb{Y}}$ is the operator norm. We say that a bilinear map $A: \mathbb{X}_1 \times \mathbb{X}_2 \rightarrow \mathbb{Y}$ is bounded if

$$\|A\|_{\mathbb{X} \rightarrow \mathbb{Y}} := \sup\{\|A(\mathbf{v}_1, \mathbf{v}_2)\|_{\mathbb{Y}} : \|\mathbf{v}_1\|_{\mathbb{X}} \leq 1, \|\mathbf{v}_2\|_{\mathbb{X}} \leq 1\} < \infty.$$

- If we let $A: \mathbb{X} \rightarrow \mathbb{Y}$ be a linear map between normed spaces, there are equivalent ways to express the operator norm:

$$\begin{aligned} \|A\|_{\mathbb{X} \rightarrow \mathbb{Y}} &= \sup_{0 \neq \mathbf{v} \in \mathbb{X}} \frac{\|A\mathbf{v}\|_{\mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}} \\ &= \sup_{\|\mathbf{v}\|_{\mathbb{X}}=1} \frac{\|A\mathbf{v}\|_{\mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}} = \sup_{\|\mathbf{v}\|_{\mathbb{X}}=1} \|A\mathbf{v}\|_{\mathbb{Y}}. \end{aligned}$$

- The limit in the definition of the second-order Fréchet derivative can be written more explicitly by expanding out the operator norm:

$$\begin{aligned} \lim_{\mathbf{v} \rightarrow \mathbf{0} \text{ in } \mathbb{X}} \frac{\|df(\mathbf{x} + \mathbf{v}) - df(\mathbf{x}) - A_{\mathbf{x}}\mathbf{v}\|_{\mathbb{X} \rightarrow \mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}} &= \lim_{\substack{\mathbf{v} \rightarrow \mathbf{0} \\ \text{in } \mathbb{X}}} \sup_{\mathbf{h} \neq \mathbf{0}} \frac{\|df(\mathbf{x} + \mathbf{v})\mathbf{h} - df(\mathbf{x})\mathbf{h} - (A_{\mathbf{x}}\mathbf{v})\mathbf{h}\|_{\mathbb{Y}}}{\|\mathbf{h}\|_{\mathbb{X}}\|\mathbf{v}\|_{\mathbb{X}}} \\ &= \lim_{\substack{\mathbf{v} \rightarrow \mathbf{0} \\ \text{in } \mathbb{X}}} \sup_{\|\mathbf{h}\|_{\mathbb{X}}=1} \frac{\|df(\mathbf{x} + \mathbf{v})\mathbf{h} - df(\mathbf{x})\mathbf{h} - (A_{\mathbf{x}}\mathbf{v})\mathbf{h}\|_{\mathbb{Y}}}{\|\mathbf{v}\|_{\mathbb{X}}}. \end{aligned}$$

Corollary A.1.7 Let $\mathbb{X} = \mathbb{R}^n$ with the standard Euclidean basis $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$, $\mathbb{Y} = \mathbb{R}$, $U \subseteq \mathbb{R}^n$ be open, and $f: U \rightarrow \mathbb{R}$ be twice Fréchet differentiable at $\mathbf{x} \in U$. Then the matrix representation of $d^2f(\mathbf{x})$ with respect to the standard basis of $\mathbb{R}^n$ is called the **Hessian matrix** $Hf(\mathbf{x})$. Its entries are the second-order partial derivatives of f at $\mathbf{x}$ i.e.

$$Hf(\mathbf{x}) := \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_1}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_1 \partial x_n}(\mathbf{x}) & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_n}(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{n \times n}$$

in the sense that for all

$$\mathbf{v} = \sum_{i=1}^n v_i \mathbf{e}_i \quad \text{and} \quad \mathbf{w} = \sum_{i=1}^n w_i \mathbf{e}_i,$$

we have that

$$d^2f(\mathbf{x})(\mathbf{v}, \mathbf{w}) = \mathbf{v}^\top Hf(\mathbf{x})\mathbf{w}.$$

A.1.1 CHAIN RULE

Theorem A.1.8 Let $(\mathbb{Z}, \|\cdot\|_{\mathbb{Z}})$ be a third normed space, $U \subseteq \mathbb{X}$ and $V \subseteq \mathbb{Y}$ be open. Let $f: U \rightarrow \mathbb{Y}$ be Fréchet differentiable at $\mathbf{x} \in U$, and $g: V \rightarrow \mathbb{Z}$ be Fréchet differentiable at $\mathbf{y} := f(\mathbf{x}) \in V$. Then $g \circ f: f^{-1}(V) \rightarrow \mathbb{Z}$ is Fréchet differentiable at $\mathbf{x} \in U$ and

$$d(g \circ f)(\mathbf{x}) = dg(f(\mathbf{x})) \circ df(\mathbf{x}): \mathbb{X} \rightarrow \mathbb{Z}.$$

Corollary A.1.9 (Chain Rule in Coordinates) Let $U \subseteq \mathbb{R}^p$ and $V \subseteq \mathbb{R}^n$ be open. Let $f: U \rightarrow \mathbb{R}^n$ be Fréchet differentiable at $\mathbf{x} \in U$, and $g: V \rightarrow \mathbb{R}^m$ be Fréchet differentiable at $\mathbf{y} := f(\mathbf{x}) \in V$. The Fréchet derivative $df(\mathbf{x}) \in \mathcal{L}(\mathbb{R}^p; \mathbb{R}^n)$ is represented by the Jacobian matrix $Jf(\mathbf{x}) \in \mathbb{R}^{n,p}$, and $dg(\mathbf{y}) \in \mathcal{L}(\mathbb{R}^n; \mathbb{R}^m)$ by $Jg(\mathbf{y}) \in \mathbb{R}^{m,n}$. By the chain rule, and because composition at the level of linear maps to matrix multiplication, we have that $J(g \circ f)(\mathbf{x}) = Jg(\mathbf{y})Jf(\mathbf{x}) \in \mathbb{R}^{m,p}$ i.e.

$$\frac{\partial(g \circ f)_i}{\partial x_j}(\mathbf{x}) = \sum_{k=1}^n \frac{\partial g_i}{\partial y_k}(\mathbf{y}) \frac{\partial f_k}{\partial x_j}(\mathbf{x}).$$

Definition A.1.10 Let $f: U \rightarrow \mathbb{Y}$ be k -times Fréchet differentiable at $\mathbf{x} \in U$. Then $T_{\mathbf{x}}^k f: \mathbb{X} \rightarrow \mathbb{Y}$ defined by

$$(T_{\mathbf{x}}^k f)(\mathbf{v}) := \sum_{j=0}^k \frac{d^j f(\mathbf{x}) \mathbf{v}^{(j)}}{j!}$$

is called the **k^{th} order Taylor expansion of f about $\mathbf{x}$** , where

$$\mathbf{v}^{(j)} := \begin{bmatrix} \mathbf{v} \\ \vdots \\ \mathbf{v} \end{bmatrix} \in \mathbb{X}^j.$$

Some reasoning for $\mathbf{v}^{(j)}$ is that, for example, the Hessian is a matrix in standard coordinates and evaluation is $\mathbf{v}^\top H \mathbf{v}$.

A.2 Matrix Calculus

Definition A.2.1 For a function $f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$, we define the derivative of f with respect to $A \in \mathbb{R}^{m \times n}$ by:

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}.$$

As a special case for a vector $\theta = [\theta_1, \dots, \theta_n]^\top \in \mathbb{R}^n$ and $J: \mathbb{R}^n \rightarrow \mathbb{R}$, we define

$$\nabla_\theta J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}.$$

Example A.2.2

- Some properties about the trace of a square matrix $\text{tr}(A) = \sum_i A_{ii}$.
 - For any two matrices A, B s.t. AB is square, $\text{tr}(AB) = \text{tr}(BA)$. As a corollary, there's a cyclic nature to the trace $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB)$.
 - $\text{tr}(A) = \text{tr}(A^\top)$
 - $\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B)$
 - $\text{tr}(aA) = a\text{tr}(A)$ for a scalar a .
- Some properties of the matrix derivative:
 - (1) $\nabla_A \text{tr}(AB) = B^\top$
 - (2) $\nabla_{A^\top} f(A) = (\nabla_A f(A))^\top$
 - (3) $\nabla_A \text{tr}(ABA^\top C) = CAB + C^\top AB^\top$
 - (4) $\nabla_A \det(A) = \det(A) \cdot (A^{-1})^\top$



The following is used in the derivation of the normal equation:
Combine (2) and (3) to make (5):

$$\begin{aligned} \nabla_{A^\top} \text{tr}(ABA^\top C) &\stackrel{(2)}{=} \left(\nabla_A \text{tr}(ABA^\top C) \right)^\top \\ &\stackrel{(3)}{=} \left(CAB + C^\top AB^\top \right)^\top \\ &= B^\top A^\top C^\top + BA^\top C \end{aligned} \tag{5}$$

From regular calculus, we know that $(d/dx)(x^2) = 2x$.

The matrix derivative trace analogue is $\nabla_A \text{tr}(AA^\top C) = CA + C^\top A$ which is similar to

$$\frac{d(a^2c)}{da} = 2ac$$

Optimisation

One notable resource that inspired the spine of this chapter is lectures 2–4, and 7 from the fantastic set of **lecture notes** by Amir Ali Ahmadi for the Princeton Spring 2016 Convex and Conic Optimisation course.

The presentation herein is unique enough in terminology and the development of theory that I include it here and don't simply point to another resource:

Definition B.0.1 An **optimisation problem** is of the form

$$\begin{aligned} &\text{extremise} && f(\mathbf{x}) \\ &\text{among} && \mathbf{x} \in \mathbb{R}^n \\ &\text{subject to} && \mathbf{x} \in \Omega \end{aligned} \tag{B.1}$$

where

- f is a scalar-valued function (from $E \subseteq \mathbb{R}^n$ to $\mathbb{R}$) called the **objective function**,
- The set $E \subseteq \mathbb{R}^n$ is called the **domain** of f . We may also denote the domain of f by

$$\mathbf{dom}(f) := \{\mathbf{x} : f(\mathbf{x}) \in \mathbb{R}\} \subseteq \mathbb{R}^n.$$

- $\mathbf{x}$ is the **decision variable**, and
- Ω is the **constraint** (or **feasible**) **set**.

The above form is often too abstract for most purposes, and the feasible region Ω is very often described by function constraints:

Definition B.0.2 The **function form** of the optimisation problem (B.1) is:

$$\begin{aligned} &\text{extremise} && f(\mathbf{x}) \\ &\text{subject to} && g_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \dots, m \\ &&& h_j(\mathbf{x}) = 0 \quad \text{for } j = 1, \dots, k \end{aligned} \tag{B.2}$$

where f , g_i and h_j are maps from their respective domains into $\mathbb{R}$.

Definition B.0.3 The **domain $\mathcal{D}$ of the optimisation problem (B.2)** is the set of all points where the objective f and constraint functions g_i , h_j are all defined:

$$\mathcal{D} = \mathbf{dom}(f) \cap \bigcap_{i=1}^n \mathbf{dom}(g_i) \cap \bigcap_{j=1}^k \mathbf{dom}(h_j).$$

Definition B.0.4

- The equations $g_i(\mathbf{x}) \leq 0$ are called inequality constraints.
- The equations $h_j(\mathbf{x}) = 0$ are called equality constraints.
- The optimisation problem (B.2) is said to be **unconstrained** if there are no constraints.

Definition B.0.5 Consider the general optimisation problem (B.1). A point $\mathbf{x}^* \in \Omega$ is called:

- a **local minimiser** for f in Ω , and its corresponding value $f(\mathbf{x}^*)$ a local minimum, if there exists some open set $U \subseteq \mathbb{R}^n$ about $\mathbf{x}^*$ s.t.

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \text{for all } \mathbf{x} \in (\Omega \cap U) \setminus \{\mathbf{x}^*\}$$

- a **local maximiser** for f in Ω , and its corresponding value $f(\mathbf{x}^*)$ a local maximum, if there exists some open set $U \subseteq \mathbb{R}^n$ about $\mathbf{x}^*$ s.t.

$$f(\mathbf{x}^*) \geq f(\mathbf{x}) \quad \text{for all } \mathbf{x} \in (\Omega \cap U) \setminus \{\mathbf{x}^*\}$$

- We use the term **local extremiser** to refer to a point $\mathbf{x}^*$ that is either a local minimiser or maximiser.
- If the inequalities become slack i.e. hold with $>$ instead of $\geq$, or $<$ instead of $\leq$, then $\mathbf{x}^*$ is a **strict local extremiser**, and the corresponding value $f(\mathbf{x}^*)$ is a **strict local extremum**.
- If any of the inequalities in the above definition hold with $U \supseteq \Omega$, replace ‘local’ with **global**.

There is a name given to the scenario in which an extremiser for f doesn’t exist but we’re still interested in the limiting largest/smallest value of f :

Definition B.0.6 The **optimal value** f^* of (B.1) is defined as

- the infimum $\inf_{\mathbf{x} \in \Omega} f(\mathbf{x})$ if the optimisation problem is to minimise f over Ω ,
- the supremum $\sup_{\mathbf{x} \in \Omega} f(\mathbf{x})$ if the optimisation problem is to maximise f over Ω .

If an optimal solution $\mathbf{x}^*$ exists¹, then the optimal value f^* is simply equal to $f(\mathbf{x}^*)$.

Example B.0.7 In general, the optimal value need not be attained. Consider the unconstrained problem of minimising $f(x) = e^{-x}$ over $\Omega = \mathbb{R}$. Observe that $f(x) > 0$ for all $x \in \mathbb{R}$. For this function, $f^* = 0$ despite not being achieved at any x . ►

Henceforth, we’ll focus on minimisation problems. The corresponding maximisation problem can be formulated by minimising $-f$.



B.1 First-Order Optimality Conditions (Unconstrained)

Definition B.1.1 Consider $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and a point $\mathbf{x} \in \mathbb{R}^n$. A direction $\mathbf{d} \in \mathbb{R}^n$ is called a **descent direction at $\mathbf{x}$** if $\exists \bar{\alpha} > 0$ s.t.

$$f(\mathbf{x} + \alpha \mathbf{d}) < f(\mathbf{x}) \quad \forall \alpha \in (0, \bar{\alpha}).$$

Lemma B.1.2 Consider a point $\mathbf{x} \in \mathbb{R}^n$ and a continuously differentiable f . Then any direction $\mathbf{d}$ that satisfies $(\nabla f(\mathbf{x}))^\top \mathbf{d} < 0$ is a descent direction. (In particular, $-\nabla f(\mathbf{x})$ is a descent direction if it’s non-zero.)

Remarks B.1.3

- In $\mathbb{R}^n$, the quantity $(\nabla f(\mathbf{x}))^\top \mathbf{d}$ is the coordinate representation of the abstract inner product $\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle$. The inner product measures the similarity of two vectors. Since the inner product in $\mathbb{R}^n$ may also be written as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta),$$

where θ is the angle between the two vectors, a negative inner product in $\mathbb{R}^n$ means that $\cos(\theta)$ is negative i.e. $\theta \in (\pi/2, 3\pi/2)$ so $\mathbf{x}$ points roughly in the opposite direction to $\mathbf{y}$.

¹An important case where $\mathbf{x}^*$ is guaranteed to exist is when f is continuous and Ω is a compact set (closed and bounded by Heine-Borel in $\mathbb{R}^n$) — the Extreme Value Theorem applies and the extrema (infimum and supremum) are attained.

- The proof of this lemma uses the Taylor expansion of a k times differentiable function **Definition A.1.10**:

Proof. I wish to show that any direction $\mathbf{d}$ that satisfies $(\nabla f(\mathbf{x}))^\top \mathbf{d} < 0$ is a descent direction i.e. that

$$\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle < 0 \implies \exists \bar{\alpha} > 0 \text{ s.t. } \forall \alpha \in (0, \bar{\alpha}) : f(\mathbf{x} + \alpha \mathbf{d}) < f(\mathbf{x}).$$

To do so, we'll obtain an expression that relates $f(\mathbf{x})$ and $f(\mathbf{x} + \alpha \mathbf{d})$ with a Taylor expansion. This will mean computing the derivative of a composition of f and a direction (by the chain rule):

Let $g: \mathbb{R} \rightarrow \mathbb{R}$ be defined as $g(\alpha) = f(\mathbf{x} + \alpha \mathbf{d})$, and denote by $h: \mathbb{R} \rightarrow \mathbb{R}^n$ the function $h(\alpha) = \mathbf{x} + \alpha \mathbf{d}$ so that $g = f \circ h$. Write

$$h(\alpha) = \begin{bmatrix} h_1(\alpha) \\ \vdots \\ h_n(\alpha) \end{bmatrix}.$$

By the chain rule in coordinates (**Corollary A.1.9**), the derivative of g is

$$g'(\alpha) = \sum_{k=1}^n \frac{\partial f}{\partial h_k}(h(\alpha)) \frac{dh_k}{d\alpha}(\alpha) = \nabla f(h(\alpha)) \cdot \underbrace{\begin{bmatrix} \frac{dh_1}{d\alpha}(\alpha) \\ \vdots \\ \frac{dh_n}{d\alpha}(\alpha) \end{bmatrix}}_{=: h'(\alpha)} = \nabla f(h(\alpha)) \cdot h'(\alpha)$$

Now note that $\frac{dh_j}{d\alpha}(\alpha) = \frac{d}{d\alpha}(x_j + \alpha d_j) = d_j$ so $h'(\alpha) = \mathbf{d}$, and so

$$\begin{aligned} g'(\alpha) &= (\nabla f(h(\alpha))) \cdot \mathbf{d} \\ &= \langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle. \end{aligned}$$

We may write the following Taylor expansion of g

$$g(\alpha) = g(0) + g'(0)\alpha + o(\alpha)$$

which is equivalent to

$$f(\mathbf{x} + \alpha \mathbf{d}) = f(\mathbf{x}) + \alpha \langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle + o(\alpha)$$

and we may re-write this as the following difference quotient

$$\frac{f(\mathbf{x} + \alpha \mathbf{d}) - f(\mathbf{x})}{\alpha} = \langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle + \frac{o(\alpha)}{\alpha}$$

for $\alpha > 0$.

Recall the following facts:

- What $l(\alpha) \in o(\alpha)$ means is that $\lim_{\alpha \rightarrow 0} \frac{l(\alpha)}{\alpha} = 0$.
- $\lim_{x \rightarrow a} f(x) = b$ means that $\forall \varepsilon > 0, \exists \delta_{\varepsilon, a} > 0$ s.t. $|x - a| < \delta_{\varepsilon, a} \implies |f(x) - b| < \varepsilon$.

For $\alpha > 0$:

$$\begin{aligned} l(\alpha) \in o(\alpha) &\iff \lim_{\alpha \rightarrow 0} \frac{l(\alpha)}{\alpha} = 0 \\ &\iff \forall \varepsilon > 0, \exists \overbrace{\delta_{\varepsilon, 0}}^{=: \bar{\alpha}} > 0 \text{ s.t. } \left(|\alpha - 0| < \delta_{\varepsilon, 0} \implies \left| \frac{l(\alpha)}{\alpha} - 0 \right| < \varepsilon \right) \\ &\iff \forall \varepsilon > 0, \exists \bar{\alpha} > 0 \text{ s.t. } \left(0 < \alpha < \bar{\alpha} \implies \frac{|l(\alpha)|}{\alpha} < \varepsilon \right). \end{aligned}$$

We assume that $\mathbf{d}$ satisfies $\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle < 0$, and so our equality

$$\frac{f(\mathbf{x} + \alpha \mathbf{d}) - f(\mathbf{x})}{\alpha} = \overbrace{\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle}^{< 0} + \underbrace{\frac{l(\alpha)}{\alpha}}$$

suggests that if we can bound the underbraced term from above in magnitude by $|\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle|$, then the RHS is less than 0 which implies that $f(\mathbf{x} + \alpha \mathbf{d}) < f(\mathbf{x})$ i.e. that $\mathbf{d}$ is a descent direction. Let $\varepsilon = |\langle \mathbf{d}, \nabla f(\mathbf{x}) \rangle|$. ■

Corollary B.1.4 (First-Order Necessary Condition for Optimality) If $\mathbf{x}^*$ is an unconstrained² local minimiser of a differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, then $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

This is a higher-dimensional analogue of Fermat's principle from real analysis: The points that give rise to (locally) extremal values of a differentiable function $f: U \rightarrow \mathbb{R}$ on an open set $U \subseteq \mathbb{R}$ are those with zero derivative $f'(x)$.

Proof. If $\nabla f(\bar{\mathbf{x}}) \neq \mathbf{0}$, then there exists some i s.t. $\frac{\partial f}{\partial x_i}(\bar{\mathbf{x}}) \neq 0$ in which case either $\mathbf{e}_i$ or $-\mathbf{e}_i$ is a descent direction. Hence, $\bar{\mathbf{x}}$ cannot be a local minimiser. ■

Definition B.1.5 Let $U \subseteq \mathbb{R}^n$ be an open set and $f: U \rightarrow \mathbb{R}$ be a Fréchet differentiable function. A point $\mathbf{x} \in U$ is called a **critical point** for f if $df(\mathbf{x}) = 0 \in \text{Hom}_{\mathbb{R}}(\mathbb{R}^n; \mathbb{R})$.

B.2 Second-Order Optimality Conditions

Definition B.2.1 An $n \times n$ complex matrix A is called:

- **positive semidefinite** if $A = \bar{A}^\top$ and $\bar{\mathbf{x}}^\top A \mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{C}^{n,1}$,
- **positive definite** if $A = \bar{A}^\top$ and $\bar{\mathbf{x}}^\top A \mathbf{x} > 0$ for all non-zero $\mathbf{x} \in \mathbb{C}^{n,1}$.

Theorem B.2.2 (Second-Order Necessary Condition for (Local) Optimality) If $\mathbf{x}^*$ is an unconstrained local minimiser of a twice continuously differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, then in addition to $\nabla f(\mathbf{x}^*) = \mathbf{0}$, we must have $\nabla^2 f(\mathbf{x}^*) \succeq 0$.

The Hessian of f at $\mathbf{x}^*$ being positive semidefinite is necessary for $\mathbf{x}^*$ being a local minimiser of f i.e.

$$\mathbf{x}^* \text{ being a local minimiser} \implies \nabla^2 f(\mathbf{x}^*) \succeq 0.$$

This is why it's called a *necessary* condition.

Proof. The proof is quite simple. Consider the second-order Taylor expansion of f around $\mathbf{x}^*$ (with $\alpha > 0$, $\mathbf{y} \in \mathbb{R}^n$):

$$f(\mathbf{x}^* + \alpha \mathbf{y}) = f(\mathbf{x}^*) + \alpha \mathbf{y}^\top \underbrace{\nabla f(\mathbf{x}^*)}_{=\mathbf{0}} + \frac{\alpha^2}{2} \mathbf{y}^\top (\nabla^2 f(\mathbf{x}^*)) \mathbf{y} + o(\alpha^2)$$

where the gradient of f at $\mathbf{x}^*$ is the zero vector because $\mathbf{x}^*$ is a local minimiser. We may re-arrange for:

$$\frac{f(\mathbf{x}^* + \alpha \mathbf{y}) - f(\mathbf{x}^*)}{\alpha^2} = \frac{1}{2} \mathbf{y}^\top (\nabla^2 f(\mathbf{x}^*)) \mathbf{y} + \frac{o(\alpha^2)}{\alpha^2},$$

the LHS of which is positive because for small enough α : $f(\mathbf{x}^*) \leq f(\mathbf{x}^* + \alpha \mathbf{y})$ in $\mathbb{B}_\varepsilon(\mathbf{x}^*) \cap \Omega$. This implies that

$$\lim_{\alpha \downarrow 0} \frac{1}{2} \mathbf{y}^\top (\nabla^2 f(\mathbf{x}^*)) \mathbf{y} + \frac{o(\alpha^2)}{\alpha^2} \geq 0$$

²Unconstrained means we minimise over all $\mathbf{x} \in \mathbb{R}^n$.

and since $\lim_{\alpha \downarrow 0} \frac{o(\alpha^2)}{\alpha^2} = 0$, and $\mathbf{y}$ is arbitrary, it follows that $\mathbf{y}^\top (\nabla^2 f(\mathbf{x}^*)) \mathbf{y} \geq 0$. All that remains is to show that the Hessian is symmetric. Since f is twice continuously differentiable, the mixed partials of f commute (this is the statement of Clairaut's theorem), so the Hessian H of f is a symmetric real matrix. Thus, the Hessian is positive semidefinite (written $\nabla^2 f(\mathbf{x}^*) \succeq 0$). ■

Theorem B.2.3 (Second-Order Sufficient Condition for Optimality) Let $f: \mathbb{R}^n$ be twice continuously differentiable. Suppose that there exists a point $\mathbf{x}^*$ such that $\nabla f(\mathbf{x}^*) = \mathbf{0}$, and that $\nabla^2 f(\mathbf{x}^*) \succ 0$. Then $\mathbf{x}^*$ is a strict local minimiser of f .

Proof. We'll again capitalise on the Taylor expansion of f , but this time it will be the second-order expansion, in order to deduce strict local minimality of $\mathbf{x}^*$:

$$\begin{aligned} f(\mathbf{x}^* + \mathbf{y}) - f(\mathbf{x}^*) &= \underbrace{\mathbf{y}^\top \nabla f(\mathbf{x}^*)}_{=0} + \frac{1}{2} \mathbf{y}^\top H \mathbf{y} + o(\|\mathbf{y}\|^2) \\ &= \frac{1}{2} \mathbf{y}^\top H \mathbf{y} + o(\|\mathbf{y}\|^2) \end{aligned}$$

We need some way to deal with the term that involves the Hessian $H = \nabla^2 f(\mathbf{x}^*)$ of f at $\mathbf{x}^*$:

Since f is twice continuously differentiable, the Hessian H of f is again real and symmetric matrix by Clairaut's theorem. This implies that the eigenvalues of H are real. Thus, they can be ordered. Let $\lambda_{\min}$ be the minimum eigenvalue of $H = \nabla^2 f(\mathbf{x}^*)$ i.e. there exists an eigenvector $\mathbf{v}_{\min}$ of H corresponding to $\lambda_{\min}$ satisfying

$$H \mathbf{v}_{\min} = \lambda_{\min} \mathbf{v}_{\min} \quad \text{i.e.} \quad (H - \lambda_{\min} \text{id}_{\mathbb{R}^n}) \mathbf{v}_{\min} = \mathbf{0}.$$

For any eigenvector $\mathbf{v}$ of H corresponding to an eigenvalue λ :

$$H \mathbf{v} - \lambda_{\min} \text{id}_{\mathbb{R}^n} \mathbf{v} = (\lambda - \lambda_{\min}) \mathbf{v}$$

i.e. $\mathbf{v}$ is an eigenvector of $(H - \lambda_{\min} \text{id}_{\mathbb{R}^n})$ with eigenvalue $\lambda - \lambda_{\min}$, the smallest eigenvalue of which is zero. All eigenvalues of $(H - \lambda_{\min} \text{id}_{\mathbb{R}^n})$ are non-negative which is equivalent to $(H - \lambda_{\min} \text{id}_{\mathbb{R}^n})$ being positive semidefinite.

The positive semidefiniteness tells us that

$$\mathbf{y}^\top H \mathbf{y} \geq \lambda_{\min} \|\mathbf{y}\|^2 \quad \text{for every } \mathbf{y} \in \mathbb{R}^{n,1}.$$

Now we return to the Taylor expansion:

$$\begin{aligned} f(\mathbf{x}^* + \mathbf{y}) - f(\mathbf{x}^*) &= \frac{1}{2} \mathbf{y}^\top H \mathbf{y} + o(\|\mathbf{y}\|^2) \\ &\geq \frac{1}{2} \lambda_{\min} \|\mathbf{y}\|^2 + o(\|\mathbf{y}\|^2) \\ &= \|\mathbf{y}\|^2 \left(\frac{\lambda_{\min}}{2} + \underbrace{\frac{o(\|\mathbf{y}\|^2)}{\|\mathbf{y}\|^2}} \right) \end{aligned}$$

For the underbraced term, we can run a similar argument to the previous proof i.e. its limit as $\|\mathbf{y}\| \downarrow$ being equal to 0 means that for all $\varepsilon > 0$, there exists some $\delta_{\varepsilon,0} > 0$ s.t.

$$\|\mathbf{y}\| < \delta_{\varepsilon,0} \implies \left| \frac{o(\|\mathbf{y}\|^2)}{\|\mathbf{y}\|^2} - 0 \right| < \varepsilon.$$

Let $\varepsilon = \lambda_{\min}/2 > 0$. Then we can conclude that $f(\mathbf{x}^* + \mathbf{y}) > f(\mathbf{x}^*)$ for all $\mathbf{y}$ with $\|\mathbf{y}\| < \delta_{\varepsilon,0}$ which is the definition of $\mathbf{x}^*$ being a strict local minimiser of f . ■

B.3 Upgrading Local to Global Solutions

So far, we've seen optimality conditions that almost³ characterise *locally* optimal solutions. Ideally, we'd like to make statements about global solutions. This comes at the expense of imposing additional structure on f and Ω . By and large, the most successful structural property that we know of to achieve this goal is convexity. Convex functions are a broad enough class of nice enough functions with few enough minimisers that are somewhat easy to locate.

Definition B.3.1 A subset $\Omega \subseteq \mathbb{R}^n$ is called **convex** if for all $\mathbf{x}, \mathbf{y} \in \Omega$ and for all $t \in [0, 1]$, $\mathbf{x} + t(\mathbf{y} - \mathbf{x}) \in \Omega$.

Such a point $\mathbf{x} + t(\mathbf{y} - \mathbf{x})$, also written $(1 - t)\mathbf{x} + t\mathbf{y}$, is called a **convex combination** of $\mathbf{x}$ and $\mathbf{y}$.

Definition B.3.2 A function $f: \Omega \rightarrow \mathbb{R}$ is called a **convex function** if its domain Ω is a convex set, and for all $\mathbf{x}, \mathbf{y} \in \Omega$ and for all $t \in [0, 1]$, we have

$$f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) \leq f(\mathbf{x}) + t(f(\mathbf{y}) - f(\mathbf{x})).$$

This is also written as $f((1 - t)\mathbf{x} + t\mathbf{y}) \leq (1 - t)f(\mathbf{x}) + tf(\mathbf{y})$.

Theorem B.3.3 Let $f: \Omega \rightarrow \mathbb{R}$ be a convex function.

- (i) Every local minimiser for f in Ω is a global minimiser.
- (ii) The collection of global minimisers for f in Ω is a convex set.

Proof. Let $\mathbf{x}^*$ be a local minimiser for f in Ω i.e. there exists some open set $U \subseteq \mathbb{R}^n$ s.t. $\mathbf{x} \in (\Omega \cap U) \setminus \{\mathbf{x}^*\} \implies f(\mathbf{x}^*) \leq f(\mathbf{x})$. Suppose for the sake of a contradiction that there exists some $\mathbf{z} \in \Omega$ with $f(\mathbf{z}) < f(\mathbf{x}^*)$. By the convexity of Ω , we have $\mathbf{x}^* + t(\mathbf{z} - \mathbf{x}^*) \in \Omega$ for every $t \in [0, 1]$, and by the convexity of f the following equality holds for all $t \in [0, 1]$:

$$\begin{aligned} f(\mathbf{x}^* + t(\mathbf{z} - \mathbf{x}^*)) &= f(t\mathbf{z} + (1 - t)\mathbf{x}^*) \\ &\leq tf(\mathbf{z}) + (1 - t)f(\mathbf{x}^*) \\ &< tf(\mathbf{x}^*) + (1 - t)f(\mathbf{x}^*) \\ &= f(\mathbf{x}^*). \end{aligned}$$

Note that $\mathbf{x}^* + t(\mathbf{z} - \mathbf{x}^*) \xrightarrow{t \rightarrow 0} \mathbf{x}^*$ and so $f(\mathbf{x}^*) < f(\mathbf{x}^*)$ which is a contradiction. Thus, $\mathbf{x}^*$ is a global minimiser for f in Ω . ■

Definition B.3.4 A **convex optimisation problem** is any optimisation problem of the form:

$$\begin{aligned} &\text{minimise} && f(\mathbf{x}) \\ &\text{subject to} && \mathbf{x} \in \Omega \end{aligned} \tag{B.3}$$

where Ω is a convex set.

Remarks B.3.5

- Some authors also require that Ω is closed. If one does, it's usually to avoid a scenario like $\min_{x > 0} x$ where the infimum is not attained at any point in $\Omega = (0, +\infty)$.
- As in the general case, we can also describe the same optimisation problem in function form:

³We did not have a necessary and sufficient condition.

Definition B.3.6 The **function form** of the convex optimisation problem (B.3) is:

$$\begin{aligned} & \text{minimise} && f(\mathbf{x}) \\ & \text{subject to} && g_i(\mathbf{x}) \leq 0 && \text{for } i = 1, \dots, m \\ & && h_j(\mathbf{x}) = 0 && \text{for } j = 1, \dots, k \end{aligned} \tag{B.4}$$

where $f, g_i: \mathbb{R}^n \rightarrow \mathbb{R}$ are convex functions, and $h_j: \mathbb{R}^n \rightarrow \mathbb{R}$ are affine functions.

Remarks B.3.7

- Each constraint $h_j(\mathbf{x}) = 0$ defines a flat hyperplane (or affine subspace) that cuts the space flatly without bending it.
- Each constraint $g_i(\mathbf{x}) \leq 0$ carves out a convex region bounded by a convex surface. One can informally think of these as bulging boundaries that never dent inwards.

Putting the above two together, the feasible set

$$\Omega = \left(\bigcap_i \{g_i \leq 0\} \right) \cap \left(\bigcap_j \{h_j = 0\} \right)$$

is formed by taking several convex, outward-bulging walls, h_j as perfectly flat slices with planes, and Ω is what survives after those walls and cuts are applied. Since any finite intersection of convex sets is also convex, Ω is convex.

Justify why these sets are convex.

B.4 Characterisations of Convex Functions

B.4.1 CONVEXITY ALONG ALL LINES

Theorem B.4.1 A function $f: \Omega \rightarrow \mathbb{R}$ is convex if and only if the function $g: \mathbb{R} \rightarrow \mathbb{R}$ given by $g(t) = f(\mathbf{x} + t\mathbf{y})$ is convex (as a univariate function) for all $\mathbf{x} \in \Omega$ and $\mathbf{y} \in \mathbb{R}^n$.

Note that the domain of g is the collection of t for which $\mathbf{x} + t\mathbf{y} \in \Omega$.

B.4.2 FIRST AND SECOND-ORDER CHARACTERISATIONS

Theorem B.4.2 Let $\Omega \subseteq \mathbb{R}^n$ be convex and open. Suppose that $f: \Omega \rightarrow \mathbb{R}$ is twice continuously differentiable. Then the following are equivalent:

- f is convex
- $f(\mathbf{y}) \geq f(\mathbf{x}) + (\nabla f(\mathbf{x}))^\top (\mathbf{y} - \mathbf{x})$ for all $\mathbf{x}, \mathbf{y} \in \Omega$.
- $(\nabla^2 f(\mathbf{x})) \succeq 0$ for all $\mathbf{x} \in \Omega$.

Remarks B.4.3

- **Theorem B.4.2 (ii)** says that the first-order Taylor expansion at any point in Ω is a global under-estimator of the function.
- **Theorem B.4.2 (iii)** says that the function has “non-negative curvature” everywhere. (In one dimension in particular, $f''(x) \geq 0$ for all $x \in \Omega$.)

Corollary B.4.4 Consider an unconstrained optimisation problem

$$\begin{aligned} & \text{minimise} && f(\mathbf{x}) \\ & \text{subject to} && \mathbf{x} \in \mathbb{R}^n \end{aligned}$$

where f is convex and differentiable. Then any point $\mathbf{x}^*$ that satisfies $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is a global minimiser.

Proof. From the first-order characterisation of convexity, we have

$$f(\mathbf{y}) \geq f(\mathbf{x}) + (\nabla f(\mathbf{x}))^\top (\mathbf{y} - \mathbf{x}) \quad \text{for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^n.$$

In particular,

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \underbrace{(\nabla f(\mathbf{x}^*))^\top}_{=\mathbf{0}} (\mathbf{y} - \mathbf{x}^*) = f(\mathbf{x}^*) \quad \text{for all } \mathbf{y} \in \mathbb{R}^n.$$

Thus, $\mathbf{x}^*$ is a global minimiser of f . ■

Remarks B.4.5

- $\mathbf{x}$ being a critical point of f is always a necessary condition for local optimality in an unconstrained problem i.e. $\mathbf{x}$ being a local extremiser $\implies \nabla f(\mathbf{x}) = \mathbf{0}$.
 - **Corollary B.4.4** states that for convex problems, $\nabla f(\mathbf{x}) = \mathbf{0} \implies$ local and global optimality.
- In the absence of convexity, $\nabla f(\mathbf{x}) = \mathbf{0}$ is not sufficient even for local optimality e.g. $f(x) = x^3$ and $x^* = 0$.
- Another necessary condition for (unconstrained) local optimality of a point $\mathbf{x}$ was $\nabla^2 f(\mathbf{x}) \succeq 0$. Note that a convex function automatically passes this test because of the equivalence of **Theorem B.4.2 (i)** and **(iii)**.

B.5 Strictly Convex Functions

Definition B.5.1 Let $\Omega \subseteq \mathbb{R}^n$ be a convex set. We call $f: \Omega \rightarrow \mathbb{R}$ **strictly convex** if whenever $\mathbf{x}$ and $\mathbf{y}$ are distinct, and $t \in (0, 1)$:

$$f((1-t)\mathbf{x} + t\mathbf{y}) < (1-t)f(\mathbf{x}) + tf(\mathbf{y}).$$

One of the uses of strict convexity is to ensure uniqueness of the extremiser.

Theorem B.5.2 Consider the optimisation problem

$$\begin{aligned} & \text{minimise} && f(\mathbf{x}) \\ & \text{subject to} && \mathbf{x} \in \Omega \end{aligned}$$

where Ω is a convex set, and $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is strictly convex on Ω . Then f has at most one global minimiser in Ω .

Proof. Suppose that there are two extremisers $\mathbf{x}, \mathbf{y} \in \Omega$ i.e. $f(\mathbf{x}) = f(\mathbf{y}) \leq f(\mathbf{z})$ for all $\mathbf{z} \in \Omega$. Consider $\mathbf{z} = (1-t)\mathbf{x} + t\mathbf{y}$ for $0 < t < 1$. Since Ω is convex, $\mathbf{z} \in \Omega$. By the strict convexity of f

$$\begin{aligned} f(\mathbf{z}) &= f((1-t)\mathbf{x} + t\mathbf{y}) < (1-t)f(\mathbf{x}) + tf(\mathbf{y}) \\ &= (1-t)f(\mathbf{x}) + tf(\mathbf{x}) \\ &= f(\mathbf{x}) \end{aligned}$$

which contradicts $f(\mathbf{x}) \leq f(\mathbf{z})$ for all $\mathbf{z} \in \Omega$. ■

Definition B.5.3 A function $f: \Omega \rightarrow \mathbb{R}$ is **strongly convex** if $\exists \alpha > 0$ s.t. $f(\mathbf{x}) - \alpha\|\mathbf{x}\|^2$ is convex.

Lemma B.5.4 Strong convexity $\implies$ strict convexity $\implies$ convexity.

The converse of neither implication is true.

B.5.1 CHARACTERISATION OF STRICT CONVEXITY

Theorem B.5.5 (Second-Order Sufficient Condition for Strict Convexity) If $\nabla^2 f(\mathbf{x}) \succ 0$ for all $\mathbf{x} \in \Omega$, then f is strictly convex on Ω .

Theorem B.5.6 (First-Order Characterisation) A function f is strictly convex on $\Omega \subseteq \mathbb{R}^n$ if and only if

$$f(\mathbf{y}) > f(\mathbf{x}) + (\nabla f(\mathbf{x}))^\top (\mathbf{y} - \mathbf{x}) \quad \text{for all } \mathbf{x}, \mathbf{y} \in \Omega \text{ with } \mathbf{x} \neq \mathbf{y}.$$

B.6 Lagrange Multipliers

Recall the general form (B.1) of an optimisation problem from the beginning of this chapter, and now suppose that the constraint set Ω is defined by some function $g: \mathbb{R}^n \rightarrow \mathbb{R}^m$ as follows:

$$\begin{aligned} &\text{extremise} && f(\mathbf{x}) \\ &\text{among} && \mathbf{x} \in \mathbb{R}^n \\ &\text{subject to} && g(\mathbf{x}) \in E \subseteq \mathbb{R}^m \end{aligned} \tag{B.5}$$

B.6.1 EQUALITY CONSTRAINTS

In particular, we'll let $E = \{0\} \subseteq \mathbb{R}^1$ and focus on maximising our objective function:

$$\begin{aligned} &\text{maximise} && f(\mathbf{x}) \\ &\text{among} && \mathbf{x} \in \mathbb{R}^n \\ &\text{subject to} && g(\mathbf{x}) = 0 \end{aligned} \tag{B.6}$$

The constraint defines a surface⁴ $S := g^{-1}(\{0\}) = \{\mathbf{x} \in \mathbb{R}^n : g(\mathbf{x}) = 0\}$. Suppose that $\mathbf{x}^* \in S$ is a local maximiser of f . Let $\gamma: (-\varepsilon, \varepsilon) \rightarrow \mathbb{R}^n$ be a short curve on S through $\mathbf{x}^* = \gamma(0)$ i.e. γ is smooth and $\gamma'(0) = \mathbf{v}$ is tangent to S at $\mathbf{x}^*$.

- Since $g(\gamma(t)) \in S$ for all $t \in (-\varepsilon, \varepsilon)$, we know that $g(\gamma(t)) = 0$ for all $t \in (-\varepsilon, \varepsilon)$. By the chain rule of differentiation,

$$0 = \frac{d(g \circ \gamma)}{dt}(t) \Big|_{t=0} = (\nabla g(\gamma(0))) \cdot \gamma'(0) = (\nabla g(\mathbf{x}^*)) \cdot \mathbf{v}$$

⁴We call $g^{-1}(\{c\})$ the **level set** of g at the constant value c . In our case, $c = 0$.

i.e. $\nabla g(\mathbf{x}^*) \perp \mathbf{v}$ as visualised below:

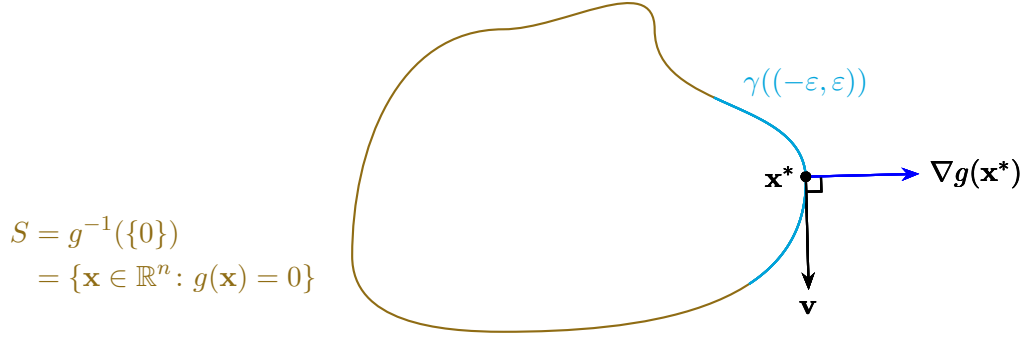


Figure B.1: The gradient is orthogonal to the level set.

- Also, since $\mathbf{x}^*$ is a local maximiser for f , $\nabla f(\mathbf{x}^*) = \mathbf{0}_{\mathbb{R}^n}$. Thus, by the chain rule of differentiation again:

$$\left. \frac{d(f \circ \gamma)}{dt}(t) \right|_{t=0} = \nabla f(\gamma(0)) \cdot \gamma'(0) = \underbrace{\nabla f(\mathbf{x}^*)}_{=\mathbf{0}_{\mathbb{R}^n}} \cdot \mathbf{v} = 0$$

i.e. $\nabla f(\mathbf{x}^*) \perp \mathbf{v}$.

Since $\nabla f(\mathbf{x})$ is the direction of greatest increase of f , the relationship $\nabla f(\mathbf{x}^*) \perp \mathbf{v}$ says that $\nabla f(\mathbf{x}^*)$ does not have any component in the direction of $\mathbf{v}$ tangent to S at $\mathbf{x}^*$ i.e. moving along S in any direction from $\mathbf{x}^*$ will not serve to increase f .

At a local maximiser, you can't improve f by moving along the surface S — if ∇f had a non-zero component in the direction of S , one could simply move $\mathbf{x}^*$ in that direction to increase f , contradiction the optimality of $\mathbf{x}^*$.

From these two facts, we conclude that $\nabla_{\mathbf{x}} f(\mathbf{x}^*) \parallel \nabla_{\mathbf{x}} g(\mathbf{x}^*)$ i.e.

$$\exists \lambda \in \mathbb{R} \quad \text{s.t.} \quad \nabla_{\mathbf{x}} f(\mathbf{x}^*) = \lambda \nabla_{\mathbf{x}} g(\mathbf{x}^*).$$

This λ is called a **Lagrange multiplier**. If we re-arrange this expression into

$$\begin{aligned} 0 &= \nabla_{\mathbf{x}} f(\mathbf{x}^*) - \lambda \nabla_{\mathbf{x}} g(\mathbf{x}^*) \\ &= \nabla_{\mathbf{x}} (f(\mathbf{x}^*) - \lambda g(\mathbf{x}^*)) \quad \text{by linearity.} \end{aligned}$$

Define the **Lagrangian functional** by

$$L(\mathbf{x}, \lambda) := f(\mathbf{x}) - \lambda g(\mathbf{x}).$$

Thus, we've re-cast our optimisation problem (B.6) into a constrained critical point problem of finding the critical points w.r.t. $\mathbf{x}$ of $L(\mathbf{x}, \lambda)$ under the constraint $g(\mathbf{x}) = 0$. We make one final observation that

$$\frac{\partial}{\partial \lambda} L(\mathbf{x}, \lambda) = -g(\mathbf{x})$$

and so

$$\frac{\partial}{\partial \lambda} L(\mathbf{x}, \lambda) = 0 \iff g(\mathbf{x}) = 0.$$

We can collect both statements $\nabla_{\mathbf{x}} L(\mathbf{x}, \lambda) = 0$ and $\partial_{\lambda} L(\mathbf{x}, \lambda) = 0$ into a single gradient equation:

$$\nabla L(\mathbf{x}, \lambda) = \mathbf{0}.$$

This concludes the re-casting of our constrained optimisation problem into an unconstrained critical point problem. In summary, one may say that we've removed the constraints of our original problem at the cost of adding a penalty to our objective function.

This method generalises naturally to multiple constraints i.e. when g maps into some subset $E \subseteq \mathbb{R}^m$. We collect it into a single theorem.

Example B.6.1 Say $E = \{\mathbf{y}\}$. Then our constraints are $g_i(\mathbf{x}) = y_i$ for $i = 1, \dots, m$. ►

Theorem B.6.2 Let U be an open subset of $\mathbb{R}^n$, and $f: U \rightarrow \mathbb{R}$, $g: U \rightarrow \mathbb{R}^m$ be continuously differentiable. Suppose that $\mathbf{x}^*$ is a local extremiser of f in $g^{-1}(\{\mathbf{y}\})$ for some $\mathbf{y} \in \mathbb{R}^m$. Then there exists a bounded linear functional $\lambda: \mathbb{R}^m \rightarrow \mathbb{R}$ s.t.

$$df(\mathbf{x}^*) = \lambda \circ dg(\mathbf{x}^*).$$

Equivalently, if $\mathbf{x}^*$ is a constrained local extremiser of f , then it's an unconstrained critical point of the map $L: U \times (\mathbb{R}^m)^* \rightarrow \mathbb{R}$ defined by

$$L(\mathbf{x}, \lambda) := f(\mathbf{x}) - \lambda(g(\mathbf{x})).$$

Bibliography

- [1] Khallil Ebrahim Benyattou. *Conditional Exkebtations*. Self-Published, 2026.
<https://kbenyattou.github.io/assets/notes/>.
- [2] Mark John Schervish. *Theory of Statistics*. Springer Science & Business Media, 2012.
- [3] Fumio Hayashi. *Econometrics*. Princeton University Press, 2000.
- [4] Khallil Ebrahim Benyattou. *Linear Alkebra*. Self-Published, 2026.
<https://kbenyattou.github.io/assets/notes/>.
- [5] Frank Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*, volume 55. Spartan Books Washinton, DC, 1962.
- [6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.